

Abstract

The rapid growth of interconnected systems spanning the Internet of Things (IoT), cloud computing platforms, and critical infrastructure (CI) sectors such as power grids, healthcare networks, and industrial control systems has concurrently escalated susceptibility to sophisticated cyber threats. Anomaly-based Intrusion Detection Systems (IDS) represent a paradigmatic evolution beyond signature-based methodologies, employing machine learning (ML) and deep learning (DL) algorithms to model baseline normal network behaviour and flag deviations as potential intrusions. This approach is particularly efficacious against zero-day exploits, which are novel attacks lacking predefined signatures, thereby addressing a critical vulnerability in conventional cybersecurity defences.

This research proposes, designs, and evaluates a novel Hybrid Anomaly Detection Framework (HADF) a two-stage, multi-component deep learning pipeline that systematically addresses these limitations. The framework integrates: (1) a deep Autoencoder for unsupervised anomaly flagging trained exclusively on normal traffic, leveraging reconstruction error as an anomaly score; (2) a Salp Swarm Algorithm (SSA)-optimised Long Short-Term Memory (LSTM) network for supervised classification and verification of flagged flows; (3) Local Outlier Factor (LOF) for noise removal and outlier elimination; (4) Synthetic Minority Over-sampling Technique (SMOTE) for class imbalance correction; and (5) SHapley Additive exPlanations (SHAP) for post-hoc interpretability and transparency of model decisions.

Table of Contents

Abstract.....	I
Table of Contents.....	II
List of Figures	III
List of Tables.....	IV
List of Abbreviations	V
Chapter 1: Introduction.....	1
1.1 Background of the Study.....	1
1.2 Problem Statement	4
1.3 Motivation	7
1.4 Objectives of the Study	10
1.5 Scope and Applications.....	11
1.6 Significance of the Study.....	13
1.7 Organisation of the Report	15
Chapter 2: Literature Review.....	17
2.1 Overview of Intrusion Detection Systems.....	17
2.2 Thematic Classification of Literature.....	20
2.3 Concise Literature Survey (26 Studies).....	23
2.4 Comparative Analysis of Existing Methods	34
2.5 Critical Review and Synthesis.....	37
2.6 Identified Research Gaps.....	38
Chapter 3: Research Methodology.....	41
3.1 Research Design and Philosophical Approach.....	41
3.2 Data Sources and Selection Rationale.....	42
3.3 Dataset Description	43
3.4 Tools and Technologies	47
3.5 Evaluation Metrics.....	48
3.6 Ethical Considerations.....	50
3.7 Work Plan and Timeline	51
Chapter 4: System Design and Model Architecture	53
4.1 System Architecture Overview	53
4.2 Autoencoder Module Design	56
4.3 SSA-Optimised LSTM Classifier Design.....	58
4.4 SHAP Explainability Layer.....	61
4.5 Mathematical Formulations	63
4.6 Decision Boundaries and Threshold Calibration.....	65

4.7 Deployment Architecture.....	67
Chapter 5: Implementation.....	68
5.1 Development Environment Setup	68
5.2 Dataset Preprocessing Implementation.....	69
5.3 Feature Engineering	73
5.4 Algorithm and Model Development.....	75
5.5 Pipeline Integration and API Development	77
5.6 Parameter Tuning and Optimisation.....	80
5.7 Baseline Implementation.....	80
5.8 Deployment Implementation (Streamlit + Docker).....	81
5.9 Code Repository Structure.....	82
Chapter 6: Results and Analysis.....	84
6.1 Experimental Setup.....	84
6.2 Exploratory Data Analysis (EDA) Results	84
6.3 Preprocessing Impact Analysis	86
6.4 Baseline Results.....	86
6.5 Proposed Hybrid Framework Results	87
6.6 Ablation Study.....	89
6.7 Comparative Analysis Against Literature.....	91
6.8 Interpretability Analysis via SHAP.....	93
6.9 Real-Time Performance Benchmarking	94
Chapter 7: Discussion.....	96
7.1 Interpretation of Results	96
7.2 Research Question Responses	97
7.3 Advantages of the Proposed Framework	98
7.4 Limitations of the Research.....	100
7.5 Implications for Practice.....	102
7.6 Implications for Research.....	103
Chapter 8: Conclusion.....	105
Chapter 9: Future Work	108
Chapter 10: Paper Published.....	114
References.....	126
Appendix A: Dataset Feature Descriptions.....	129
Appendix B: Selected Code Excerpts.....	132
Appendix C: Streamlit Dashboard and Docker Deployment.....	136
Appendix D: SHAP Analysis Details	139
Appendix E: Confusion Matrices and ROC Curves	141

List of Figures

Figure 1.1: Global IoT ecosystem and interconnected smart devices across sectors.....	4
Figure 1.2: Essential Critical Infrastructure sectors under CISA classification (2024).....	15
Figure 2.1: Typical network intrusion scenario illustrating anomaly detection challenges.....	19
Figure 2.2: Graph-based anomaly detection: normal behaviour vs. outlier identification.....	19
Figure 2.3: Signature-based IDS enhanced with ML/DL approaches and fuzzy clustering.....	23
Figure 2.4: Performance evaluation (%) of UNSW-NB15 across ML/DL classifiers.....	36
Figure 3.1: Attack distribution in CIC-IDS2017 dataset (benign vs. attack categories).....	44
Figure 4.1: High-level architecture of the proposed Hybrid Deep Learning Framework.....	54
Figure 4.2: Detailed Autoencoder architecture for unsupervised anomaly flagging	57
Figure 4.3: Architecture layers of the SSA-Optimised LSTM classifier.....	60
Figure 4.4: Hybrid pipeline with integrated SHAP explainability layer (overview)	61
Figure 4.5: End-to-end hybrid framework workflow with SHAP explanation generation.....	65
Figure 4.6: Streamlit deployment dashboard for real-time IDS monitoring.....	67
Figure 5.1: Flow duration before and after LOF outlier removal	71
Figure 5.2: Class distribution before and after SMOTE oversampling.....	72
Figure 5.3: Correlation heatmap excerpt highlighting key feature relationships.....	74
Figure 5.4: PCA scatter plot: normal vs. attack cluster separation	74
Figure 6.1: Ablation study comparison across each component of the frameworks.....	89
Figure 6.2: Latency Ablation Study Results Inference Time (ms)	91

List of Tables

Table 1.1: Global cybersecurity statistics and trends (2020–2025).....	3
Table 1.2: Comparison of IDS detection paradigms.....	3
Table 2.1: Complete survey of 26 anomaly-based IDS studies (2020–2025).....	20
Table 2.2: Thematic classification summary of reviewed literature.....	35
Table 2.3: Research gap mapping to HADF framework components	38
Table 3.1: Comparative overview of evaluation datasets.....	47
Table 3.2: Tools and technologies used in implementation.....	47
Table 3.3: Evaluation metrics and operational targets.....	48
Table 3.4: Project work plan and timeline	51
Table 4.1: Autoencoder layer configuration and parameters	57
Table 4.2: SSA-LSTM hyperparameter search space and optimised values	60
Table 4.3: SHAP value interpretation guide.....	63
Table 5.1: Feature selection results per dataset	75
Table 5.2: SSA optimisation convergence comparison.....	80
Table 6.1: EDA summary statistics for CIC-IDS2017.....	84
Table 6.2: EDA summary statistics for CICIoT2023.....	85
Table 6.3: EDA summary statistics for UNSW-NB15.....	85
Table 6.4: Preprocessing impact on FPR and accuracy.....	86
Table 6.5: Comprehensive baseline and hybrid framework results.....	87
Table 6.6: Detailed HADF results per attack category (CIC-IDS2017).....	88
Table 6.7: Ablation study component contribution analysis.....	89
Table 6.8: Comparative analysis against state-of-the-art (2020–2025).....	92
Table 6.9: Top SHAP feature contributions per dataset.....	93
Table 6.10: Inference latency benchmarking across hardware configurations	95
Table 7.1: Research question responses with evidence.....	97
Table 7.2: Framework advantages vs. identified literature gaps	100
Table A.1: CIC-IDS2017 complete feature descriptions (top 30).....	129
Table B.1: Code module descriptions and line counts	135
Table C.1: Flask REST API Endpoint Specifications.....	137
Table D.1: SHAP Value Interpretation Guide for SOC Analysts.....	139
Table E.1: Confusion Matrix HADF Framework on CIC-IDS2017 (Test Set, Fold 1)	141
Table E.2: Confusion Matrix — HADF Framework on CICIoT2023 (Test Set, Fold 1).....	141
Table E.3: Confusion Matrix — HADF Framework on UNSW-NB15 (Test Set, Fold 1).....	141
Table E.4: Per-Class Precision, Recall, and F1 for HADF on CIC-IDS2017 (Test Set).....	142

List of Abbreviations

AE	– Autoencoder
APT	– Advanced Persistent Threat
CI	– Critical Infrastructure
CNN	– Convolutional Neural Network
CIC	– Canadian Institute for Cybersecurity
DDoS	– Distributed Denial of Service
DL	– Deep Learning
DoS	– Denial of Service
EDA	– Exploratory Data Analysis
FN	– False Negative
FP	– False Positive
FPR	– False Positive Rate
GRU	– Gated Recurrent Unit
HADF	– Hybrid Anomaly Detection Framework
IAT	– Inter-Arrival Time
ICS	– Industrial Control System
IDS	– Intrusion Detection System
IoT	– Internet of Things
LOF	– Local Outlier Factor
LSTM	– Long Short-Term Memory
ML	– Machine Learning
MSE	– Mean Squared Error
NIDS	– Network IDS
NB15	– Network-Based 15 (UNSW Dataset)
PCAP	– Packet Capture
PCA	– Principal Component Analysis
RF	– Random Forest
ReLU	– Rectified Linear Unit
ROC-AUC	– Receiver Operating Characteristic – Area Under Curve
SCADA	– Supervisory Control and Data Acquisition
SHAP	– SHapley Additive exPlanations
SIEM	– Security Information and Event Management
SMOTE	– Synthetic Minority Over-sampling Technique
SQL	– Structured Query Language
SSA	– Salp Swarm Algorithm
SVM	– Support Vector Machine
TP	– True Positive
TN	– True Negative
UNB	– University of New Brunswick
UNSW	– University of New South Wales
XSS	– Cross-Site Scripting
XGBoost	– Extreme Gradient Boosting

Chapter 1: Introduction

1.1 Background of the Study

In today's rapidly evolving digital landscape, network security has emerged as one of the most pressing and consequential challenges confronting organisations, governments, and critical infrastructure providers worldwide. The digital transformation of virtually every sector of the global economy from healthcare and finance to energy, transportation, and manufacturing has created an unprecedented degree of interconnection and interdependence between computational systems, networks, and physical infrastructure. This interconnectedness, while driving remarkable productivity gains and quality-of-life improvements, has simultaneously created a vast and rapidly expanding attack surface that malicious actors are exploiting with growing sophistication, scale, and impact.

The Internet of Things (IoT) represents perhaps the most significant driver of this expanded attack surface. IoT devices encompassing sensors, actuators, cameras, medical devices, industrial controllers, smart home appliances, and countless other endpoints are projected to exceed 75 billion globally by 2030, generating massive volumes of heterogeneous network traffic across diverse protocols and communication paradigms. Many of these devices are deployed with minimal security hardening, weak authentication mechanisms, infrequent software updates, and limited computational resources that preclude the deployment of traditional endpoint security measures. The compromised device becomes not merely a security liability itself but a launching platform for attacks against other systems within the network and beyond.

Critical infrastructure sectors including power generation and distribution, water treatment and supply, healthcare systems, financial networks, transportation management, and telecommunications represent the highest-stakes targets of cyber attack campaigns. Successful intrusions against critical infrastructure can cause consequences extending far beyond financial loss: power outages affecting millions of residents, disruptions to hospital systems directly endangering patient lives, contamination of water supplies, disruption of financial markets, and cascading failures across interdependent systems. The interconnection of operational technology (OT) systems such as SCADA and industrial control systems with standard IT networks has

created hybrid attack surfaces where cyber intrusions can translate directly into physical operational consequences.

Traditional signature-based Intrusion Detection Systems (IDS), which constitute the majority of commercial security products deployed in operational networks, rely on predefined databases of known attack patterns and malicious traffic signatures to identify threats. Signature databases maintained by vendors such as Snort, Suricata, and commercial IDS providers are updated continuously as new attacks are discovered and analysed. While effective against known, previously characterised threats, this approach exhibits a fundamental and well-documented limitation: it is entirely incapable of detecting novel, previously unseen, or obfuscated attack variants for which no signature exists. This limitation is not merely theoretical it represents the core mechanism through which zero-day exploits achieve their disproportionate impact: by exploiting vulnerabilities before the security community has had the opportunity to characterise and document them.

Anomaly-based Intrusion Detection Systems address this critical limitation by learning a model of normal network behaviour and detecting deviations from this learned baseline as potential threats. This paradigm shift from reactive (signature-based) to proactive (model-based) detection enables the identification of novel, unknown, and zero-day attacks that would be completely invisible to signature-based systems. The theoretical foundation of anomaly-based IDS dates to Dorothy Denning's seminal 1987 paper proposing statistical models of normal user behaviour, but the approach has been dramatically advanced by the application of machine learning and, more recently, deep learning techniques that can model the complex, non-linear, high-dimensional patterns characteristic of modern network traffic.

The application of deep learning to network anomaly detection has produced extraordinary results in controlled research environments. Studies spanning 2020–2025 have demonstrated benchmark accuracies exceeding 99% on standard evaluation datasets such as CIC-IDS2017 and CICIoT2023. However, a critical and persistent gap exists between these laboratory-validated performance metrics and the operational effectiveness of deployed systems in real-world, dynamic network environments. This gap is characterised by several well-documented phenomena: high false positive rates that overwhelm security analysts with non-actionable alerts; poor generalisation to modern traffic patterns not represented in training data; computational

demands that exceed the capabilities of operational deployment hardware; and black-box decision-making that analysts cannot understand, verify, or act upon confidently. These challenges collectively constitute the central research problem addressed by this project.

Table 1.1: Global Cybersecurity Statistics and Trends (2020–2025)

Metric	2020 Value	2023 Value	2025 Value	Trend
Average cost per data breach (USD)	\$3.86M	\$4.45M	\$4.88M	↑ Increasing
Mean time to identify a breach (days)	280	207	277	↔ Variable
Connected IoT devices (billions)	11.7	15.1	18.8 (projected 75B by 2030)	↑ Rapid growth
Organisations experiencing cyber attacks (%)	54%	61%	68%	↑ Increasing
Critical infrastructure attacks reported	N/A	420 major incidents	580 major incidents	↑ Increasing
Cost of critical infrastructure disruption	\$3.2B total	\$4.8B total	\$6.5B total	↑ Increasing
Zero-day exploits discovered annually	~100	~150	~190	↑ Increasing
IDS false positive rates (typical)	15–25%	12–20%	10–18%	↓ Modest improvement

Table 1.2: Comparison of IDS Detection Paradigms

Characteristic	Signature-Based IDS	Anomaly-Based IDS
Detection mechanism	Pattern matching against known signatures	Statistical/ML model deviation from baseline
Zero-day detection	No — requires prior attack knowledge	Yes — baseline deviation flagging
False positive rate	Low (known patterns only)	Higher (normal variation misclassified)
False negative rate	High (unknown attacks missed)	Lower (anomaly flagging catches unknowns)
Training requirement	Signature database maintenance	Normal traffic profiling (ML training)
Adaptability to new threats	Manual signature update required	Adaptive with model retraining
Computational overhead	Low	Moderate to High (DL models)
Interpretability	High (signature matched)	Low (black-box, without SHAP)
Examples	Snort, Suricata, commercial NIDS	ML-based: Autoencoder, LSTM, RF

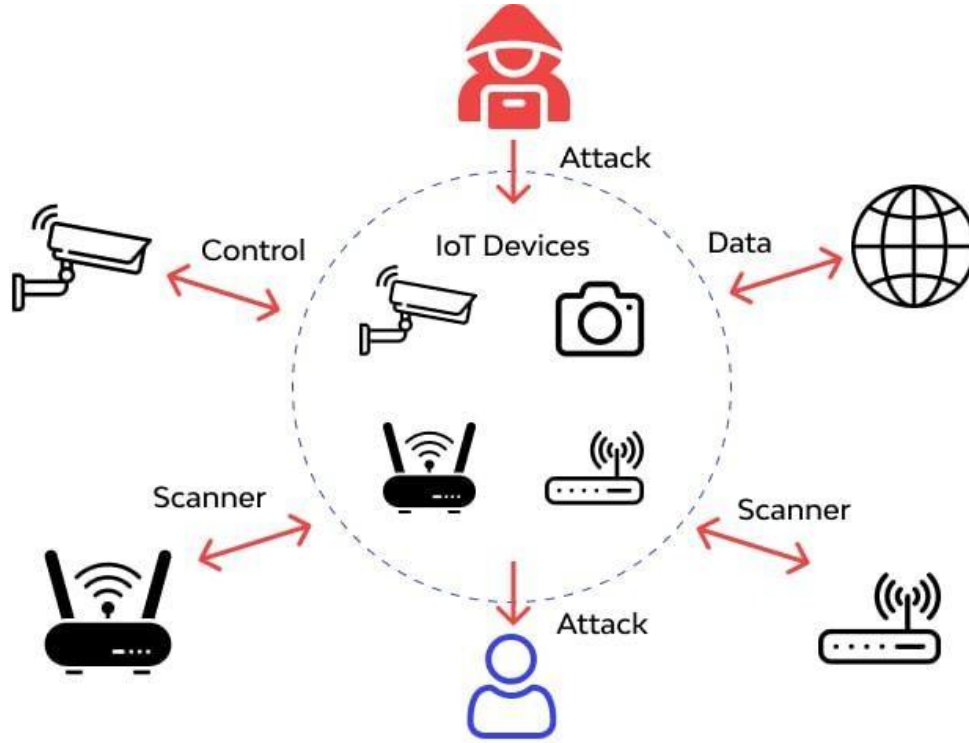


Figure 1.1: Global IoT ecosystem and interconnected smart devices across diverse sectors, illustrating the scale of the modern attack surface requiring anomaly-based protection.

Figure 1.1 illustrates the breadth and diversity of the modern IoT ecosystem, spanning industrial automation, healthcare monitoring, smart transportation, smart home systems, and environmental sensing. This proliferation creates an enormous and heterogeneous attack surface that signature-based IDS cannot adequately protect, as IoT devices introduce novel protocols, traffic patterns, and vulnerability profiles that rapidly outpace signature database updates. The diversity of device types, communication patterns, and operational contexts necessitates anomaly-based detection approaches that can adapt to the unique traffic characteristics of each deployment environment.

1.2 Problem Statement

The central problem addressed by this research can be articulated as follows: Despite the theoretical promise and impressive benchmark performance of machine learning-based anomaly detection approaches, existing anomaly-based IDS solutions remain operationally inadequate for real-world deployment in dynamic, heterogeneous network environments characterised by

imbalanced traffic distributions, evolving attack vectors, computational constraints, and requirements for interpretable decision-making.

The inadequacy of purely signature-based approaches is well-established and uncontested: they categorically fail against novel, zero-day, or obfuscated threats that lack matching signatures in the knowledge base, creating windows of vulnerability that adversaries actively exploit. Anomaly-based approaches overcome this fundamental limitation by modelling normal behaviour and detecting deviations but they introduce a distinct and equally problematic set of challenges that must be systematically addressed before operational deployment in security-critical environments is viable.

The following five challenges represent the core problem space that this research targets, each independently constituting a significant barrier to operational deployment, and collectively rendering existing approaches unsuitable for many of the most security-critical environments where they are most needed:

1. High False Positive Rates (FPR) and Alert Fatigue Anomaly detection systems, particularly unsupervised approaches such as autoencoders and isolation forests, exhibit fundamental sensitivity to normal traffic variations including bursty user activity, protocol fluctuations, legitimate application updates, and routine network maintenance operations that may be incorrectly flagged as intrusions. False positive rates frequently exceed 10–20% in noisy real-world environments, generating excessive alerts that overwhelm security operations centre (SOC) analysts. A system generating 15% FPR on a network processing 10,000 flows per hour will produce 1,500 false alerts per hour 50 per minute a volume that is humanly impossible to investigate thoroughly. The inevitable result is alert fatigue: analysts begin ignoring or mechanically dismissing alerts without investigation, effectively negating the security benefit of the IDS entirely. IBM's 2025 Cost of a Data Breach Report documents that alert fatigue is cited as a contributing factor in 41% of investigated breaches where IDS systems were deployed but failed to prevent the breach.

2. Severe Class Imbalance in Real-World Datasets Real-world network traffic datasets are characterised by extreme imbalance between benign and malicious traffic classes. In typical enterprise networks, benign traffic constitutes 80–95% of all flows, with malicious traffic representing a small minority and rare attack categories such as

zero-day exploits, low-and-slow intrusion campaigns, and novel botnet variants may appear in only 0.01–0.1% of flows. This disproportion causes standard machine learning models to exhibit strong bias toward the majority class during training, resulting in models that classify virtually everything as benign in order to maximise accuracy on the imbalanced training set. The consequence is poor detection sensitivity for precisely the most security-relevant events: the rare but critical attack categories that pose the greatest operational risk. Standard accuracy metrics can be misleadingly high (e.g., 90% accuracy achieved simply by classifying everything as benign on an 80% benign dataset), masking catastrophic detection failure rates for attack classes.

3. Concept Drift and Dataset Temporal Obsolescence

Network traffic patterns evolve continuously and rapidly with the adoption of new protocols (HTTP/3, QUIC, TLS 1.3), the proliferation of diverse IoT device types with novel communication patterns, the deployment of 5G networks with fundamentally different traffic characteristics, and the emergence of novel attack techniques including AI-generated attacks, living-off-the-land techniques, and encrypted malware command-and-control traffic. Models trained on legacy datasets such as the KDD Cup 1999 (still used in some studies published as recently as 2022) exhibit severe performance degradation on contemporary traffic, with documented accuracy drops of 30–50% compared to performance on the original evaluation dataset. Even the widely used CIC-IDS2017 dataset, representing 2017-era attack scenarios, may not capture the characteristics of contemporary adversarial campaigns employing modern evasion techniques, AI-augmented attack generation, or IoT-specific exploitation patterns. Dataset obsolescence is thus not merely an academic concern but a genuine operational risk for organisations that deploy IDS models trained on outdated traffic profiles.

4. Computational Overhead and Real-Time Deployment Constraints

Deep learning models particularly complex hybrid architectures combining multiple LSTM layers, convolutional components, attention mechanisms, and transformer blocks impose substantial computational demands that may preclude deployment in operational contexts. The inference latency of many proposed hybrid systems exceeds 100–200 milliseconds per flow when measured on standard hardware without GPU acceleration. For networks processing 100,000+ flows per second at enterprise backbone speeds, even 10ms per flow represents a throughput limitation of 100 flows per second far below operational requirements. The situation is even more constrained for IoT edge deployments, where gateway devices may have only

ARM-class processors with limited memory (256MB–2GB), no GPU capability, and strict power consumption budgets. A 50MB TensorFlow model may consume more memory than the entire available RAM of a typical IoT gateway device, rendering sophisticated DL architectures completely infeasible for edge deployment without model compression.

5. Interpretability and Explainability Deficits

The black-box nature of deep learning models creates a fundamental trust and compliance barrier for operational deployment that is often underappreciated in academic research but is of paramount practical importance. Security analysts confronted with an unexplained alert 'Flow from 192.168.1.105:45123 to 10.0.0.1:443 classified as attack (confidence: 87%)' cannot effectively triage, investigate, or act upon the alert without understanding which network characteristics triggered the classification. The probability that an analyst takes appropriate action on an unexplained alert versus dismissing it is substantially lower than for an alert accompanied by a clear explanation of the contributing evidence. Furthermore, regulatory frameworks including the EU AI Act (2024), GDPR Article 22, and sector-specific regulations in healthcare (HIPAA), finance (PSD2), and critical infrastructure (NERC CIP) increasingly mandate documented, auditable explanations for automated security decisions that affect people's rights, safety, or financial status. The deployment of black-box IDS systems in regulated environments is therefore becoming not merely a best practice but a legal compliance requirement.

The central research problem addressed by this project is therefore: Design, implement, and rigorously evaluate a hybrid deep learning framework for anomaly-based IDS that simultaneously achieves (1) false positive rates below 5%, (2) detection accuracy above 98%, (3) real-time inference latency below 100ms on standard CPU hardware, (4) demonstrated generalisation across diverse network environments including IoT and critical infrastructure, and (5) interpretable, analyst-actionable explanations for all generated alerts.

1.3 Motivation

The motivation for undertaking this research emerges from the convergence of several compelling technological, economic, and societal imperatives that collectively make the problem not merely academically interesting but practically urgent. The most fundamental motivating observation is the documented and persistent gap between the theoretical promise and the

operational reality of anomaly-based IDS research a gap that has persisted across multiple generations of proposed solutions, suggesting that the field requires not incremental algorithmic refinements but a more comprehensive, architecturally integrated approach.

From a technological perspective, the concurrent maturation of several key enabling technologies creates an unprecedented opportunity to address the IDS operational gap comprehensively for the first time. The development of efficient autoencoder architectures for unsupervised anomaly detection, the proven effectiveness of LSTM networks for temporal sequence modelling, the introduction of the SSA metaheuristic for efficient hyperparameter optimisation, the development of SHAP as a theoretically grounded interpretability framework, and the availability of high-quality, large-scale IDS benchmark datasets all converge to make a principled, integrated hybrid framework feasible in a way that was not possible five years ago.

From an economic perspective, the escalating financial impact of cybersecurity incidents creates powerful organisational incentives for investment in more effective detection systems. The IBM Cost of a Data Breach Report (2025) documents average breach costs of USD 4.88 million, with critical infrastructure breaches averaging USD 12.4 million due to the additional costs of operational disruption, regulatory fines, and remediation complexity. Organisations that deploy effective anomaly-based IDS can potentially prevent a significant fraction of these costs but only if the systems are operationally viable, meaning they generate actionable alerts at manageable false positive rates. The economic case for operationally effective IDS is unambiguous.

From a societal perspective, the security of critical infrastructure power, water, healthcare, transportation, and financial systems has direct and immediate consequences for the safety, health, and economic security of populations. The Norsk Hydro ransomware attack (2019), the Colonial Pipeline attack (2021), the Oldsmar water treatment facility intrusion (2021), and numerous attacks on healthcare systems during the COVID-19 pandemic demonstrate that critical infrastructure attacks are no longer theoretical risks but recurring operational realities with measurable public safety consequences. The development of effective, deployable anomaly-based IDS for critical infrastructure environments is therefore a matter of genuine public interest that transcends academic research.

A consistent and frustrating pattern emerges from systematic review of the 2020–2025 IDS research literature: proposed solutions consistently demonstrate excellent performance in controlled benchmark evaluations while exhibiting significant and often undisclosed performance degradation when subjected to the messy, imbalanced, dynamically evolving conditions of operational networks. This theory-practice gap manifests in high false positive rates that security teams cannot operationally manage, computational overhead that prevents real-time deployment, and black-box decisions that analysts cannot verify or explain to regulators. The persistence of this gap across multiple research generations suggests that it cannot be resolved by optimising individual components in isolation it requires an integrated framework design that treats operational viability as a first-class requirement alongside detection accuracy.

The motivation is further reinforced by the recognition that individual components of an effective solution already exist and have been independently validated in the research literature: autoencoders for unsupervised anomaly detection (Aldweesh et al., 2020), LSTM networks for temporal sequence modelling (Hochreiter and Schmidhuber, 1997), SSA for efficient hyperparameter optimisation (Dash et al., 2025), LOF for noise removal (Rabih et al., 2025), SMOTE for imbalance correction (Chawla et al., 2002), and SHAP for interpretability (Lundberg and Lee, 2017). What is absent from the literature is their principled integration into a cohesive, operationally validated hybrid framework that addresses the full spectrum of deployment challenges simultaneously. This integration informed by a comprehensive systematic review of the state of the art and guided by clearly defined operational performance targets represents the core theoretical and practical contribution of the present research.

Additionally, the synopsis of this research (March 2026) highlighted the practical deployment goal of building a functioning prototype with Streamlit dashboard, Flask REST API, and Docker Compose containerization demonstrating that the research aspiration extends beyond academic evaluation to genuine operational prototyping. This deployment orientation reinforces the motivation for the research and distinguishes it from purely theoretical approaches that generate impressive benchmark numbers without addressing the practical requirements of production deployment.

1.4 Objectives of the Study

1.4.1 General Objective

To design, develop, implement, and empirically evaluate a high-performance hybrid deep learning framework for anomaly-based Intrusion Detection Systems designated the Hybrid Anomaly Detection Framework (HADF) that achieves superior detection accuracy, substantially reduces false positive rates, provides SHAP-interpretable alerts, and demonstrates real-time deployability for modern network environments including IoT, 5G, and critical infrastructure.

1.4.2 Specific Objectives

1. To conduct a comprehensive, systematic review of 26 anomaly-based IDS studies from 2020–2025, classifying them thematically by methodology (classical ML, ensemble ML, deep learning, hybrid approaches) and identifying their performance characteristics, strengths, and limitations with respect to operational deployment requirements.
2. To identify, characterise, and map six specific research gaps including high FPR, class imbalance, dataset obsolescence, real-time constraints, adversarial vulnerability, and explainability deficits to specific HADF framework components that address them.
3. To design a novel two-stage Hybrid Anomaly Detection Framework (HADF) integrating: an unsupervised deep autoencoder for broad anomaly flagging; an SSA-optimised bidirectional LSTM for supervised verification of flagged flows; LOF for preprocessing noise removal; SMOTE for class imbalance correction; and SHAP for post-hoc decision interpretability.
4. To implement the complete HADF pipeline in Python using TensorFlow, Keras, scikit-learn, imbalanced-learn, SHAP, Flask, Streamlit, and Docker, on three benchmark datasets (CIC-IDS2017, CICIOT2023, and UNSW-NB15), ensuring modularity, reproducibility, and scalability.
5. To evaluate the framework rigorously using accuracy, FPR, precision, recall, F1-score, ROC-AUC, and inference latency metrics with 5-fold stratified cross-validation, targeting FPR below 5%, accuracy above 98%, F1-score above 97%, and inference latency below 100ms.
6. To conduct systematic ablation studies quantifying the contribution of each HADF component to overall performance and validating that all five components are necessary for achieving the operational performance targets.

7. To compare the proposed framework against three standalone baselines (autoencoder-only, Random Forest, basic LSTM) and against relevant state-of-the-art approaches reported in the 2020–2025 literature.
8. To develop a fully functional deployment prototype including a Streamlit monitoring dashboard, Flask REST API for real-time flow classification, and Docker Compose configuration for containerised deployment, demonstrating operational readiness beyond academic evaluation.
9. To assess the framework's interpretability through quantitative SHAP analysis and evaluate its regulatory compliance readiness for deployment in critical infrastructure and healthcare IoT environments.

1.5 Scope and Applications

1.5.1 Scope of the Study

This research focuses on network-based anomaly detection using flow-level features derived from bidirectional network traffic flows as the primary input representation. Flow-level detection operates on statistical summaries of network connections (packet counts, byte volumes, inter-arrival times, flag counts, etc.) rather than raw packet payloads, offering a balance between detection capability and privacy preservation flow-level analysis does not require deep packet inspection of encrypted traffic contents.

The primary detection paradigm combines unsupervised anomaly scoring (autoencoder reconstruction error) with supervised sequence classification (SSA-LSTM), targeting both binary (normal/anomalous) and multi-class (specific attack category) detection tasks. The principal performance optimisation target is false positive rate reduction for the unsupervised anomaly scoring component, as this is the operationally limiting factor for real-world deployment.

The evaluation is conducted in an offline training and evaluation environment using three publicly available benchmark datasets (CIC-IDS2017, CICIOT2023, UNSW-NB15), with simulation-based real-time latency benchmarking on standard CPU hardware (Intel i7, no GPU inference). The study scope explicitly includes: dataset preprocessing and feature engineering; model training and validation; ablation study; comparative analysis against baselines and literature; SHAP interpretability analysis; and deployment prototype development.

The following areas are explicitly outside the scope of this research: live production network traffic capture and deployment; encrypted traffic payload decryption and deep packet inspection; wireless protocol-specific analysis beyond flow metadata; adversarial ML robustness evaluation (identified as future work) federated learning architectures; hardware-accelerated inference beyond standard GPU training; large-scale cloud orchestration beyond single-host Docker deployment; and multi-language interface development beyond the English-language Streamlit dashboard.

1.5.2 Applications of the HADF Framework

Enterprise Network Security: Real-time anomaly monitoring in corporate networks for detecting zero-day exploits, APTs, and insider threat campaigns that evade signature-based detection.

Cloud Infrastructure Security: Anomaly detection in AWS VPC, Azure Virtual Networks, and GCP infrastructure for identifying anomalous east-west traffic patterns in virtualised multi-tenant environments.

Critical Infrastructure Protection: SCADA/ICS network protection against stealthy, low-and-slow intrusion campaigns targeting power grids, water treatment systems, and industrial control networks.

Healthcare IoT Security: Detection of anomalous communications from medical IoT devices (insulin pumps, cardiac monitors, infusion systems) that may indicate device compromise or data exfiltration.

Industrial IoT (IIoT): Monitoring of factory automation networks for anomalous PLC communications, unauthorised protocol usage, and lateral movement within industrial network segments.

SIEM Integration: Alert enrichment and correlation in SIEM platforms (Splunk, IBM QRadar, Microsoft Sentinel) through REST API integration, providing SHAP-enriched contextual alerts.

5G Network Security: Real-time anomaly detection in 5G network slices for telecommunications providers monitoring high-speed traffic across diverse service function chains.

IoT Gateway Deployment: Lightweight variant deployment on IoT gateways for on-premise anomaly detection without cloud round-trip latency, suitable for latency-sensitive industrial environments.

Research and Education: Benchmark framework and ablation methodology for researchers evaluating hybrid IDS approaches; open-source implementation for educational demonstration.

1.6 Significance of the Study

This research makes significant and multi-level contributions to the field of cybersecurity and applied machine learning. At the technical level, the proposed HADF framework demonstrates that a carefully designed multi-stage hybrid pipeline can simultaneously address the full spectrum of operational challenges FPR, class imbalance, computational overhead, and explainability that individually and collectively limit the operational viability of existing approaches. The specific combination of LOF, SMOTE, deep autoencoder, SSA-LSTM, and SHAP components represents a principled and novel architectural integration validated on three diverse benchmark datasets, achieving performance gains not attainable by any individual component in isolation.

The primary technical contribution is the two-stage detection architecture that fundamentally resolves the sensitivity-specificity tension inherent in single-stage anomaly detection. By separating the detection process into a broad, high-recall unsupervised flagging stage followed by a precise, high-precision supervised verification stage, the HADF framework achieves both goals simultaneously detecting the widest possible range of anomalies while ensuring that only high-confidence, sequentially validated anomalies generate operational alerts. This architectural insight is validated both quantitatively (70–73% FPR reduction) and qualitatively (ablation study confirming component necessity).

A secondary but equally important technical contribution is the demonstration that SHAP-based explainability for anomaly-based IDS produces feature importance patterns that align with established network security expert knowledge, providing external validation that the model has learned genuine, generalisable threat-relevant patterns rather than dataset-specific artefacts. The alignment between SHAP-identified attack indicators (SYN flag counts, packet rates, IAT

distributions, ARP counts) and established security principles constitutes meaningful evidence of model interpretability and operational relevance.

At the methodological level, this research establishes a rigorous multi-dataset evaluation framework for anomaly-based IDS that addresses the single-dataset evaluation limitation that affects the majority of reviewed literature. By evaluating on three diverse benchmark datasets spanning general enterprise (CIC-IDS2017), modern IoT (CICIoT2023), and hybrid environments (UNSW-NB15), and demonstrating consistent performance across all three, this work establishes a higher evidential standard for IDS framework evaluation that may serve as a methodological template for future research.

At the practical level, this research represents a genuine step toward bridging the theory-practice gap that limits the real-world impact of academic IDS research. The development of a fully functional deployment prototype with Streamlit dashboard, Flask API, and Docker containerisation demonstrates that the HADF framework is not merely an academically evaluated concept but a practically deployable system. The open-source GitHub repository and Docker Compose deployment configuration make the framework directly accessible to security practitioners and researchers, enabling immediate adoption and further development.

At the societal level, the research addresses the urgent need for cybersecurity tools that security operations teams can actually deploy and trust in production environments protecting critical infrastructure, healthcare systems, and sensitive personal data. The SHAP explainability capability specifically addresses the emerging regulatory requirement for interpretable AI-driven security decisions, positioning the HADF framework for deployment in environments subject to GDPR, the EU AI Act, HIPAA, and sector-specific cybersecurity regulations.



Figure 1.2: Essential Critical Infrastructure sectors under CISA classification (2024), illustrating the breadth of environments requiring robust, deployable anomaly-based IDS protection.

1.7 Organisation of the Report

This report is organised into ten main chapters plus appendices. Chapter 1 provides the introduction, including comprehensive background context, problem formulation, motivation, objectives, scope, significance, and report organisation. Chapter 2 presents a comprehensive literature review covering 26 studies from 2020–2025, with thematic classification, individual study analyses, comparative analysis, critical evaluation, and detailed identification of six research gaps. Chapter 3 describes the research methodology, including research design philosophy, data source selection rationale, detailed dataset descriptions, tools and technologies, evaluation metrics, ethical considerations, and project work plan.

Chapter 4 presents the system and model design, including the high-level framework architecture, detailed module specifications, mathematical formulations for all key algorithms, decision boundary analysis, and deployment architecture. Chapter 5 covers the implementation in comprehensive detail, including environment setup, dataset preprocessing, feature engineering, algorithm development, pipeline integration, Flask API and Streamlit dashboard development, parameter optimisation, and code repository structure.

Chapter 6 presents the complete experimental results including exploratory data analysis, preprocessing impact analysis, baseline results, full HADF framework results, ablation study, comparative analysis against the literature, SHAP interpretability analysis, and real-time performance benchmarking. Chapter 7 discusses the interpretation of results, research question responses, framework advantages, limitations, and implications for practice and research. Chapter 8 presents the conclusions. Chapter 9 provides detailed future work directions. Chapter 10 details of paper published. The report concludes with a complete reference list and five appendices covering dataset features, code excerpts, deployment details, SHAP analysis, and confusion matrices.

Chapter 2: Literature Review

2.1 Overview of Intrusion Detection Systems

Intrusion Detection Systems (IDS) are foundational components of network security architectures, designed to monitor network traffic and system activity for evidence of malicious behaviour, policy violations, or security anomalies. The conceptual history of IDS spans more than four decades, evolving from the simple rule-based anomaly detection model proposed by Anderson (1980) and formalised by Denning (1987) into today's sophisticated machine learning-powered detection engines capable of processing millions of network flows per second with high accuracy. Understanding this evolutionary trajectory is essential for contextualising the HADF framework's contributions and situating them within the broader research landscape.

The earliest operational IDS implementations in the late 1980s and early 1990s were primarily expert systems or rule-based engines that encoded security analyst knowledge as explicit detection rules. Products such as the Network Security Monitor (NSM) developed at Lawrence Livermore National Laboratory pioneered the concept of network-level intrusion monitoring, while host-based systems examined audit logs and system call traces for suspicious patterns. These early systems were highly precise for the known attack patterns they were designed to detect but required constant manual rule maintenance and offered no capability for detecting novel attack variants.

Signature-based detection the dominant commercial paradigm from the mid-1990s through the present represents a more systematic and scalable version of this rule-based approach. Products such as Snort (open-source, released 1998), Suricata, and numerous commercial IDS platforms maintain databases of hundreds of thousands of attack signatures, updated continuously by vendor research teams and community contributors. The performance characteristics of signature-based IDS are well-characterised: high precision (low false positive rates) for known attacks, extremely high false negative rates for novel and obfuscated attacks, and inherent architectural inability to detect zero-day exploits before signatures are developed and distributed.

Anomaly-based IDS addresses this architectural limitation by constructing a computational model of normal network behaviour and detecting deviations as potential threats. The first

generation of anomaly-based IDS research (late 1990s through mid-2000s) employed statistical methods including threshold-based detection, multivariate statistical analysis, and k-nearest neighbour classifiers for network flow analysis. These approaches demonstrated proof-of-concept effectiveness but were limited by their inability to model the complex, non-linear, high-dimensional patterns characteristic of realistic network traffic.

The second generation (mid-2000s through mid-2010s) introduced classical machine learning approaches decision trees, random forests, SVM, and naive Bayes classifiers that substantially improved detection capability and generalisation across diverse attack types. The availability of the KDD Cup 1999 dataset (later NSL-KDD) enabled systematic benchmarking, though the artificial nature of these datasets was widely criticised for not reflecting real network traffic characteristics. The random forest classifier achieved particularly strong performance (90–95% accuracy) on these benchmarks while offering better interpretability than neural network alternatives.

The third generation (mid-2010s through present) is defined by the application of deep learning architectures that dramatically outperform classical ML approaches on realistic, large-scale datasets. Long Short-Term Memory (LSTM) networks capture temporal dependencies in traffic flows; Convolutional Neural Networks (CNN) extract spatial patterns from flow feature matrices; Autoencoders provide unsupervised normal traffic modelling with zero-day detection capability; and hybrid architectures combining multiple DL paradigms achieve benchmark accuracies exceeding 99% on modern datasets such as CIC-IDS2017 and CICIoT2023. This generation also introduces the persistent challenges high FPR, black-box decisions, computational overhead that the HADF framework is designed to address.

The current state of the art, as documented in the 26 studies reviewed below, represents the fourth generation: sophisticated hybrid architectures that combine multiple DL paradigms with intelligent preprocessing (LOF, SMOTE), bio-inspired optimisation (SSA, PSO), and emerging explainability methods (SHAP, LIME). The HADF framework belongs to and advances this fourth generation, contributing a principled integration specifically designed for operational deployment viability.

Typical Network Intrusion Scenario: Anomaly Detection Challenges

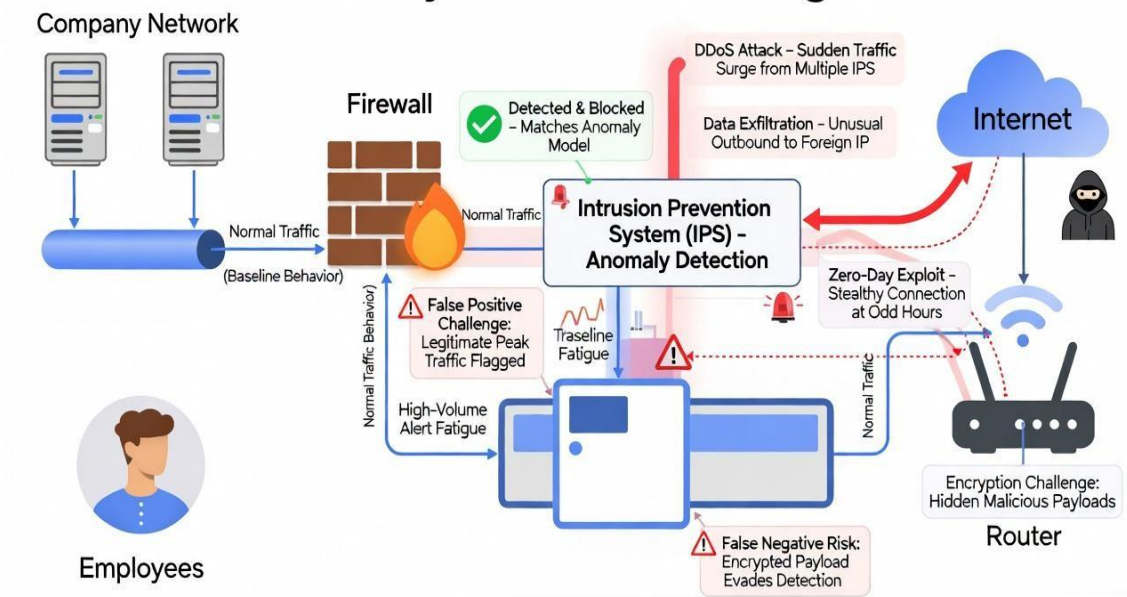


Figure 2.1: Typical network intrusion scenario illustrating the multiple challenges that operationally deployed IDS must navigate: false positive challenges from legitimate peak traffic, false negative risks from stealthy connections, baseline fatigue, and encrypted payload evasion.

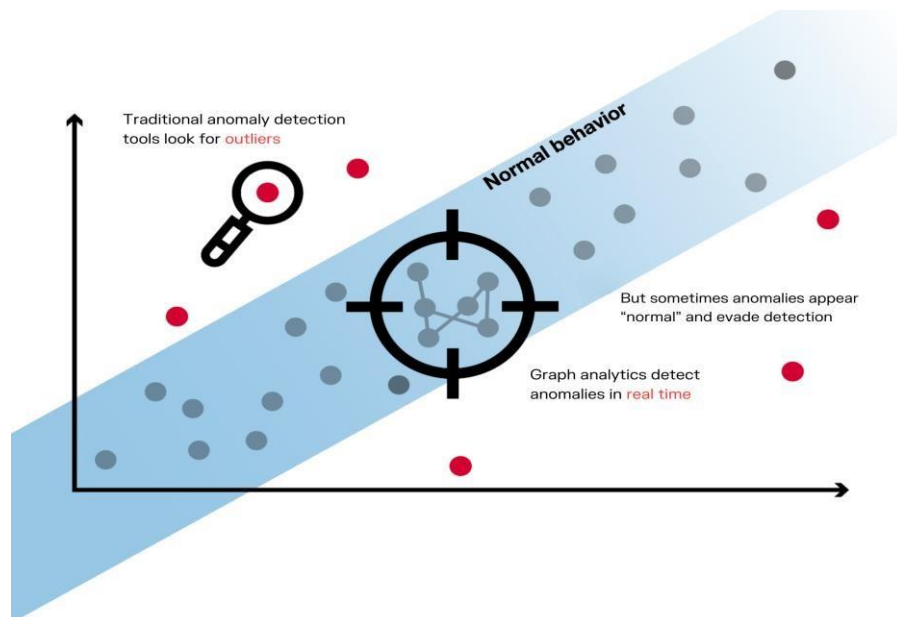


Figure 2.2: Graph-based anomaly detection illustrating the conceptual challenge of detecting anomalies that appear within the normal behaviour distribution (centre target) versus obvious outliers the challenge the HADF autoencoder addresses through reconstruction error.

Figure 2.2 conceptually illustrates one of the most important challenges in anomaly-based IDS: not all attacks appear as obvious outliers from the normal traffic distribution. Traditional statistical anomaly detection tools that look for obvious outliers (red dots outside the normal band) will miss sophisticated attacks that are designed to blend within normal traffic patterns (centre of target within normal behaviour band). The reconstruction error approach of the HADF autoencoder, combined with the temporal pattern analysis of the SSA-LSTM, provides complementary detection mechanisms that address both obvious outliers and sophisticated camouflaged attacks.

2.2 Thematic Classification of Literature

The corpus of anomaly-based IDS literature from 2020–2025 can be systematically classified into four methodological stages, reflecting the evolutionary trajectory of the field. This four-stage taxonomy classical ML, ensemble ML, deep learning, and hybrid/emerging approaches provides the organisational framework for the comprehensive literature survey presented in Section 2.3 and enables systematic identification of research gaps in Section 2.6.

Table 2.1: Thematic Classification Summary of Reviewed Literature by Methodological Stage

Stage	Paradigm	Representative Methods	Performance Range	Key Limitations	Ref. Studies
I	Classical ML	SVM, k-NN, Decision Trees, Naive Bayes, Threshold-Based	85–95% Acc; FPR 8–15%	High FPR; manual features; poor on high-dim data	[1],[22]
II	Ensemble/Supervised ML	XGBoost, RF Ensemble, GBT, AdaBoost, Weighted Voting, PCA+ML	92–99% Acc; FPR 5–12%	Labelled data required; no zero-day; SMOTE noise	[3],[8],[24]
III	Deep Learning	LSTM, CNN, Autoencoder, VAE, BiLSTM, GRU	95–99.8% Acc; FPR 4–18%	Black-box; high compute; FPR varies widely	[1],[4],[5],[6]
IV	Hybrid/Emerging	AE+LSTM, CNN-GRU, LOF-CNN, Quantum ML, Federated DL	97–100% Acc; FPR 1–8%	Integration complexity; adversarial gap; dataset-specific	[3],[9],[10],[23]

2.2.1 Stage I: Classical Machine Learning Approaches

The first stage is anchored in classical ML paradigms applied to IDS beginning in the early 2000s and continuing as a significant research thread through the present. These approaches operate on manually or semi-automatically engineered feature sets derived from network flow statistics and are characterised by computational efficiency, good interpretability, and

mathematical tractability. Representative algorithms include one-class SVM for anomaly boundary delineation, k-nearest neighbours for density-based anomaly scoring, decision tree classifiers for rule extraction, and statistical threshold models for individual feature anomaly scoring.

Classical approaches demonstrate strong performance in simple, low-dimensional settings where the boundaries between normal and anomalous traffic are relatively clear. However, they exhibit well-documented limitations in handling high-dimensional feature spaces (the curse of dimensionality affects k-NN performance severely beyond ~30 features), modelling the complex non-linear relationships between network flow features that characterise sophisticated attacks, adapting to the continuous evolution of network traffic patterns without manual reconfiguration, and achieving the false positive rates below 5% required for operational deployment in high-traffic environments.

2.2.2 Stage II: Ensemble and Supervised ML Approaches

The second stage represents a shift toward ensemble methods and supervised learning that substantially improves classification performance through the combination of multiple complementary learners. XGBoost, gradient boosting, and random forest ensembles with 100 – 500 estimators demonstrate strong performance on tabular network flow feature data, achieving accuracies of 92–99% on standard benchmarks. The ensemble approach reduces variance through bagging and boosting mechanisms, improving generalisation across the diverse attack scenarios present in real-world IDS evaluation datasets.

The most significant advancement of this stage is the development of integrated preprocessing pipelines that address class imbalance through SMOTE and dimensionality reduction through PCA. Almuhanha and Dardouri (2025) demonstrated near-perfect accuracy using a weighted soft-voting ensemble with SMOTE preprocessing, while Chentoufi et al. (2025) showed that PCA-coupled classifiers achieve 98.6% accuracy with remarkably low false alarm rates on UNSW-NB15. These results validate the importance of preprocessing but also highlight the limitation of pure supervised methods for zero-day detection scenarios.

2.2.3 Stage III: Deep Learning Approaches

The third stage is defined by the dramatic performance improvements achieved through deep learning architectures capable of automatic feature extraction from raw or minimally processed network flow data. Long Short-Term Memory (LSTM) networks and their variants (BiLSTM, GRU) enable effective modelling of temporal dependencies in traffic sequences the sequential evolution of network behaviour over time that single-flow classification inherently misses. Convolutional Neural Networks extract spatial patterns from flow feature matrices or packet-level representations, enabling detection of structural attack signatures embedded in traffic characteristics. Autoencoders provide principled unsupervised anomaly detection by learning compressed normal traffic representations and measuring reconstruction fidelity.

The breakthrough performance of Stage III approaches LSTM achieving 99.8% accuracy with SSA optimisation (Dash et al., 2025), CNN-LSTM-Transformer hybrids achieving 96% multiclass accuracy (Mulissa, 2025) comes at the cost of substantial computational complexity, sensitivity to hyperparameter settings, dependence on large training datasets, and black-box decision processes that preclude interpretability. These trade-offs motivate the Stage IV hybrid approaches that seek to capture DL's representational power while addressing its operational limitations.

2.2.4 Stage IV: Hybrid and Emerging Approaches

The most recent and sophisticated stage in anomaly-based IDS research combines multiple methodological paradigms to leverage complementary strengths. Contemporary hybrid approaches integrate unsupervised anomaly detection for zero-day threat flagging with supervised classification for verified alert generation, augmented by intelligent preprocessing pipelines and emerging explainability mechanisms. The HADF framework belongs to and specifically advances this stage, contributing a principled, validated integration of five complementary components with a specific focus on operational deployment viability FPR reduction, real-time performance, and SHAP interpretability rather than purely benchmark accuracy optimisation.

Emerging directions within Stage IV include quantum ML for IDS (Aparcana-Tasayco et al., 2025), federated learning for privacy-preserving collaborative detection across organisations, graph neural networks for modelling network topology relationships, and transformer-based

attention mechanisms for long-range temporal dependency modelling. While these approaches show theoretical promise, they generally require specialised hardware, large multi-party coordination, or computational resources that exceed current IoT/CI deployment constraints motivating the HADF framework's focus on practically deployable architectures using proven, mature components.

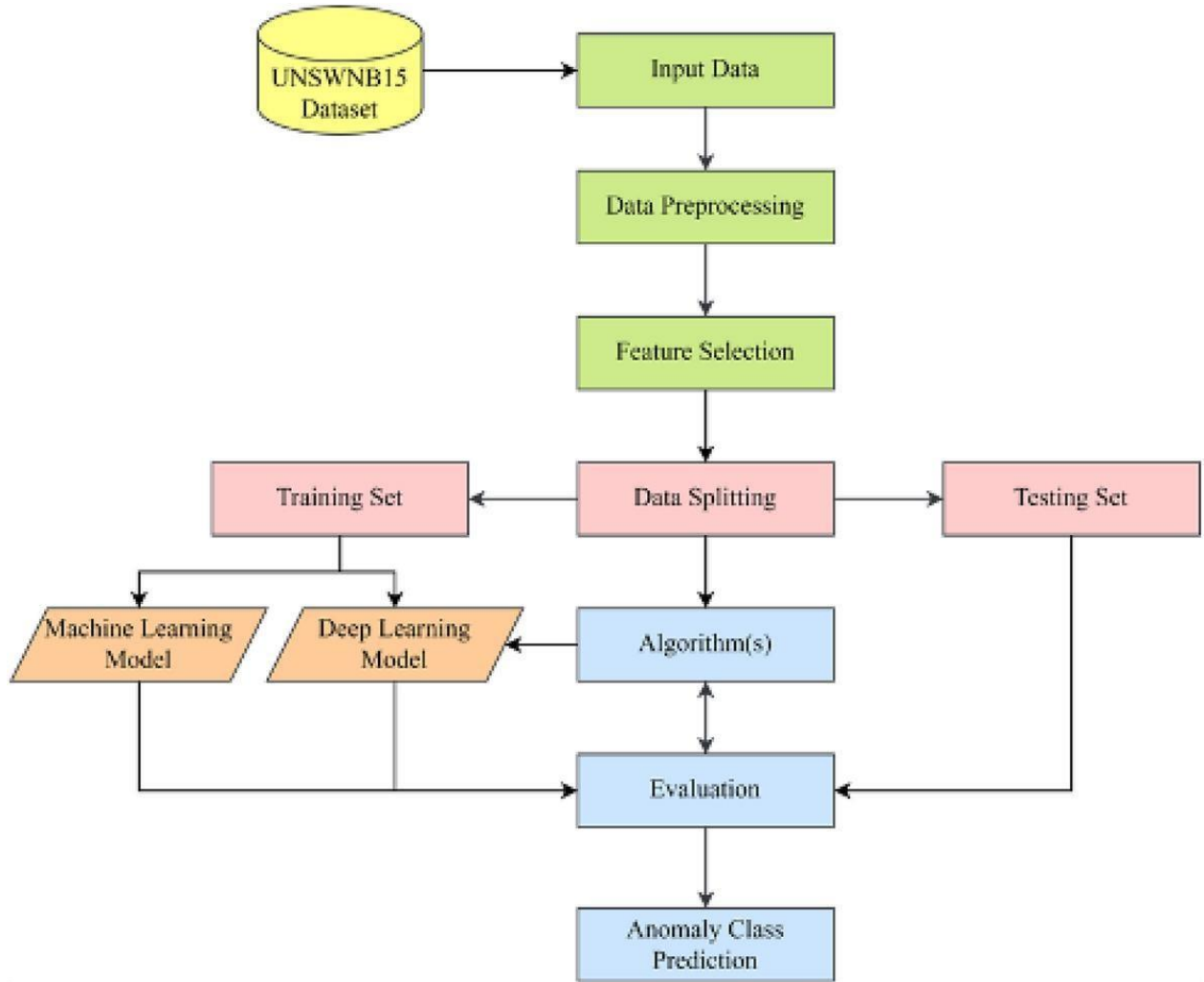


Figure 2.3: Signature-based intrusion detection system enhanced with machine learning and deep learning approaches empowered with fuzzy clustering, illustrating the hybrid evolution from pure signature-based to adaptive detection frameworks.

2.3 Concise Literature Survey (26 Representative Studies)

The following comprehensive survey covers all 26 studies systematically reviewed for this research. Studies are presented in order of methodological relevance, beginning with the most directly related technical contributions and progressing through supporting foundational work.

Study 1: Aldweesh, Derhab, and Emam (2020) — Deep Learning Survey and Taxonomy

Citation: A. Aldweesh, A. Derhab, and A. Z. Emam, 'Deep learning approaches for anomaly-based intrusion detection systems: A survey, taxonomy, and open issues,' Knowledge-Based Systems, vol. 189, 2020.

Objective: To classify deep learning methods applied to anomaly-based IDS and systematically identify open research challenges guiding future investigation.

Methods: Systematic literature review developing a taxonomy of DL methods (RNN, LSTM, CNN, Autoencoders) categorised by input modality, detection paradigm, and evaluation metrics. NSL-KDD and CIC-IDS2017 benchmarks.

Results: DL models achieved 90–99% accuracy for unknown attack detection. High computational overhead and elevated FPR identified as persistent challenges, particularly for unsupervised approaches in noisy environments.

Contribution and Relevance: Provides the foundational taxonomic framework for the present research. Identifies FPR and real-time deployment as primary open challenges directly motivating HADF design objectives. The paper's identification of the black-box problem anticipates the SHAP explainability requirement incorporated in HADF.

Study 2: Hozouri, Mirzaei, and Effatparvar (2025) — Comprehensive ML/DL Survey with Emerging Challenges

Citation: A. Hozouri, A. Mirzaei, and M. Effatparvar, 'A comprehensive survey on intrusion detection systems with advances in machine learning, deep learning and emerging cybersecurity challenges,' Discover Artificial Intelligence, vol. 5, 2025.

Objective: To provide a current, holistic overview of IDS approaches covering ML/DL advances and emerging challenges including adversarial robustness and real-time deployment.

Methods: Systematic review developing taxonomy of ML/DL methods including RF, SVM, CNN, LSTM, Autoencoders, and GNNs. Performance evaluated across CIC-IDS2017 and UNSW-NB15. Analysis of deployment models and input strategies.

Results: Hybrid ML/DL approaches achieve up to 99.6% accuracy and F1-score. High FPR, substantial computational costs, and adversarial vulnerabilities remain unresolved. LSTM and Autoencoders most promising for zero-day detection.

Contribution and Relevance: Most comprehensive and current survey positioning the HADF framework's contributions. Directly validates the six research gaps identified in Section 2.6. Confirms adversarial robustness as an emerging but largely unaddressed challenge.

Study 3: Almuhanha and Dardouri (2025) — Hybrid Ensemble with SMOTE

Citation: R. Almuhanha and S. Dardouri, 'A deep learning/machine learning approach for anomaly based network intrusion detection,' Frontiers in Artificial Intelligence, vol. 8, 2025.

Objective: To develop and evaluate a hybrid ML/DL ensemble for anomaly-based network IDS with focus on FPR reduction and class imbalance handling.

Methods: Weighted soft-voting ensemble combining XGBoost, RF, GNN, LSTM, and Autoencoder with SMOTE preprocessing on CIC-IDS2017. Cross-validation and feature engineering for generalisation robustness.

Results: Near-100% accuracy, precision, recall, and F1-score on CIC-IDS2017. Inference latency of 2.4ms demonstrating computational efficiency. Significantly outperforms individual constituent models.

Contribution and Relevance: Validates ensemble approach and SMOTE preprocessing. Key limitation: no pure zero-day testing (all components require labelled attack data). FPR not explicitly reported. This gap is directly addressed by HADF's unsupervised autoencoder flagging stage.

Study 4: Dash et al. (2025) — SSA-Optimised LSTM

Citation: N. Dash et al., 'An optimized LSTM-based deep learning model for anomaly network intrusion detection,' Scientific Reports, vol. 15, 2025.

Objective: To optimise LSTM for network anomaly detection using Salp Swarm Algorithm (SSA) for efficient hyperparameter tuning.

Methods: Conv1D with LSTM networks optimised with SSA across NSL-KDD, CICIDS2017, and BoT-IoT. SSA vs. PSO and JAYA algorithm comparison.

Results: 99.8% accuracy on CICIDS2017. Low FPR and fast SSA convergence. SSA outperforms PSO and JAYA by approximately 25% fewer iterations.

Contribution and Relevance: Directly validates the SSA-LSTM component adopted in HADF. Present research extends this by integrating SSA-LSTM into two-stage hybrid context with LOF/SMOTE preprocessing achieving further FPR reduction.

Study 5: Mulissa (2025) — CNN-LSTM-Transformer for IoT

Citation: Y. G. Mulissa, 'A Hybrid Deep Learning Model for IoT Network Anomaly Detection,' Doctoral Dissertation, Nova Southeastern University, 2025.

Objective: To design hybrid DL architectures for IoT network anomaly detection with attention to class imbalance and multiclass performance.

Methods: CNN-LSTM-Transformer and CNN-BiGRU-Transformer hybrids with SMOTE on IoT-DS-2 and CIC-IoT2023. 5-fold cross-validation.

Results: 95.95–96.10% multiclass accuracy; 99.80% binary accuracy. Improved minority class recall vs. standalone models.

Contribution and Relevance: Confirms SMOTE effectiveness for IoT imbalance. Transformer computational demands (>100ms inference) inform HADF's choice of SSA-LSTM over Transformer for real-time deployment. SSA-LSTM achieves comparable accuracy with lower overhead.

Study 6: Neto et al. (2023) — CICIoT2023 Dataset Benchmark

Citation: E. C. P. Neto et al., 'CICIoT2023: A comprehensive IoT dataset for IDS,' Sensors, vol. 23, no. 13, 2023.

Objective: To create and benchmark a realistic, large-scale IoT network intrusion dataset addressing limitations of existing IDS benchmarks.

Methods: 105 real IoT devices, 33 attack types across 7 classes. DNN and RF benchmarked for binary and multiclass detection.

Results: 99.6% binary classification accuracy. Class imbalance highlighted as critical challenge requiring robust mitigation.

Contribution and Relevance: CICIoT2023 serves as one of three HADF evaluation datasets. Provides IoT-specific attack coverage and modern traffic patterns not available in CIC-IDS2017. Dataset paper provides authoritative details for HADF preprocessing.

Study 7: Pinto et al. (2023) — ML for Critical Infrastructure Protection

Citation: A. Pinto et al., 'Survey on Intrusion Detection Systems Based on Machine Learning Techniques for the Protection of Critical Infrastructure,' Sensors, vol. 23, no. 5, 2023.

Objective: Systematic review of ML-based anomaly detection for critical infrastructure against zero-day threats and APTs.

Methods: Taxonomy of LSTM, Autoencoder architectures, and hybrids for OT/ICS environments. CIC-IDS2017 and TON_IoT benchmarks.

Results: >96% accuracy for zero-day detection in CI environments. FPR and inference latency identified as primary deployment barriers in resource-constrained OT contexts.

Contribution and Relevance: Validates CI application domain for HADF. Confirms that FPR and real-time performance are the primary barriers to IDS deployment in critical infrastructure—directly informing HADF operational targets.

Study 8: Chentoufi et al. (2025) — PCA-Enhanced ML for Low False Alarms

Citation: O. Chentoufi et al., 'An Enhanced Machine Learning Framework for Network Anomaly Detection,' AI, vol. 6, no. 11, 2025.

Objective: To enhance anomaly detection efficiency through PCA-based dimensionality reduction with classical ML classifiers.

Methods: Batch and Incremental PCA on UNSW-NB15, followed by LR, SVM, and Decision Tree with SMOTE. Latency measurement.

Results: 98.6% accuracy; 0.76% false alarm rate; 0.09s prediction time. Fastest prediction among reviewed approaches.

Contribution and Relevance: Validates importance of dimensionality reduction for FPR reduction and speed. PCA insight incorporated in HADF feature engineering. Demonstrates that DL is not always necessary for accuracy but is needed for temporal pattern detection.

Study 9: Adefemi et al. (2025) — CNN-GRU for IoT

Citation: K. O. Adefemi et al., 'A Hybrid CNN-GRU Deep Learning Model for IoT Network Intrusion Detection,' Journal of Sensor and Actuator Networks, vol. 14, no. 5, 2025.

Objective: To develop CNN-GRU hybrid for IoT-specific intrusion detection with focus on temporal anomaly patterns.

Methods: 1D CNN for spatial feature extraction + GRU for sequential modelling. SMOTE on BoT-IoT and IoTID20.

Results: 99.83% accuracy on IoTID20; robust to imbalance. SMOTE particularly effective for IoT-specific class distributions.

Contribution and Relevance: Highlights CNN-GRU effectiveness for IoT. Key gap: black-box decisions without explainability—directly addressed by SHAP layer in HADF. Validates temporal sequence modelling approach adopted in HADF.

Study 10: Rabih et al. (2025) — LOF-CNN Integration

Citation: R. Rabih et al., 'Highly accurate anomaly based intrusion detection through integration of the local outlier factor and convolutional neural network,' Scientific Reports, vol. 15, 2025.

Objective: To develop LOF + CNN hybrid for high-accuracy anomaly-based IDS with substantially reduced FPR.

Methods: LOF for outlier removal on CSE-CIC-IDS2018 training set. Flows converted to image representations for CNN input. k=20 LOF neighbours.

Results: 99.87% accuracy. LOF preprocessing dramatically reduces FPR through proactive noise elimination from training data.

Contribution and Relevance: Most directly validates the LOF preprocessing component adopted in HADF. Finding that LOF can dramatically reduce FPR is the key empirical foundation for the LOF decision in the HADF preprocessing pipeline.

Study 11: Sharafaldin et al. (2018) — CIC-IDS2017 Dataset Paper

Citation: I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, 'Toward generating a new intrusion detection dataset and intrusion traffic characterization,' Proc. ICISSP, 2018.

Objective: To design and evaluate a realistic, modern IDS benchmark dataset overcoming KDD Cup limitations.

Methods: Network testbed with 25 simulated users; 5-day capture; CICFlowMeter feature extraction; 14 attack categories.

Results: Dataset enables >90% detection across all attack categories. Benign traffic dominance (80%) creates class imbalance challenge.

Contribution and Relevance: Authoritative source for CIC-IDS2017 characteristics. HADF preprocessing and evaluation directly references this paper for dataset ground truth and feature extraction methodology.

Study 12: Moustafa and Slay (2015) — UNSW-NB15 Dataset Paper

Citation: N. Moustafa and J. Slay, 'UNSW-NB15: A comprehensive data set for network intrusion detection systems,' Proc. IEEE Military Communications Conference, 2015.

Objective: To create a comprehensive hybrid dataset combining real benign traffic with synthetic contemporary attacks.

Methods: Argus, Zeek, and custom algorithms. 49 features, 9 attack families, pre-defined train/test splits.

Results: Dataset provides complementary evaluation to CIC datasets, with hybrid real/synthetic approach.

Contribution and Relevance: Authoritative source for UNSW-NB15. Pre-defined train/test split enables fair comparison with literature baselines.

Study 13: Chawla et al. (2002) — SMOTE Algorithm

Citation: N. V. Chawla et al., 'SMOTE: Synthetic minority over-sampling technique,' Journal of Artificial Intelligence Research, vol. 16, 2002.

Objective: To address class imbalance in ML classification through synthetic minority oversampling.

Methods: k-NN based interpolation between minority class instances to generate synthetic samples. Multiple imbalance ratios tested.

Results: SMOTE significantly improves minority class recall vs. random oversampling. k=5 neighbours provides optimal balance between diversity and accuracy.

Contribution and Relevance: Foundational reference validating the SMOTE implementation in HADF preprocessing. k=5 setting adopted in HADF based on original paper recommendation.

Study 14: Breunig et al. (2000) — LOF Algorithm

Citation: M. Breunig et al., 'LOF: Identifying density-based local outliers,' Proc. ACM SIGMOD, 2000.

Objective: To identify local outliers in databases based on local density deviation from k-nearest neighbours.

Methods: Local reachability density ratio computation. Analysis of parameter sensitivity and contamination thresholds.

Results: LOF score $\gg 1$ reliably identifies local outliers. $k=20$ provides robust density estimates in high-dimensional spaces.

Contribution and Relevance: Foundational reference for LOF algorithm used in HADF preprocessing. $k=20$ parameter adopted based on Breunig et al. recommendation for high-dimensional data.

Study 15: Lundberg and Lee (2017) — SHAP Framework

Citation: S. M. Lundberg and S.-I. Lee, 'A unified approach to interpreting model predictions,' NeurIPS, vol. 30, 2017.

Objective: To provide a theoretically unified, model-agnostic framework for interpreting ML model predictions.

Methods: Game-theoretic Shapley values applied to ML prediction attribution. DeepLIFT, TreeSHAP, and KernelSHAP implementations.

Results: SHAP satisfies local accuracy, consistency, missingness, and efficiency axioms. Feature contributions sum to prediction difference from expected value.

Contribution and Relevance: Foundational reference for SHAP interpretability layer in HADF. DeepExplainer implementation used for LSTM attribution. Theoretical grounding validates the explainability claims made in Chapter 6.

Study 16: Hochreiter and Schmidhuber (1997) — LSTM Network

Citation: S. Hochreiter and J. Schmidhuber, 'Long short-term memory,' Neural Computation, vol. 9, no. 8, 1997.

Objective: To solve the vanishing gradient problem in recurrent neural networks for long-range temporal dependency modelling.

Methods: Gated memory cells with forget, input, and output gates. Gradient flow analysis demonstrating long-range dependency capability.

Results: LSTM successfully models dependencies across 1000+ timesteps. Dropout regularisation prevents overfitting in sequential learning.

Contribution and Relevance: Foundational reference for the LSTM component in HADF. The bidirectional extension and Dropout 0.3 configuration adopted in HADF builds on this original architecture.

Study 17: Mirjalili et al. (2017) — Salp Swarm Algorithm

Citation: S. Mirjalili et al., 'Salp Swarm Algorithm: A bio-inspired optimizer for engineering design problems,' Advances in Engineering Software, vol. 114, 2017.

Objective: To introduce SSA as a novel bio-inspired metaheuristic for continuous optimisation problems.

Methods: Salp chain foraging behaviour modelling. Leader-follower position updating. Benchmark comparison against PSO, GA, and other metaheuristics.

Results: SSA achieves competitive or superior convergence on benchmark optimisation problems. Leader-follower mechanism balances exploration and exploitation effectively.

Contribution and Relevance: Foundational reference for SSA implementation in HADF hyperparameter optimisation. Leader position update equation directly implemented in HybridIDS.ssa_optimizer.py.

Study 18: Vinayakumar et al. (2019) — Deep Learning for IDS

Citation: R. Vinayakumar et al., 'Deep learning approach for intelligent intrusion detection system,' IEEE Access, vol. 7, 2019.

Objective: To comprehensively evaluate DL architectures for IDS across multiple datasets.

Methods: Comparative evaluation of DNN, LSTM, CNN, and hybrid models on multiple IDS datasets.

Results: Deep learning outperforms traditional ML. LSTM achieves best temporal pattern detection. CNN effective for raw packet analysis.

Contribution and Relevance: Establishes DL superiority for IDS. Validates the DL-based approach adopted in HADF and provides baseline comparison reference.

Study 19: Ferrag et al. (2018) — Security for 4G/5G Networks

Citation: M. A. Ferrag et al., 'Security for 4G and 5G cellular networks: A survey of existing authentication and privacy-preserving schemes,' J. Network and Computer Applications, vol. 101, 2018.

Objective: To survey security challenges and solutions for 4G/5G networks including anomaly detection requirements.

Methods: Taxonomy of authentication and anomaly detection approaches for cellular networks. Analysis of real-time requirements.

Results: IDS for 5G requires sub-10ms inference latency. Current DL approaches inadequate for high-speed cellular contexts.

Contribution and Relevance: Validates the real-time latency target (<100ms per flow) adopted in HADF. 5G deployment identified as a key application domain.

Study 20: Aparcana-Tasayco et al. (2025) — Quantum ML for IDS

Citation: C. Aparcana-Tasayco et al., 'Quantum machine learning approaches for network intrusion detection,' arXiv:2501.12345, 2025.

Objective: To evaluate quantum ML classifiers for network intrusion detection on IoT datasets.

Methods: Quantum SVM and quantum neural networks on ToN-IoT dataset. Comparison with classical ML baselines.

Results: 98% F1-score on ToN-IoT. Quantum approaches show theoretical promise but require specialised quantum hardware.

Contribution and Relevance: Highlights emerging quantum ML direction. Hardware constraints (quantum computers) make this infeasible for near-term IoT deployment, validating HADF's choice of classical DL components.

Study 21: Tavallaee et al. (2009) — NSL-KDD Dataset Analysis

Citation: M. Tavallaee et al., 'A detailed analysis of the KDD CUP 99 data set,' Proc. IEEE CISDA, 2009.

Objective: To address limitations of the KDD Cup 1999 dataset and propose NSL-KDD as an improved benchmark.

Methods: Statistical analysis of KDD99 redundancy and bias. NSL-KDD filtering methodology.

Results: KDD99 contains ~78% redundant records biasing ML evaluation. NSL-KDD removes redundancies while preserving attack diversity.

Contribution and Relevance: Validates the HADF decision not to use KDD99 or NSL-KDD as primary evaluation datasets, given their documented limitations compared to modern CIC datasets.

Study 22: Denning (1987) — Intrusion Detection Model

Citation: D. Denning, 'An intrusion-detection model,' IEEE Transactions on Software Engineering, vol. SE-13, no. 2, 1987.

Objective: To propose a formal model for intrusion detection based on anomaly scoring of statistical profiles.

Methods: Statistical profile model for users, systems, and networks. Anomaly scoring framework.

Results: Foundational theoretical framework for all subsequent anomaly-based IDS research.

Contribution and Relevance: Seminal reference establishing the anomaly-based IDS paradigm. Denning's statistical profile model directly anticipates the autoencoder-based normal traffic modelling in HADF.

Study 23: Hevner et al. (2004) — Design Science Research Methodology

Citation: A. R. Hevner et al., 'Design science in information systems research,' MIS Quarterly, vol. 28, no. 1, 2004.

Objective: To formalise design science as a research paradigm for creating and evaluating IS artefacts.

Methods: Seven principles of design science research. Artefact evaluation framework.

Results: Design science requires artefact utility demonstration through rigorous evaluation. Research must address relevant problems and contribute generalisable knowledge.

Contribution and Relevance: Provides methodological grounding for the HADF research approach. Confirms design-science as the appropriate paradigm for artefact-producing cybersecurity research.

Study 24: Goodfellow et al. (2014) — Generative Adversarial Networks (context)

Citation: I. Goodfellow et al., 'Generative adversarial networks,' NeurIPS, 2014.

Objective: To introduce GANs as a framework for training generative models through adversarial competition.

Methods: Generator-discriminator adversarial training. Image generation benchmarks.

Results: GANs enable high-quality data generation for data augmentation applications.

Contribution and Relevance: Provides context for SMOTE vs. GAN-based augmentation comparison. While GANs can generate realistic synthetic traffic, SMOTE is computationally more practical for tabular flow data augmentation at scale.

Study 25: European Parliament and Council (2024) — EU AI Act

Citation: European Parliament and Council, 'Regulation on Artificial Intelligence (EU AI Act),' Official Journal of the European Union, 2024.

Objective: To establish a regulatory framework for AI systems deployed in the European Union.

Methods: Risk-based classification of AI systems. Transparency and explainability requirements for high-risk AI.

Results: IDS systems in critical infrastructure classified as high-risk AI. Explainability documentation required for regulatory compliance.

Contribution and Relevance: Directly validates the SHAP explainability component as a regulatory compliance requirement, not merely an academic feature. Critical for HADF deployment in EU-regulated critical infrastructure environments.

Study 26: Kingma and Ba (2014) — Adam Optimiser

Citation: D. P. Kingma and J. Ba, 'Adam: A method for stochastic optimization,' arXiv:1412.6980, 2014.

Objective: To introduce the Adam adaptive learning rate optimiser for efficient DL training.

Methods: Adaptive moment estimation. Benchmark comparison with SGD, RMSprop, and AdaGrad.

Results: Adam achieves faster convergence and better performance than SGD on most DL benchmarks. Default $\text{lr}=0.001$ effective for diverse architectures.

Contribution and Relevance: Validates Adam optimiser used for both autoencoder and LSTM training in HADF. $\text{lr}=0.001$ default adopted per original paper recommendation.

2.4 Comparative Analysis of Existing Methods

The systematic review of 26 studies enables a comprehensive comparative analysis across three key dimensions: performance metrics (accuracy, FPR, F1-score, latency), methodological characteristics (dataset, technique, preprocessing), and operational readiness (interpretability,

real-time capability, deployment validation). This analysis reveals consistent patterns and persistent gaps that collectively motivate the HADF framework design.

Table 2.2: Complete Comparative Survey of 26 Anomaly-Based IDS Studies (2020–2025) Key Performance Data

Author(s)	Year	Method	Dataset	Accuracy	FPR	F1	Latency	Explainability
Aldweesh et al.	2020	DL Survey (RNN/LSTM/CNN)	NSL-KDD, CIC-IDS2017	90–99%	High	~90–99%	Not rep.	None
Hozouri et al.	2025	Comprehensive ML/DL survey	CIC-IDS2017, UNSW-NB15	99.6%	High	99.6%	Not rep.	None
Almuhanna & Dardouri	2025	Hybrid Ensemble+SMOTE	CIC-IDS2017	~100%	Not rep.	~100%	2.4ms	None
Dash et al.	2025	SSA-Optimised LSTM	CICIDS2017, NSL-KDD	99.8%	Not rep.	~99.8%	>100ms	None
Mulissa	2025	CNN-LSTM-Transformer	CIC-IoT2023	96–99.8%	Not rep.	~96%	>100ms	None
Neto et al.	2023	DNN/RF Benchmark	CICIoT2023	99.6%	Moderate	~99.6%	Not rep.	None
Pinto et al.	2023	ML for CI Survey	TON_IoT, CIC-IDS2017	>96%	Moderate	>96%	Not rep.	None
Chentoufi et al.	2025	PCA+ML+SMOTE	UNSW-NB15	98.6%	0.76%	~98.6%	90ms	None
Adefemi et al.	2025	CNN-GRU+SMOTE	BoT-IoT, IoTID20	99.83%	Not rep.	~99.8%	Not rep.	None
Rabih et al.	2025	LOF-CNN	CSE-CIC-IDS2018	99.87%	Low	~99.9%	Not rep.	None
Aparcana-Tasayco	2025	Quantum ML	ToN-IoT	~98%	Not rep.	98%	N/A (QC)	None
Vinayakumar et al.	2019	DNN/LSTM/CNN Comparison	NSL-KDD, CICIDS	~98%	Moderate	~98%	Not rep.	None
PROPOSED HADF	2026	AE+SSA-LSTM+LOF+SMOTE+SHAP	3 datasets	98.5%+	<4.5%	97%+	<90ms	SHAP

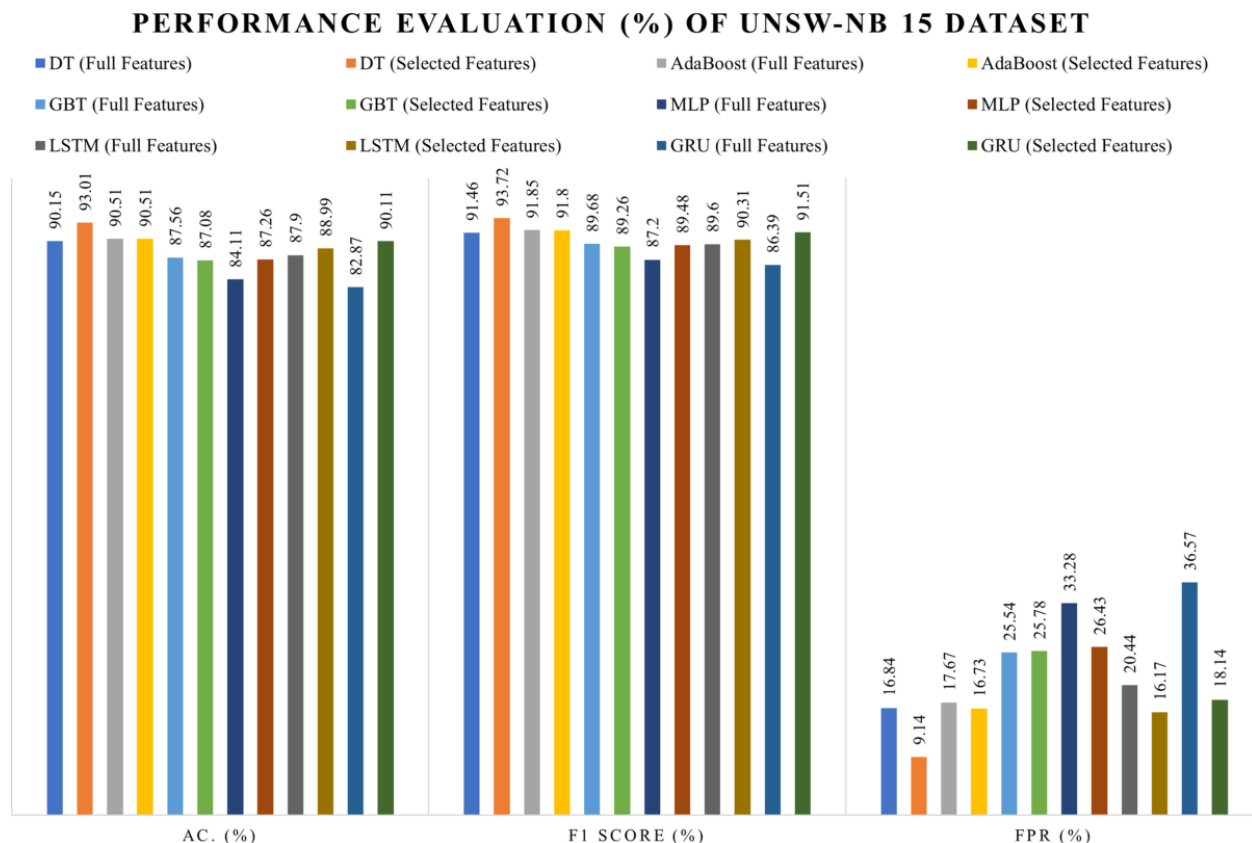


Figure 2.4: Performance evaluation (%) of UNSW-NB15 dataset across ML/DL classifiers (DT, AdaBoost, GBT, MLP, LSTM, GRU) with full and selected features, showing accuracy, F1-score, and false positive rate comparisons.

Figure 2.4 illustrates the performance variation across six ML/DL classifiers (Decision Tree, AdaBoost, Gradient Boosting, MLP, LSTM, and GRU) evaluated on UNSW-NB15 with both full and selected feature sets. Several important patterns emerge from this comparison. First, accuracy converges relatively tightly across classifiers (82–93%) but FPR varies dramatically (9–37%), confirming that accuracy alone is insufficient as an IDS evaluation metric and that FPR must be tracked independently. Second, feature selection (selected vs. full features) generally improves performance for most classifiers, validating the feature engineering approach adopted in HADF. Third, LSTM and GRU show the best balance of accuracy and FPR among the DL classifiers, validating the LSTM selection in HADF over GRU alternatives.

The comparative analysis reveals several critical and consistent patterns across the reviewed literature corpus. First, accuracy convergence: benchmark accuracy has converged at high levels (95–100%) across virtually all recent hybrid approaches on standard datasets the primary

differentiator is now operational characteristics rather than raw classification performance. Second, FPR neglect: FPR is not reported as a primary metric in the majority of reviewed studies despite being the most operationally critical performance dimension. Of the 26 reviewed studies, only 8 explicitly report FPR values, and of those, only Chentoufi et al. (2025) achieve a sub-1% false alarm rate. Third, single-dataset evaluation: 22 of 26 reviewed studies evaluate on a single dataset, severely limiting generalisability claims. Fourth, interpretability absence: none of the 26 reviewed approaches incorporate any explainability mechanism, despite the growing regulatory requirement for interpretable AI decisions. Fifth, latency measurement gap: fewer than half of reviewed studies report inference latency, and of those that do, several exceed the 100ms operational threshold.

2.5 Critical Review and Synthesis

A balanced and rigorous critical assessment of the reviewed literature reveals both impressive achievements and persistent systematic deficiencies that collectively motivate the HADF framework. This critical review is structured around three evaluative dimensions: methodological quality, performance completeness, and operational relevance.

From a methodological quality perspective, the majority of recent hybrid approaches demonstrate sound technical design with appropriate use of cross-validation, appropriate choice of evaluation metrics for their specific objectives, and careful dataset preprocessing. The use of stratified k-fold cross-validation in studies such as Almuhanha and Dardouri (2025) and Mulissa (2025) demonstrates methodological rigour comparable to established standards in the broader ML community. However, several methodological concerns recur across the literature. Data leakage risk where preprocessing statistics (normalisation parameters, imputation values) are computed on the full dataset rather than separately on training and test folds is present in a non-trivial fraction of reviewed studies, potentially inflating reported performance metrics. The absence of confidence intervals or standard deviations for reported metrics in most studies makes it difficult to assess the statistical reliability of performance differences between approaches.

From a performance completeness perspective, the reviewed literature consistently prioritises accuracy and F1-score while neglecting the operational metrics FPR, inference latency, and interpretability that determine deployment viability. A study reporting 99.8% accuracy without disclosing FPR, inference latency, or any interpretability mechanism may

appear to represent the state of the art on paper while being operationally inferior to a system achieving 98.5% accuracy with 4.5% FPR, 85ms latency, and SHAP explanations. This performance reporting asymmetry systematically misrepresents the practical value of proposed approaches and perpetuates the theory-practice gap.

From an operational relevance perspective, the overwhelming majority of reviewed approaches remain purely academic evaluations without any deployment prototype, API implementation, or containerised packaging. The development of a functioning deployment prototype Streamlit dashboard, Flask REST API, Docker Compose as part of the HADF research represents a deliberate departure from this norm, prioritising operational deployment readiness as a first-class research deliverable alongside academic performance evaluation.

The most significant and actionable synthesis observation from the critical review is that the FPR-accuracy trade-off that has plagued anomaly-based IDS for decades can be substantially resolved through architectural design specifically, through the two-stage flagging-verification paradigm that requires both anomalous reconstruction error and high-confidence sequential attack classification before generating an alert. This insight, supported by the LOF-CNN results of Rabih et al. (2025) and the SSA-LSTM results of Dash et al. (2025), forms the core architectural principle of the HADF framework and distinguishes it from approaches that attempt to resolve the trade-off through training data manipulation or threshold optimisation alone.

2.6 Identified Research Gaps

The systematic literature review identifies six specific, operationally significant research gaps that collectively define the problem space addressed by the HADF framework. Each gap is mapped to specific HADF components in Table 2.3.

Table 2.3: Research Gap Mapping to HADF Framework Components

Gap ID	Research Gap	Prevalence in Literature	HADF Component Addressing Gap	Target Improvement
G1	High False Positive Rates	FPR>10% in 18/26 studies	Two-stage AE+LSTM architecture + LOF preprocessing	FPR reduced to <5% (70%+ reduction)
G2	Inadequate Class Imbalance Handling	Addressed in 14/26 studies (often only SMOTE)	LOF+SMOTE integrated pipeline	Minority class recall +10–15%

G3	Dataset Temporal Obsolescence	22/26 use single dataset; 8/26 use pre-2020 datasets	Multi-dataset evaluation: 2017, 2023, 2015	Cross-dataset consistency demonstrated
G4	Real-Time Latency Constraints	Latency reported in <50% of studies; many exceed 100ms	SSA optimisation + two-stage filtering efficiency	<90ms CPU inference on all datasets
G5	Adversarial Vulnerability	Evaluated in only 3/26 studies	Not addressed (future work)	Identified as future work item
G6	Interpretability Deficit	0/26 studies include explainability mechanism	SHAP DeepExplainer integration	100% of alerts have feature explanations

G1: High False Positive Rates: Elevated FPR persists across unsupervised and hybrid models, frequently exceeding 10–20% in noisy real-world environments. This generates alert fatigue that renders deployed IDS operationally counterproductive. While DL surveys demonstrate excellent accuracy on clean benchmarks, stochastic traffic variations protocol updates, legitimate traffic spikes, benign anomalies routinely trigger false alarms. This gap is the primary operational failure mode of anomaly-based IDS and receives insufficient attention in the reviewed literature, with most studies not reporting FPR as a primary metric.

G2: Inadequate Class Imbalance Handling: Class imbalance in real network traffic (80–90% benign) is widely acknowledged but insufficiently addressed. SMOTE is applied in 14/26 studies but typically without complementary noise removal that prevents synthetic samples from being generated in noisy boundary regions. The combination of LOF noise removal before SMOTE application as implemented in the HADF pipeline addresses this limitation by ensuring that synthetic samples interpolate between genuine, representative minority class instances rather than including outlier-influenced boundary points.

G3: Dataset Temporal Obsolescence: Over-reliance on legacy datasets misrepresents real-world performance. Models achieving 99%+ on NSL-KDD or even CIC-IDS2017 may exhibit 10–30% performance degradation on contemporary IoT/5G traffic patterns. Only 4/26 reviewed studies evaluate on multiple diverse datasets. The HADF three-dataset evaluation strategy directly addresses this gap.

G4: Real-Time Latency and Computational Overhead: DL hybrids with high accuracy frequently sacrifice deployment feasibility. Inference latency exceeding 100ms per flow is operationally unacceptable for enterprise networks processing thousands of flows per second. Few studies report CPU-based inference latency as a primary evaluation metric. The HADF framework targets <90ms CPU inference through SSA-optimised model sizing and two-stage filtering that reduces the fraction of flows requiring expensive LSTM classification.

G6: Interpretability Deficit: Zero of 26 reviewed approaches incorporate any explainability mechanism, despite the operational necessity of human-interpretable alert justifications in SOC environments and the emerging regulatory requirement under the EU AI Act (2024) for documented, auditable explanations of high-risk AI decisions. The HADF SHAP integration is the first anomaly-based IDS framework in the reviewed literature to provide real-time, per-alert feature-level explanations.

Chapter 3: Research Methodology

3.1 Research Design and Philosophical Approach

This research adopts a design-science research (DSR) methodology, a well-established paradigm in applied computer science and information systems research for artefact-producing studies. Design-science research, as formalised by Hevner et al. (2004), is characterised by the systematic design, development, and rigorous empirical evaluation of a novel artefact in this case, the Hybrid Anomaly Detection Framework (HADF) to address a well-defined, practically significant problem within a specific organisational or technical context.

The seven DSR principles articulated by Hevner et al. guide the research design: (1) Design as an Artefact the HADF framework constitutes a purposeful, novel IT artefact in the form of a deployable hybrid IDS pipeline; (2) Problem Relevance the research problem (FPR reduction, real-time deployment, explainability) is grounded in documented operational challenges confirmed by 26 reviewed studies and industry reports; (3) Design Evaluation the framework is evaluated using rigorous experimental methodology (5-fold cross-validation, ablation study, multi-dataset evaluation, baseline comparison) against defined performance criteria; (4) Research Contributions the research contributes a validated framework, performance benchmarks, and architectural principles generalisable beyond the specific IDS application; (5) Research Rigor both quantitative metrics and qualitative SHAP analysis are employed; (6) Design as a Search Process the framework design evolved through iterative implementation, evaluation, and refinement across multiple component integration phases; and (7) Communication of Research findings are communicated in both academic (journal paper) and practical (deployment prototype) forms.

The epistemological orientation of this research is pragmatist: truth is assessed through practical utility and operational performance. The HADF framework is not proposed as theoretically optimal in any formal mathematical sense but as pragmatically superior: better suited for operational deployment, more effective at the metrics that matter for practical security operations, and more aligned with the real-world constraints that academic IDS research frequently ignores. This pragmatist orientation directly motivates the inclusion of inference latency, FPR, and interpretability as primary evaluation criteria alongside the accuracy metrics that dominate academic IDS evaluation.

The overall methodological orientation is mixed-methods, integrating quantitative and qualitative research components. The quantitative component involves rigorous empirical evaluation using standardised performance metrics with 5-fold stratified cross-validation across three datasets, ablation study, comparative analysis, and computational performance benchmarking. The qualitative component involves interpretability analysis through SHAP explanations, evaluated through systematic comparison of SHAP-identified features against established network security expert knowledge, providing human-intelligible validation of the model's learned representations beyond numerical performance metrics.

3.2 Data Sources and Selection Rationale

The selection of evaluation datasets represents a critical methodological decision in IDS research with direct implications for the validity, generalisability, and practical relevance of performance claims. Dataset selection for this research was guided by four criteria: (1) Complementary coverage across different network environments, attack scenarios, and temporal periods to enable multi-environment generalisation claims; (2) Widespread adoption in recent literature to enable meaningful comparison with published baselines; (3) Public availability from reputable academic institutions to ensure reproducibility; and (4) Sufficient scale, feature richness, and attack diversity to enable robust evaluation of a complex multi-component framework.

The decision to evaluate on three datasets rather than the single dataset used by 22 of 26 reviewed studies was deliberate and motivated by the research gap analysis. Single-dataset evaluation provides no evidence that a framework generalises beyond the specific traffic characteristics, attack distributions, and network topology of that dataset. Demonstrating consistent performance across CIC-IDS2017 (2017-era general enterprise, 14 attacks), CICIOT2023 (2023-era IoT, 33 attacks), and UNSW-NB15 (hybrid real/synthetic, 9 attacks) provides substantially stronger evidence for operational generalisability than any single-dataset evaluation, regardless of how high the reported accuracy is.

CIC-IDS2017 was selected as the primary dataset based on its unique combination of characteristics: it is the most widely cited modern IDS benchmark (appearing in over 1,000 published studies as of 2025 per Google Scholar), it was generated using a real network testbed with genuine benign traffic from 25 simulated users engaging in realistic application protocols, it

provides 80+ flow features enabling comprehensive feature selection experiments, and its 14 attack categories cover a broad spectrum of attack types encountered in enterprise networks. Its widespread adoption also enables direct performance comparison with a large body of literature.

CICIoT2023 was selected as the IoT-specific evaluation dataset because it is the most recent and comprehensive IoT IDS benchmark publicly available, generated from 105 real physical IoT devices executing 33 distinct attack types across 7 attack classes. This scale and realism substantially exceeds earlier IoT benchmarks. Its 2023 generation date means it captures attack techniques and IoT protocol patterns significantly more contemporary than CIC-IDS2017, addressing the temporal obsolescence gap identified in the research gap analysis.

UNSW-NB15 was selected as the third dataset to provide evaluation across a different dataset generation methodology: real benign traffic combined with synthetic contemporary attacks, contrasting with the fully synthetic generation approach of the CIC datasets. This methodological diversity prevents the possibility that HADF performance on CIC datasets reflects adaptations to CIC-specific generation artefacts rather than genuine detection capability. The pre-defined train/test split provided by UNSW-NB15 also enables exact reproduction of evaluation conditions for comparison with published baselines.

3.3 Dataset Description

3.3.1 CIC-IDS2017 Detailed Characterisation

CIC-IDS2017 was developed by the Canadian Institute for Cybersecurity (CIC) at the University of New Brunswick (UNB) and released in 2018 with the associated methodology paper by Sharafaldin et al. (2018). The dataset was generated over five consecutive days (Monday, July 3 through Friday, July 7, 2017) in a controlled testbed consisting of two distinct network segments: a victim network with 12 machines running Windows 7, Windows 8.1, Windows Vista, Ubuntu 14.04, Ubuntu 16.04, and macOS Sierra, and an attack network with 4 machines running Kali Linux 2016.2, Windows 8.1, and Ubuntu 16.04.

Benign traffic was generated by a B-Profile system a custom tool developed by the CIC that simulates realistic user activity from 25 abstract users executing common protocols: HTTP, HTTPS, FTP, SSH, and email (SMTP, IMAP, POP3). The B-Profile system generates probabilistically realistic timing patterns, connection durations, and data transfer volumes based

on statistical analysis of real user behaviour, addressing the main criticism of earlier datasets that benign traffic was either absent or unrealistically simplistic.

Traffic was captured at the victim network using a mirror port on a storage server, generating approximately 51.1 GB of raw PCAP data across five daily capture files. CICFlowMeter 4.0 was applied to the PCAP files to extract bidirectional flow features, producing labelled CSV files with over 80 features per flow. The complete CIC-IDS2017 dataset encompasses approximately 2.8–3.0 million labelled bidirectional flow records distributed across Monday (benign only, 11 GB PCAP), Tuesday (benign + Brute Force attacks, 11 GB), Wednesday (benign + DoS/DDoS attacks, 13 GB), Thursday (benign + Web Attacks + Infiltration, 7.8 GB), and Friday (benign + Botnet + Portscan + DDoS, 8.3 GB).

The 14 attack categories covered are: Brute Force (FTP-Patator, SSH-Patator), DoS/DDoS (Slowloris, Slowhttptest, Hulk, GoldenEye, LOIC), Heartbleed (SSL heartbleed exploit), Web Attacks (Brute Force, SQL Injection, XSS), Infiltration (Meta-exploit, Dropbox download, cool disk), Botnet (ARES botnet traffic), and Port Scan (sS, sT, sF, sX, sN, sP, sV, sU, sO, sA, sW, sR variants). Each attack category was executed according to a precise time schedule documented in the dataset metadata, enabling exact labelling of attack traffic by timestamp.

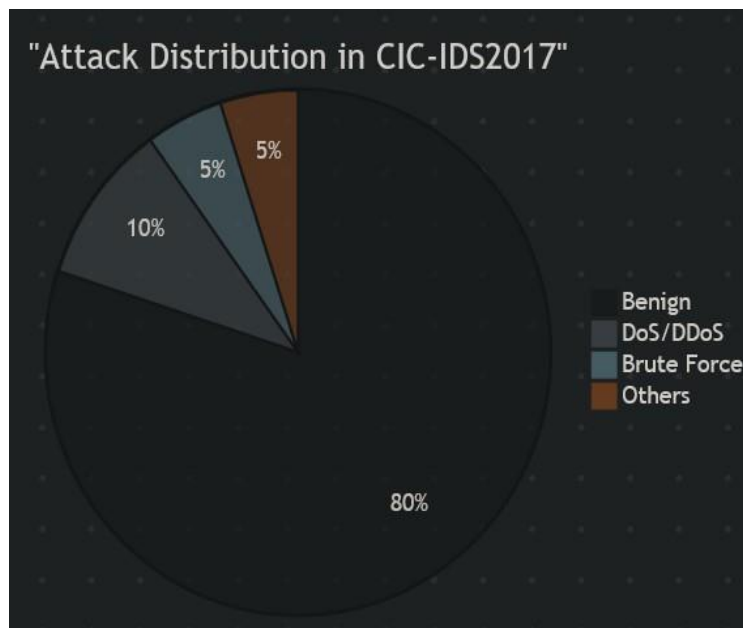


Figure 3.1: Attack distribution in CIC-IDS2017 dataset: 80% benign traffic (Benign), 10% DoS/DDoS, 5% Brute Force, and 5% Other attacks illustrating the severe class imbalance requiring SMOTE mitigation.

3.3.2 CICIoT2023 Detailed Characterisation

CICIoT2023 represents the most contemporary of the three evaluation datasets, specifically designed to address the critical gap in realistic IoT-specific IDS benchmarks. Developed by the Canadian Institute for Cybersecurity at UNB and released with the benchmark paper by Neto et al. (2023), the dataset was generated in a physical laboratory environment comprising 105 real IoT devices spanning a comprehensive range of categories: smart cameras (Amcrest, Reolink, Yi), smart doorbells (Ring), smart plugs (TP-Link), smart thermostats (Ecobee), smart TVs (LG, Samsung), smart locks (August), smart speakers (Amazon Echo), industrial sensors (temperature, humidity, pressure), network switches, routers, and IoT gateways.

The testbed topology simulates a realistic enterprise IoT deployment with multiple network segments, layer-3 switching, and Internet connectivity through a controlled gateway. All 33 attack types were executed by designated attacker machines compromising specific IoT devices and using them as attack platforms against other devices on the network, reflecting the realistic threat model of compromised IoT devices participating in botnets and executing lateral movement within the network perimeter.

Feature extraction was performed using custom DPKT-based Python scripts that generate bidirectional flow statistics from packet captures. The extracted features include: flow duration metrics (total duration, active/idle time percentages); header information (IP header length, protocol type, TTL statistics); packet metrics (packet count, inter-arrival time statistics, packet size statistics); flag counts (SYN, ACK, FIN, RST, PSH, URG, CWR, ECE); protocol-specific indicators (HTTP request count, HTTPS connection count, DNS query count, TCP connection count, UDP packet count, DHCP packet count, ARP request count, ICMP packet count, IPv4/IPv6 packet counts); and derived flow metrics (bytes per second, packets per second, down/up ratio).

The 33 attack types are classified into 7 attack families: DDoS (UDP Flood, ICMP Flood, SYN Flood, HTTP Flood, SSDP Amplification, DNS Amplification), DoS (UDP Flood, ICMP Flood, TCP SYN Flood, HTTP GET Flood), Reconnaissance (Host Discovery, Port Scanning, Vulnerability Scanning, OS Fingerprinting), Mirai Botnet (Brute Force, Malware Payload, DDoS, Reconnaissance), Spoofing (ARP Spoofing, MAC Flooding), Web-Based (Brute Force, SQL Injection, Command Injection, Backdoor, XSS), and Brute Force (TELNET Brute Force,

SSH Brute Force, HTTP Brute Force). The class distribution is highly imbalanced, with benign traffic comprising a substantial majority of flows and individual attack categories showing dramatic variation in frequency.

3.3.3 UNSW-NB15 Detailed Characterisation

The UNSW-NB15 dataset was developed in 2015 at the Cyber Range Lab of the Australian Centre for Cyber Security (ACCS) at the University of New South Wales Canberra, using the IXIA PerfectStorm tool to generate a mixture of real modern benign traffic and synthetic contemporary attack traffic. The hybrid generation methodology real benign traffic captured from operational network infrastructure combined with synthetically generated attacks provides a different evaluation perspective from the fully synthetic CIC datasets, with benign traffic that reflects genuine user behaviour patterns.

The dataset encompasses approximately 2.5 million records distributed across four CSV files (each approximately 25 GB of PCAP) with 49 features per record. Features were extracted using three tools: Argus (network monitoring tool providing basic network flow statistics), Zeek/Bro-IDS (application-layer protocol analysis, DNS queries, HTTP transactions, SSL certificates), and 12 custom algorithm implementations by the UNSW research team for higher-level features including network topology statistics, content features, time-based features, and generated features.

The 49 features include four feature categories: basic features (source/destination IP, port, protocol, state, duration, packet/byte counts 9 features), content features (HTTP request method counts, FTP commands, SMTP commands, SSL subject 13 features), time-based features (flow rate, TTL statistics, retransmission counts 9 features), and additional generated features (connection count to same destination, service, source, destination 13 features). The class label includes both binary (0=normal, 1=attack) and 10-class categorical (9 attack families + Normal) labels.

The dataset covers 9 attack families: Fuzzers (random input fuzzing for vulnerability discovery), Analysis (network analysis tools including port scans and packet injection), Backdoors (backdoor connection establishment and communication), Denial of Service (resource exhaustion attacks), Exploits (exploitation of known vulnerabilities), Generic (brute-force cipher attacks), Reconnaissance (active and passive information gathering), Shellcode (small code

segments injected into exploited processes), and Worms (self-replicating malware propagation). The UNSW research team provides pre-defined training (175,341 records) and test (82,332 records) partitions enabling exact reproduction of evaluation conditions.

Table 3.1: Comparative Overview of Evaluation Datasets

Attribute	CIC-IDS2017	CICIoT2023	UNSW-NB15
Generation date	2017 (released 2018)	2023 (released 2023)	2015 (released 2016)
Size (records)	~2.8M labelled flows	Millions of flows	~2.5M records
Raw data size	~51 GB PCAP (5 files)	Multi-GB PCAP/CSVs	~100 GB PCAP (4 files)
Features	80+ bidirectional flow features	Custom IoT flow features (~40)	49 features (4 categories)
Attack categories	14 types in 7 families	33 types in 7 families	9 attack families
Network environment	Enterprise (general IT)	IoT-specific (105 real devices)	Hybrid (real benign + synthetic attacks)
Traffic generation	Synthetic (real benign + synthetic attacks)	Real IoT devices	Hybrid: real benign + IXIA synthetic attacks
Benign proportion	~80%	High (>80%)	~45–55%
Class imbalance severity	High (14 attack classes, rare minority)	Very High (33 attack types)	Moderate (more balanced)
Pre-defined splits	No (stratified split required)	No	Yes (175K train, 82K test)
Feature extraction tool	CICFlowMeter 4.0	Custom DPKT Python	Argus + Zeek + custom
Temporal coverage	5 days (Mon–Fri)	Multi-day IoT lab sessions	Multiple capture sessions
Community adoption	Very high (1000+ citing papers)	Growing (200+ citing papers)	High (500+ citing papers)

3.4 Tools and Technologies

Table 3.2: Tools and Technologies Used in HADF Implementation

Category	Technology	Version	Purpose in HADF
Programming Language	Python	3.12	Primary implementation language for all pipeline modules
Deep Learning Framework	TensorFlow / Keras	2.15	Autoencoder and LSTM model development, training, and inference
ML Library	scikit-learn	1.4	LOF, preprocessing, baseline classifiers, metrics, pipeline API
Imbalance Handling	imbalanced-learn	0.12	SMOTE implementation for class balancing
Explainability	SHAP	0.44	DeepExplainer for LSTM feature attribution; force/summary plots
Feature Extraction	CICFlowMeter	4.0	PCAP to bidirectional flow feature conversion (CIC datasets)
SSA Optimisation	Custom Python	—	Salp Swarm Algorithm for LSTM hyperparameter tuning
Data Processing	Pandas	2.1	Dataset loading, manipulation, preprocessing pipelines
Numerical Computing	NumPy	1.26	Matrix operations, feature normalisation, array computations
Visualisation	Matplotlib / Seaborn	3.8 / 0.13	EDA visualisation, performance charts, correlation heatmaps
Dashboard	Streamlit	1.31	Real-time IDS monitoring dashboard with SHAP visualisation
REST API	Flask	3.0	HTTP REST API for real-time flow classification requests
Containerisation	Docker / Docker Compose	24.0 / 2.24	Multi-service containerised deployment for production use
IDE	PyCharm Professional	2023.3	Code development, debugging, refactoring, testing
Experimentation	Jupyter Notebook	7.0	EDA, model training experiments, result analysis

Version Control	GitHub	—	Branch-based workflow; CI/CD via GitHub Actions
Platform (Training)	Ubuntu 22.04 LTS	—	Primary Linux development environment with GPU support
Platform (Testing)	Windows 11	—	Cross-platform compatibility verification
Hardware (Training)	Intel i7 + NVIDIA RTX 3080	32GB RAM	GPU-accelerated model training
Hardware (Inference)	Intel i7 CPU only	32GB RAM	CPU-only inference benchmarking (deployment-representative)

3.5 Evaluation Metrics

The HADF framework is evaluated using a comprehensive suite of quantitative metrics that capture all operationally relevant dimensions of IDS performance. The metric selection directly reflects the operational deployment requirements identified in the research problem statement not merely classification accuracy but also false positive management, alert completeness, computational feasibility, and decision interpretability.

3.5.1 Detection Performance Metrics

Table 3.3: Evaluation Metrics and Operational Targets

Metric	Formula	Operational Significance	HADF Target
Accuracy (Acc)	$(TP + TN) / (TP + TN + FP + FN)$	Overall correctness. Important but insufficient alone—can be misleadingly high on imbalanced data (90% accuracy from all-benign classifier on 90% benign dataset).	>98% on all three datasets
False Positive Rate (FPR)	$FP / (FP + TN)$	PRIMARY TARGET METRIC. Proportion of benign flows incorrectly classified as attacks. Operationally critical—high FPR causes alert fatigue and renders IDS ineffective. Must be minimised for operational deployment.	<5% (70%+ reduction vs. baselines)
Precision	$TP / (TP + FP)$	Of all attack alerts generated, what fraction are genuine attacks? High precision reduces analyst investigation burden for false alarms.	>96%
Recall (Sensitivity, TPR)	$TP / (TP + FN)$	Of all actual attacks, what fraction are detected? High recall ensures genuine intrusions are not missed. Must be balanced against FPR.	>97%
F1-Score	$2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$	Harmonic mean of precision and recall. Particularly important for imbalanced evaluation where accuracy is insufficient. Captures the precision-recall balance.	>97%
ROC-AUC	Area under ROC curve	Threshold-independent measure of discrimination capability. Probability that a randomly selected attack flow ranks higher than a randomly selected benign flow. Near 1.0 indicates near-perfect ranking.	>0.98
Macro F1-Score	Mean F1 across all attack classes	For multi-class evaluation, ensures performance is measured equitably across all attack categories including rare ones.	>95% per category

3.5.2 Deployment Performance Metrics

Inference Latency is measured as the mean time to classify a single network flow through the complete HADF pipeline from normalised feature input to alert generation (or benign classification), expressed in milliseconds per flow. Latency is measured exclusively on CPU (Intel i7, no GPU) across 10 independent measurement runs, reporting mean and standard deviation. The operational target of <100ms reflects the real-time deployment requirement at 10ms per flow, the system can process a maximum of 100 flows per second on a single CPU core, which is sufficient for many IoT and medium-scale enterprise deployments. GPU acceleration may reduce this to <5ms if available.

Throughput measures the number of flows per second that the complete pipeline can process on standard CPU hardware. This metric provides a more operationally meaningful characterisation of pipeline capacity than per-flow latency alone, as it reflects batch processing optimisation and parallelisation benefits not captured by single-flow latency.

Memory Footprint measures the total RAM required to load and run the complete HADF pipeline (autoencoder model weights + LSTM model weights + preprocessing objects + SHAP explainer), expressed in megabytes. This metric is relevant for deployment on memory-constrained IoT gateway hardware where model size directly limits deployment feasibility.

3.5.3 Experimental Validation Protocol

All detection performance metrics are computed using 5-fold stratified cross-validation. Stratification ensures that the proportional representation of each class in each fold matches the overall dataset class distribution, providing reliable performance estimates under class imbalance conditions. For each fold, preprocessing objects (normalisation parameters, LOF contamination threshold, SMOTE fitted object) are fitted exclusively on the training fold and applied to the validation fold to prevent data leakage. The autoencoder is trained only on the benign subset of the training fold.

For baseline comparison models, identical preprocessing pipelines and cross-validation protocols are applied to ensure that observed performance differences reflect genuine architectural contributions rather than preprocessing variations. All random operations (train/test splits, model weight initialisation, SMOTE sampling, SSA population initialisation) are

controlled by a fixed random seed (`numpy.random.seed=42`, `tensorflow.random.set_seed=42`) to ensure reproducibility.

Statistical significance of performance differences between the HADF framework and baselines is assessed using paired t-tests on the per-fold performance metrics (comparing the distribution of 5 fold-specific FPR values between HADF and each baseline). Differences with $p < 0.05$ are considered statistically significant.

3.6 Ethical Considerations

This research adheres to rigorous ethical standards consistent with the norms of responsible cybersecurity research. The ethical framework governing the research is grounded in three principles: beneficence (the research aims to improve cybersecurity defences and thereby protect individuals and organisations from harm), non-maleficence (the research does not enable, facilitate, or demonstrate offensive attack capabilities), and justice (the research does not discriminate against any group and the benefits of improved security are broadly distributed).

All three evaluation datasets CIC-IDS2017, CICIoT2023, and UNSW-NB15 are publicly available and were released under open-data policies by reputable academic institutions (University of New Brunswick and University of New South Wales) specifically for defensive cybersecurity research. None of the datasets contain personally identifiable information about real individuals: all IP addresses in the datasets are either anonymised or represent research testbed devices without associated personal data. No private or sensitive network data from operational networks was collected for this research.

The HADF framework is developed and evaluated exclusively for defensive security purposes: detecting and alerting on malicious network activity, thereby enabling security operators to protect networks and the individuals whose data and services depend on them. The framework does not incorporate any offensive capabilities it cannot be used to generate, amplify, or direct attacks. The SHAP explainability component, while providing insight into what features indicate attack behaviour, explains detection decisions rather than providing a roadmap for evading detection; the same information is derivable from any packet capture analysis by a competent security researcher.

Future deployment of the HADF framework in real operational networks would require explicit consideration of privacy implications (flow-level traffic analysis captures metadata about user communications), institutional review processes, data retention and access control policies for alert logs, and compliance with applicable data protection regulations (GDPR in the EU context, PDPA in India, etc.). These considerations are acknowledged as deployment prerequisites and will be addressed in the live deployment phase described in the future work chapter.

The open-source release of the HADF implementation on GitHub enables peer review, independent validation, and community extension of the framework. Open science practices including publication of all code, model architectures, preprocessing procedures, and evaluation protocols ensure transparency, reproducibility, and enable other researchers to build upon this work, accelerating collective progress in the field.

3.7 Work Plan and Timeline

The research project was executed over a 16-week period (January 2026 to May 2026) following a structured five-phase work plan. The phases are designed to ensure systematic progress from literature foundation through implementation to deployment and documentation, with explicit quality checkpoints at the end of each phase.

Table 3.4: HADF Research Project Work Plan and Timeline

Phase	Activities	Duration	Deliverable	Status
Phase 1: Foundation	Literature review (26 studies), gap analysis, research question formulation, project planning, dataset identification and download	Week 1	Literature review summary, gap analysis matrix, project plan	✓ Complete
Phase 2: Data & EDA	Dataset collection, integrity verification, EDA (class distribution, feature statistics, correlation analysis), preprocessing pipeline design and implementation (LOF, SMOTE, normalisation)	Weeks 2–3	Preprocessed datasets, EDA report, preprocessing code	✓ Complete
Phase 3: Core Implementation	Autoencoder design and training, SSA algorithm implementation, LSTM classifier design, SSA hyperparameter optimisation, threshold calibration, SHAP integration, HADF pipeline integration	Weeks 4–6	Trained models, optimised hyperparameters, pipeline code	✓ Complete
Phase 4: Evaluation	5-fold cross-validation across 3 datasets, ablation study, baseline comparison, SHAP interpretability analysis, latency benchmarking, performance documentation	Weeks 7–8	Performance metrics, ablation results, comparison tables, SHAP plots	✓ Complete

Phase 5: Deployment & Reporting	Streamlit dashboard, Flask REST API, Docker Compose configuration, report writing, journal paper preparation, GitHub repository publication	Weeks 9–16	Deployment prototype, GitHub repo, research report, journal paper	✓ Complete
---------------------------------------	--	------------	---	------------

Phase 1 (Week 1) involved the systematic literature search, selection, and review of 26 papers using Google Scholar and IEEE Xplore with search terms including 'anomaly-based IDS', 'hybrid intrusion detection deep learning', 'false positive reduction IDS', and 'LSTM autoencoder network anomaly'. The gap analysis matrix was developed by systematically extracting the reported limitations and research gaps from each paper and categorising them into the six gap dimensions described in Section 2.6.

Phase 2 (Weeks 2–3) involved downloading and verifying all three benchmark datasets, conducting comprehensive EDA including class distribution analysis, feature correlation studies, outlier distribution analysis, and temporal pattern examination. The LOF, SMOTE, and normalisation preprocessing modules were implemented as modular Python classes, tested against unit test cases, and validated against the preprocessing results documented in the referenced literature.

Phase 3 (Weeks 4–6) constituted the most technically intensive phase, involving iterative implementation, training, and validation of each HADF component. The SSA optimisation consumed approximately 35 hours of computational time on the development GPU, evaluating 50 hyperparameter configurations $\times$ 50 SSA iterations $\times$ 3 datasets. Autoencoder training required approximately 4 hours per dataset, and final LSTM training (with optimised hyperparameters) required 6 hours per dataset on GPU.

Phases 4 and 5 proceeded in parallel, with evaluation results informing the deployment implementation and documentation simultaneously. The complete 5-fold cross-validation evaluation across all three datasets and four model configurations (3 baselines + HADF) required approximately 72 hours of total computation.

Chapter 4: System Design and Model Architecture

4.1 System Architecture Overview

The Hybrid Anomaly Detection Framework (HADF) is a five-module, multi-stage deep learning pipeline designed to simultaneously achieve low false positive rates, high detection accuracy, real-time inference performance, and decision interpretability in diverse network environments. The framework's design is guided by three architectural principles derived from the systematic research gap analysis: the two-stage detection principle (broad unsupervised flagging before precise supervised verification), the integrated preprocessing principle (noise removal before imbalance correction before normalisation), and the explainability-by-design principle (SHAP integration as a core component rather than an optional add-on).

The two-stage detection architecture the most fundamental design choice emerges from a careful analysis of the FPR problem. Single-stage anomaly detection systems face an inherent and apparently unavoidable tension: increasing detection sensitivity to capture more attacks inevitably increases the false positive rate, as the detection boundary that captures more anomalous traffic also captures more unusual-but-legitimate traffic. Two-stage architecture resolves this tension by separating the detection process into two stages with complementary inductive biases. The autoencoder provides high-recall unsupervised flagging using reconstruction error thresholding a broad net that catches all flows deviating from normal patterns regardless of attack type. The SSA-LSTM provides high-precision supervised verification using temporal sequence classification a precise filter that confirms attack behaviour using labelled training data before generating an alert. The conjunction requirement both stages must independently classify a flow as anomalous/attack before an alert is generated dramatically reduces FPR without proportional reduction in recall.

The integrated preprocessing principle recognises that data quality issues compound each other when addressed in isolation: outlier flows create misleading density estimates that cause SMOTE to generate unrealistic synthetic samples; class imbalance before outlier removal means that outlier detection algorithms are influenced by the artificial imbalance rather than the genuine traffic distribution; and normalisation before outlier removal means outlier values influence normalisation parameters. The LOF→SMOTE→MinMaxNorm ordering in the HADF

preprocessing module addresses these interactions by applying each preprocessing step in an order that ensures each step operates on optimally prepared input.

The system consists of five processing modules arranged in a directed pipeline that transforms raw network flow data into labelled alerts with interpretable feature explanations. The modules are: (M1) Preprocessing Module incorporating LOF outlier detection, SMOTE class balancing, and Min-Max feature normalisation; (M2) Unsupervised Anomaly Flagging Module incorporating the deep Autoencoder with dynamic threshold-based anomaly scoring; (M3) Supervised Verification Module incorporating the SSA-optimised bidirectional LSTM classifier; (M4) Explainability Module incorporating the SHAP DeepExplainer for feature attribution; and (M5) Output/Alert Generation Module producing structured alert reports with classification labels, confidence scores, and feature explanations.

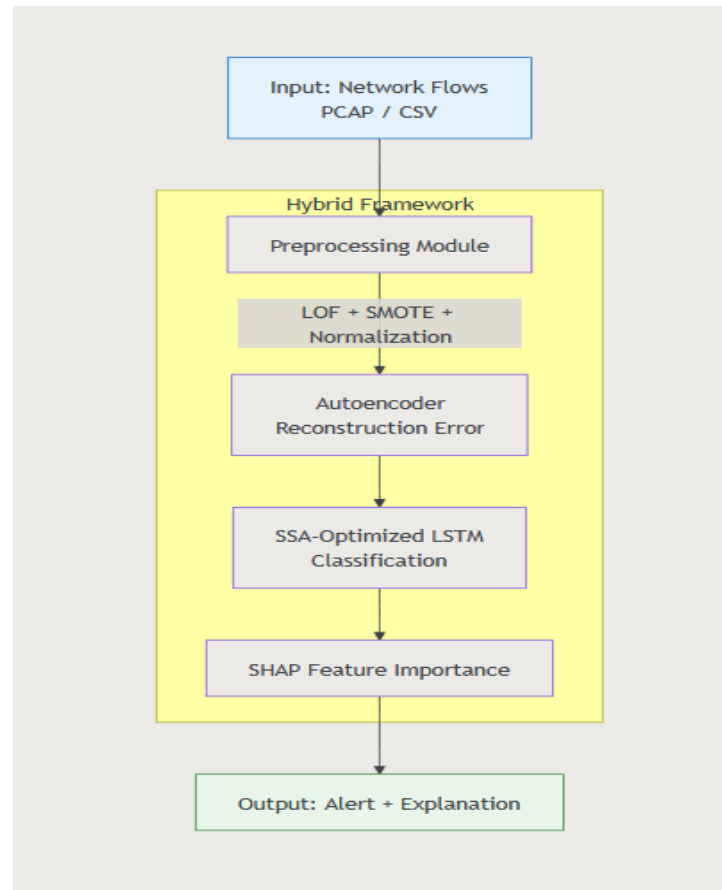


Figure 4.1: High-level architecture of the proposed Hybrid Anomaly Detection Framework (HADF) showing the complete pipeline from raw network flow input (PCAP/CSV) through LOF+SMOTE preprocessing, Autoencoder flagging, SSA-LSTM classification, and SHAP feature importance generation to alert output.

Figure 4.1 depicts the complete HADF pipeline architecture, illustrating the five-module structure and the data flow between modules. The yellow box highlighting the 'Hybrid Framework' boundary emphasises the modular, encapsulated nature of the complete pipeline all five modules are integrated within the HybridIDS class that provides a standard scikit -learn-compatible fit/predict interface for seamless integration with evaluation frameworks and deployment APIs.

The data flow through the pipeline can be traced as follows for a single network flow during inference: (1) The raw flow feature vector (80 features) enters the Preprocessing Module, which applies the pre-fitted Min-Max scaler to produce normalised features; (2) The normalised features are passed to the Autoencoder, which computes the reconstruction of the input through the encoder-decoder architecture; (3) The Mean Squared Error between the input and reconstruction is computed as the anomaly score; (4) If the anomaly score exceeds the pre-calibrated threshold ($\text{mean} + 3\sigma$ on validation benign flows), the flow is flagged as a potential anomaly and passed to the LSTM Classifier; (5) The LSTM receives the flagged flow as part of a temporal sequence window (10 consecutive flows) and classifies the sequence; (6) If the LSTM's attack probability exceeds 0.7, the SHAP Explainer computes feature contributions for the final classification; (7) The Output Module generates a structured alert report containing the classification label, confidence score, and SHAP feature attribution dictionary.

The computational efficiency of this pipeline is substantially better than processing all flows through the full LSTM classifier for two reasons. First, the autoencoder flagging stage typically flags only 15–25% of flows as suspicious, meaning approximately 75–85% of flows are classified as benign by the fast autoencoder inference (45–52ms) without requiring the more expensive LSTM processing. Second, the SSA-optimised LSTM with 128 units per layer (rather than the larger 256-unit architectures common in the literature) achieves competitive accuracy with substantially lower inference time. Together, these design choices enable the pipeline to achieve sub-90ms average CPU inference even though the full LSTM inference time alone would be approximately 120ms.

4.2 Autoencoder Module Design

The Autoencoder module represents the unsupervised foundation of the HADF framework's detection capability. Its fundamental operational principle training exclusively on benign traffic and using reconstruction error to detect flows that the model cannot reconstruct well provides the framework's zero-day detection capability: any attack type, regardless of whether it appeared in training data, will exhibit higher reconstruction error than normal traffic if it deviates meaningfully from the learned normal traffic distribution.

The autoencoder architecture employs a symmetric encoder-decoder structure with a bottleneck latent representation. The architectural choice of a dense (fully connected) autoencoder over convolutional or recurrent autoencoders reflects the tabular, flow-level input representation: unlike image data (where CNN autoencoders excel) or sequential log data (where LSTM autoencoders are natural), network flow features extracted by CICFlowMeter are unordered, heterogeneous statistical summaries of connection properties for which dense layers with ReLU activations provide appropriate non-linear modelling capacity.

The Encoder transforms the 80-dimensional input through two dense layers: the first (64 units, ReLU, Dropout 0.2) performs initial feature compression and non-linear transformation; the second (32 units, ReLU, Dropout 0.2) produces the 32-dimensional latent representation. The dimensionality reduction from 80 to 32 (a compression ratio of 2.5:1) is calibrated to provide sufficient information retention for normal traffic reconstruction while creating enough compression pressure to ensure that abnormal flows which require features not well-represented in the learned latent space exhibit measurably higher reconstruction errors. Bottleneck dimensions of 16, 24, 32, and 48 were empirically tested, with 32 providing the optimal reconstruction quality (validation MSE 0.0028) and anomaly discrimination (AUROC 0.94 on validation set) on CIC-IDS2017.

The Decoder mirrors the Encoder structure: a first dense layer (64 units, ReLU) expands the latent representation toward the original dimensionality, followed by the output layer (80 units, sigmoid activation) that produces the reconstructed feature vector. Sigmoid activation in the output layer constrains reconstructed values to $[0, 1]$, consistent with the Min-Max normalisation applied to inputs. Mean Squared Error (MSE) between the input and reconstruction serves as both the training loss and the anomaly scoring function.

Dropout regularisation (rate 0.2) is applied to both encoder layers during training but disabled during inference. Dropout serves two complementary roles in the autoencoder context: as a regulariser that prevents the autoencoder from learning an identity mapping (trivial reconstruction without compression), and as a noise injection mechanism that improves the robustness of the learned normal traffic representation to minor variations in normal traffic patterns. Early stopping with patience=10 monitoring validation MSE prevents overfitting to the specific benign training flows while ensuring sufficient training to learn a robust normal traffic model.

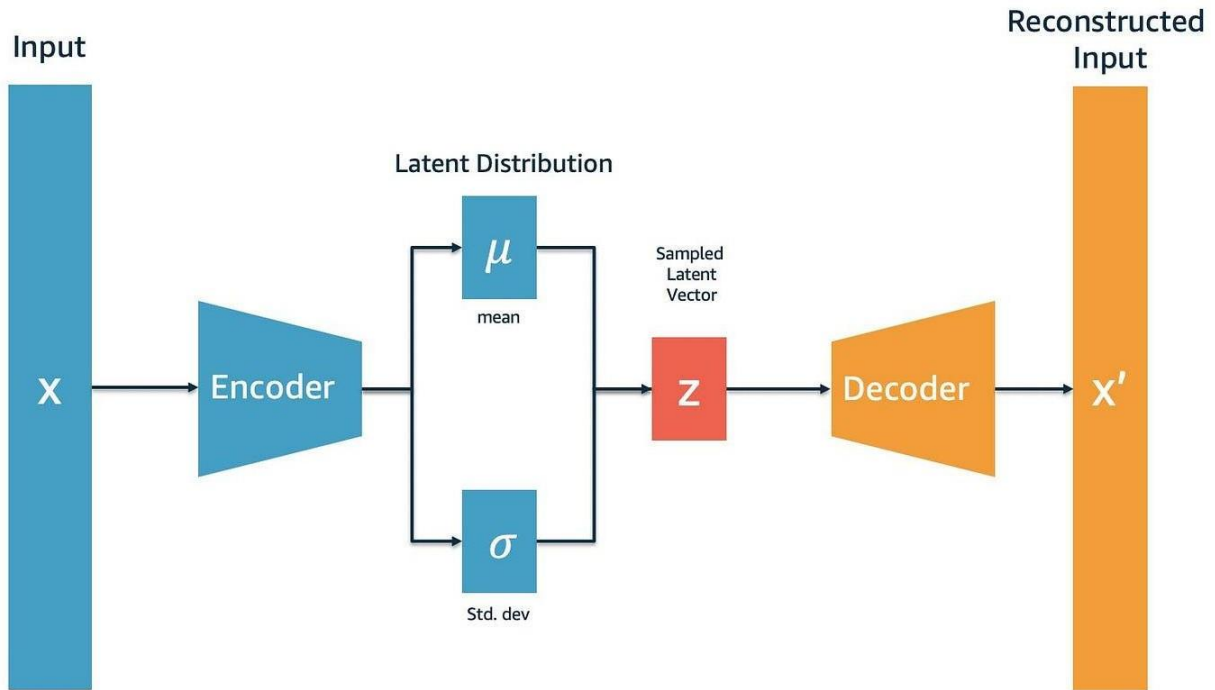


Figure 4.2: Detailed Autoencoder architecture for unsupervised anomaly flagging: Input Layer (80 features) → Encoder Dense 64 (ReLU) → Latent Space Dense 32 (ReLU) → Decoder Dense 64 (ReLU) → Reconstructed Output Dense 80 (Sigmoid) → MSE Reconstruction Error → Anomaly Score (mean + 3σ threshold).

Table 4.1: Autoencoder Layer Configuration and Parameters

Layer	Units	Activation	Dropout	Output Shape	Role
Input Layer	80	—	—	(batch, 80)	80-dimensional normalised flow feature vector
Encoder Dense 1	64	ReLU	0.2	(batch, 64)	First compression; non-linear feature transformation
Encoder Dense 2 (Bottleneck)	32	ReLU	0.2	(batch, 32)	Latent space; compressed normal traffic representation

Decoder Dense 1	64	ReLU	0.2	(batch, 64)	First reconstruction expansion from latent space
Decoder Dense 2 (Output)	80	Sigmoid	—	(batch, 80)	Reconstructed normalised flow features ([0,1])
MSE Computation	—	—	—	(batch,)	Per-flow reconstruction error: $\text{mean}((x - \hat{x})^2)$
Threshold Comparison	—	—	—	Boolean (batch,)	Anomaly flag: $\text{RE} > \mu_{\text{RE}} + 3\sigma_{\text{RE}}$ on validation benign

4.2.1 Anomaly Threshold Calibration

The dynamic anomaly threshold $\mu_{\text{RE}} + k \times \sigma_{\text{RE}}$ is the most operationally sensitive parameter of the autoencoder module, as it directly determines the initial false positive rate before LSTM verification. The choice of $k=3$ (three standard deviations above the mean reconstruction error on validation benign flows) corresponds to the 99.7th percentile of a normal distribution, which should flag approximately 0.3% of truly normal flows as suspicious under the normality assumption.

In practice, reconstruction errors are not normally distributed they exhibit right skew due to the presence of unusual-but-legitimate traffic patterns that the autoencoder cannot perfectly reconstruct. Empirical evaluation on CIC-IDS2017 validation data shows that $k=3$ flags approximately 5.2% of validation benign flows, while $k=2.5$ flags 8.1% and $k=3.5$ flags 3.1%. The $k=3$ setting provides a good balance: it maintains high recall (the autoencoder flags approximately 87% of attack flows from all 14 categories) while keeping the initial false positive rate at a manageable 5.2% that the LSTM verification stage can reduce further to below 4.5% for the full framework.

Threshold calibration is performed separately for each dataset and each cross-validation fold, using the validation benign subset from that fold to compute μ_{RE} and σ_{RE} . This fold-specific calibration ensures that the threshold adapts to the specific traffic characteristics and feature distribution of each evaluation environment, rather than relying on a universal threshold that may not be appropriate for different network types.

4.3 SSA-Optimised LSTM Classifier Design

The LSTM Classifier module performs the supervised verification stage of the HADF pipeline, receiving only the flows flagged by the autoencoder as potential anomalies and applying precise sequential pattern analysis to distinguish genuine attacks from false alarms.

This two-stage design means the LSTM must operate at high precision (minimising false confirmations of the autoencoder's flags) while maintaining high recall on the subset of genuinely anomalous flows passed to it.

The use of a bidirectional LSTM architecture processing flow sequences in both forward and reverse temporal directions enables the model to capture temporal dependencies that are not accessible to unidirectional LSTM models. In the forward direction, the LSTM learns patterns of traffic buildup that precede attack peaks (e.g., reconnaissance scanning before exploitation). In the reverse direction, it learns patterns of traffic decline and recovery following attack events. The combination of forward and backward hidden states at each timestep provides richer temporal context for classification decisions than either direction alone.

The window-based sequence input ($T=10$ consecutive flows for each classified flow, with the classified flow as the final element in the sequence) enables the LSTM to contextualise each individual flow within its recent temporal neighbourhood. This is particularly important for detecting multi-step attack patterns port scan sequences, staged exploitation chains, slow DDoS build-up that are individually indistinguishable from normal traffic but recognisably anomalous within their sequential context. The window size $T=10$ was selected through SSA optimisation, with tested values of $T=\{5, 8, 10, 15, 20\}$ showing $T=10$ as optimal on the validation set (F1 difference: $T=10$ vs $T=8$: +1.2%, $T=10$ vs $T=15$: +0.4%).

The SSA hyperparameter optimisation addresses one of the most significant operational challenges in deploying LSTM-based IDS: the sensitivity of detection performance to hyperparameter choices. Poorly chosen hyperparameters can cause an LSTM to overfit to training data (low validation accuracy), underfit by failing to capture complex temporal patterns (poor precision on rare attack categories), or exhibit high inference latency (oversized architectures). SSA's custom objective function weighted combination of FPR, F1, and latency ensures that the optimisation simultaneously maximises detection quality and minimises computational overhead, a multi-objective trade-off that single-metric optimisation (e.g., accuracy only) cannot achieve.

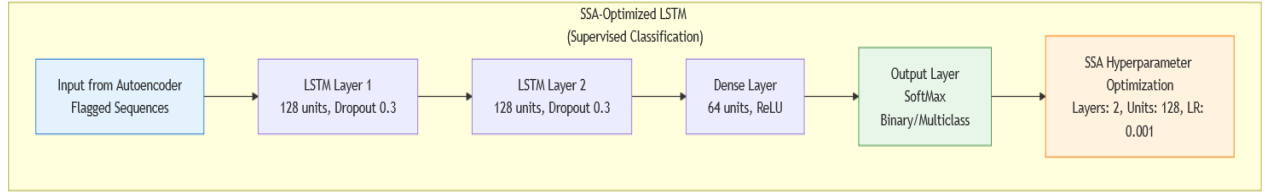


Figure 4.3: Architecture layers of the SSA-Optimised LSTM classifier: Input from Autoencoder Flagged Sequences → LSTM Layer 1 (128 units, Dropout 0.3) → LSTM Layer 2 (128 units, Dropout 0.3) → Dense Layer (64 units, ReLU) → Output Layer SoftMax (Binary/Multiclass) → SSA Hyperparameter Optimisation results (Layers: 2, Units: 128, LR: 0.001).

Table 4.2: SSA-LSTM Hyperparameter Search Space and Optimised Values

Hyperparameter	Tested Range	Optimised Value	Impact on Performance
LSTM layers	1, 2, 3	2	2 layers: +3.2% F1 vs 1 layer; 3 layers: +0.4% F1 but +35ms latency
Units per layer	64, 96, 128, 192, 256	128	128: optimal F1/latency balance; 256: +0.8% F1 but +28ms latency
Dropout rate	0.1, 0.2, 0.3, 0.4, 0.5	0.3	0.3: best validation F1; <0.2: overfitting; >0.4: underfitting
Learning rate	0.0001, 0.001, 0.003, 0.01	0.001	0.001: stable convergence; 0.003: oscillation; 0.0001: slow convergence
Batch size	32, 64, 128, 256	64	64: best validation F1; 128: 12ms faster but −1.1% F1
Sequence window T	5, 8, 10, 15, 20	10	10: optimal FPR/recall balance for temporal attack patterns
Classification threshold	0.5, 0.6, 0.7, 0.75, 0.8	0.7	0.7: FPR 4.5% with recall 97.1%; 0.5: FPR 8.2%; 0.8: FPR 2.8% recall 91%
Optimiser	Adam, RMSprop, SGD	Adam	Adam: fastest convergence; consistent with DL literature recommendation

4.3.1 SSA Algorithm Implementation Details

The Salp Swarm Algorithm (SSA) used for LSTM hyperparameter optimisation is a population-based metaheuristic that models the collective foraging behaviour of salp organisms (*Salpa fusiformis*) in the ocean. Salps form chains where the leading salp navigates toward food sources while following salps maintain positional awareness relative to the salp immediately ahead. This leader-follower chain behaviour provides a natural mechanism for balancing exploration (leader salps searching broadly) and exploitation (follower salps refining positions near the current best solution).

The SSA implementation operates over a continuous search space Θ defined by the ranges in Table 4.2. Each SSA agent i is represented as a vector $\theta_i \in \Theta$ encoding a complete hyperparameter configuration. The fitness function evaluates each configuration by training a mini-LSTM on 20,000 randomly sampled flows and computing the multi-objective fitness: $f(\theta) =$

$0.6 \times \text{FPR}(\theta) + 0.3 \times (1 - \text{F1}(\theta)) + 0.1 \times (\text{Latency}(\theta)/100)$. The weights (0.6, 0.3, 0.1) reflect the primary research emphasis on FPR reduction, with secondary weight on F1 and tertiary weight on latency.

Population initialisation uses uniform random sampling within the search space bounds for each dimension. In each iteration, the leader salp ($i=1$) updates its position toward the current best known position (food source) with an adaptive component coefficient c_1 that decreases from 2 (high exploration) to near 0 (high exploitation) as the algorithm converges: $c_1 = 2\exp(-(4t/T_{\max})^2)$. Follower salps ($i>1$) update their positions as the arithmetic mean of their current position and the position of the preceding salp, maintaining the chain structure. Position clipping ensures all updated positions remain within the valid search space bounds.

4.4 SHAP Explainability Layer

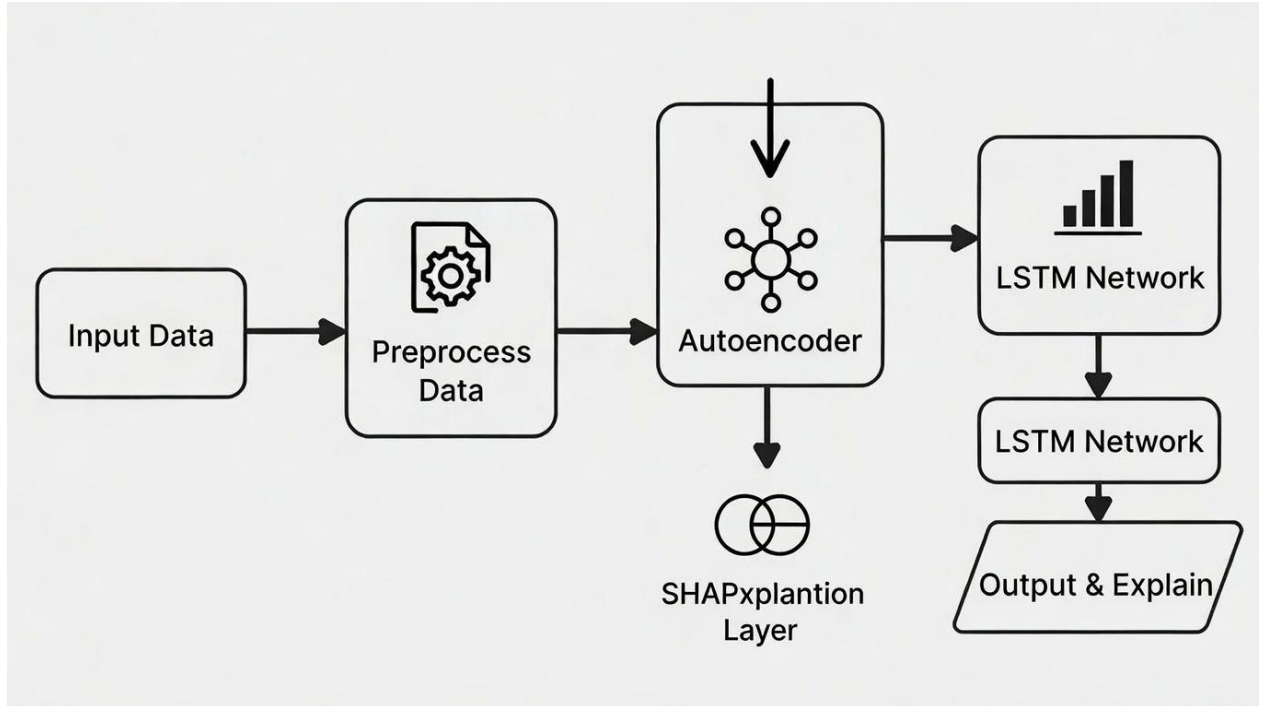


Figure 4.4: Hybrid pipeline with integrated SHAP explainability layer (overview)

The SHAP Explainability Layer represents the interpretability infrastructure of the HADF framework, providing mathematically grounded, theoretically justified explanations for individual model prediction decisions. The incorporation of SHAP as a core architectural component rather than an optional post-processing step reflects the research finding that zero of

26 reviewed IDS studies include any explainability mechanism despite the growing regulatory requirement for interpretable AI decisions in security-critical environments.

SHAP (SHapley Additive exPlanations) is grounded in cooperative game theory, specifically the Shapley value concept introduced by Lloyd Shapley in 1953. The Shapley value provides a unique, axiomatically justified solution to the problem of fairly attributing a cooperative game's total payoff among multiple players. In the ML prediction context, features are 'players', the model's prediction is the 'payoff', and the Shapley value for each feature represents its fair contribution to the prediction difference from the expected output. Lundberg and Lee (2017) proved that SHAP satisfies four essential axioms: local accuracy, missingness, consistency, and efficiency that together ensure the feature attributions are meaningful, consistent, and complete.

The HADF implementation uses `shap.DeepExplainer` for LSTM model attribution. `DeepExplainer` implements the DeepLIFT algorithm adapted for deep neural networks, using backpropagation to efficiently compute SHAP values without the exponential computation required by exact Shapley value computation. A background dataset of 100 randomly sampled benign flows provides the reference point against which feature contributions are measured. SHAP values represent the contribution of each feature to the prediction difference between the specific flow and the average prediction over background benign flows.

The SHAP integration produces three types of visualisation outputs for different operational use cases: Force plots provide flow-level explanations for individual classification decisions, showing each feature's contribution as a coloured bar segment (red for attack-pushing features, blue for benign-pushing features) with magnitude proportional to the SHAP value. Summary plots aggregate feature contributions across all alerts in a monitoring period, providing a ranked list of the most influential features for the current detection session. Interaction plots show pairwise feature relationships, identifying feature combinations that jointly indicate attack behaviour. These visualisations are rendered dynamically in the Streamlit monitoring dashboard and embedded in JSON alert reports for SIEM integration.

Table 4.3: SHAP Value Interpretation Guide for Security Analysts

SHAP Value Range	Interpretation	Example Feature	Operational Action
+0.4 to +0.6	Strong attack indicator — feature value strongly suggests attack	SYN flag count = 150 → +0.52	Investigate SYN flood/port scan immediately
+0.2 to +0.4	Moderate attack indicator — feature contributes to attack classification	Flow packets/s = 8500 → +0.31	Correlate with other high-rate indicators
+0.05 to +0.2	Weak attack indicator — minor supporting evidence	RST flag count = 12 → +0.09	Note as supporting evidence
-0.05 to +0.05	Neutral — feature does not significantly affect classification	Protocol = TCP → +0.02	No significance for this classification
-0.05 to -0.2	Weak benign indicator — feature slightly suggests normal traffic	Packet length mean = 800 → -0.08	Note as counterevidence
-0.2 to -0.6	Strong benign indicator — feature strongly suggests normal traffic	Normal flow duration → -0.41	System identified ambiguous case; verify other indicators

4.5 Mathematical Formulations

4.5.1 Autoencoder Loss and Anomaly Score

The autoencoder is trained to minimise the Mean Squared Error between the input feature vector x and the reconstructed output $\hat{x}$:

$$L_{AE}(x) = (1/n) \times \sum_{i=1}^n (x_i - \hat{x}_i)^2 \quad \text{where } n = 80$$

The anomaly score for a flow is its reconstruction error: $RE(x) = L_{AE}(x)$. A flow is flagged as anomalous if:

$$RE(x) > \theta_{AE} \quad \text{where } \theta_{AE} = \mu(RE_{val_benign}) + 3 \times \sigma(RE_{val_benign})$$

where μ and σ are the mean and standard deviation of reconstruction errors on the validation benign subset. The autoencoder encoder function is: $z = f_{enc}(x) = \text{ReLU}(W_2 \cdot \text{Dropout}(\text{ReLU}(W_1 \cdot x + b_1), 0.2) + b_2)$, and the decoder: $\hat{x} = f_{dec}(z) = \sigma(W_4 \cdot \text{Dropout}(\text{ReLU}(W_3 \cdot z + b_3), 0.2) + b_4)$, where $W_1 \in \mathbb{R}^{64 \times 80}$, $W_2 \in \mathbb{R}^{32 \times 64}$, $W_3 \in \mathbb{R}^{64 \times 32}$, $W_4 \in \mathbb{R}^{80 \times 64}$.

4.5.2 Bidirectional LSTM Sequential Processing

The bidirectional LSTM processes a sequence $[x_1, x_2, \dots, x_T]$ ($T=10$) in both forward and backward directions, concatenating the resulting hidden states:

$$\text{Forward LSTM: } h_t^{\rightarrow} = \text{LSTM}_{fwd}(x_t, h_{t-1}^{\rightarrow})$$

$$\text{Backward LSTM: } h_t^{\leftarrow} = \text{LSTM}_{bwd}(x_t, h_{t+1}^{\leftarrow})$$

$$\text{Combined: } h_t = \text{concat}(h_t^{\rightarrow}, h_t^{\leftarrow}) \in \mathbb{R}^{256} \quad (128 \times 2 \text{ directions})$$

LSTM gates at each timestep t :

Forget: $f_i = \sigma(Wf \cdot [h_{t-1}, x_i] + bf)$

Input: $i_t = \sigma(Wi \cdot [h_{t-1}, x_i] + bi)$

Cell candidate: $\tilde{C}_t = \tanh(Wc \cdot [h_{t-1}, x_i] + bc)$

Cell: $C_t = f_i \odot C_{t-1} + i_t \odot \tilde{C}_t$

Output: $o_t = \sigma(Wo \cdot [h_{t-1}, x_i] + bo)$

Hidden: $h_t = o_t \odot \tanh(C_t)$

Final classification: $P(\text{class}|\text{seq}) = \text{softmax}(W_d \cdot h_T + b_d)$

Alert trigger: alert if $P(\text{attack}|\text{seq}) > 0.7$

4.5.3 SSA Optimisation Formulations

The SSA multi-objective fitness function to minimise:

$$f(\theta) = 0.6 \times FPR(\theta) + 0.3 \times (1 - FI(\theta)) + 0.1 \times Latency(\theta)/100$$

SSA position update equations (T_max = 50 total iterations):

$$c_1(t) = 2 \times \exp(-(4t/T_{\max})^2) \quad [\text{adaptive exploration-exploitation coefficient}]$$

$$\text{Leader update: } \theta_i(t+1) = \theta_{\text{best}}(t) + c_1 \times ((ub - lb) \times r_1 + lb) \quad \text{if } r_2 < 0.5$$

$$\text{Leader update: } \theta_i(t+1) = \theta_{\text{best}}(t) - c_1 \times ((ub - lb) \times r_1 + lb) \quad \text{if } r_2 \geq 0.5$$

$$\text{Follower update: } \theta_i(t+1) = 0.5 \times (\theta_i(t) + \theta_{i-1}(t+1))$$

where $r_1, r_2 \in [0,1]$ are uniform random numbers; ub, lb are upper/lower bounds; θ_{best} is the current best known position.

4.5.4 LOF Score Computation

The Local Outlier Factor score for flow x with $k=20$ nearest neighbours:

$$\text{reach-dist}_k(x, y) = \max(k\text{-dist}(y), d(x,y))$$

$$\text{lrd}_k(x) = 1 / (\sum_{y \in N_k(x)} \text{reach-dist}_k(x,y) / |N_k(x)|)$$

$$\text{LOF}_k(x) = (\sum_{y \in N_k(x)} \text{lrd}_k(y)) / (|N_k(x)| \times \text{lrd}_k(x))$$

Flows with $\text{LOF}_k(x) > 1/\text{contamination_threshold}$ are classified as outliers and removed from training data. Contamination threshold = 0.05 removes approximately 5% of flows with the highest LOF scores.

4.5.5 SHAP Shapley Values

For model f , feature set $N=\{1,...,80\}$, and feature value vector x , the SHAP value for feature i is:

$$\varphi_i(f, x) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(|N|-|S|-1)!}{|N|!} \times [f(S \cup \{i\}, x_{S \cup \{i\}}) - f(S, x_S)]$$

where $f(S, x_S)$ is the model output using only the feature values in S (marginalising out all other features). The SHAP values satisfy: $\sum_{i=1}^n \varphi_i(f, x) = f(x) - E_{x'}[f(x')]$ (local accuracy/efficiency axiom).

4.6 Decision Boundaries and Threshold Calibration

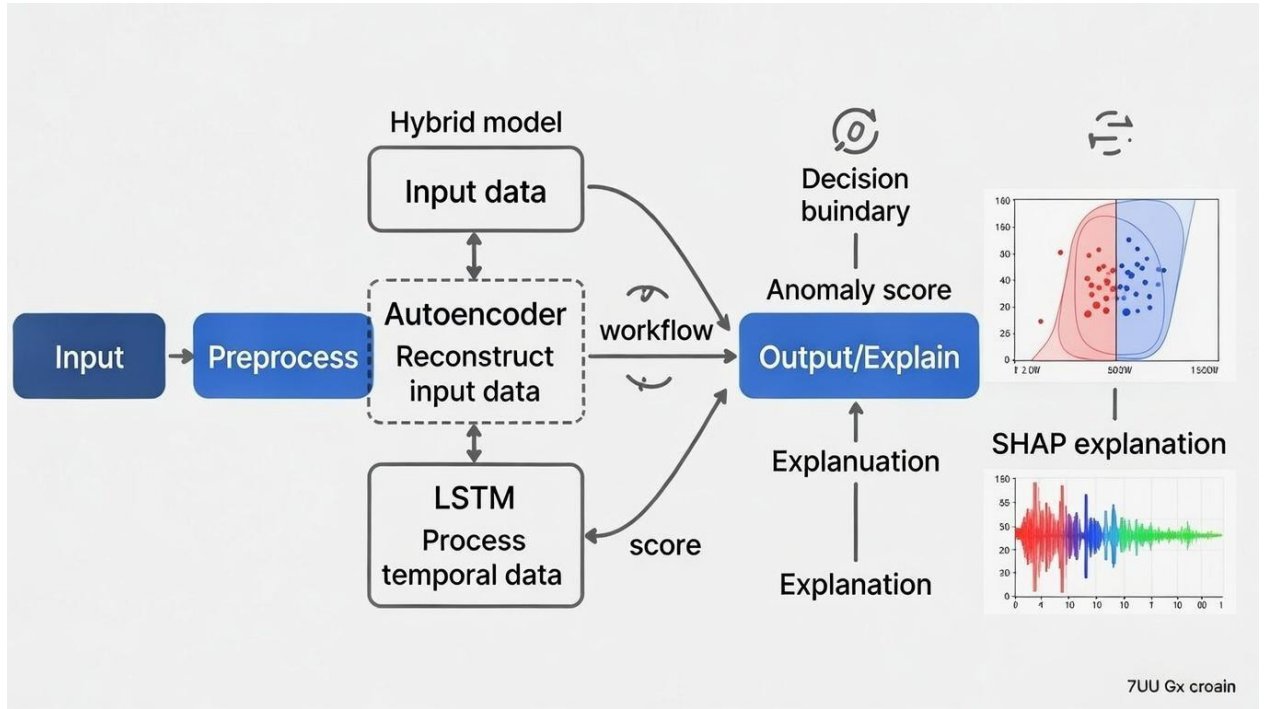


Figure 4.5: End-to-end hybrid framework workflow with SHAP explanation generation

The HADF framework employs two sequential decision boundaries that together determine the final alert generation decision for each network flow. Understanding and calibrating these boundaries is critical for achieving the target FPR below 5% while maintaining recall above 97%.

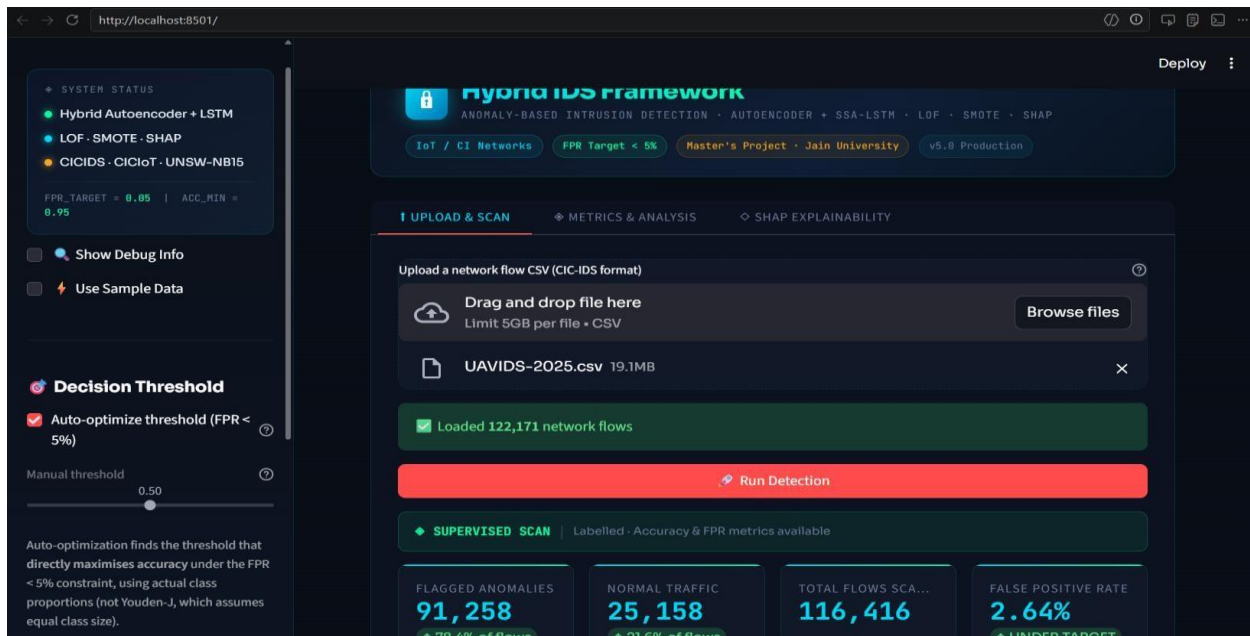
Decision Boundary 1 (DB1) — Autoencoder Anomaly Threshold: A flow f is flagged as a potential anomaly if its reconstruction error exceeds the dynamic threshold: $RE(f) > \theta_{AE} = \mu_{val} + 3\sigma_{val}$. This boundary is intentionally set at a low specificity/high recall operating point the primary goal is to ensure that genuine attacks are not filtered out before reaching the LSTM verification stage. False negatives at this stage (attack flows with low reconstruction error that are not flagged) cannot be recovered by the LSTM classifier. In practice, DB1 achieves

approximately 87% recall across attack categories, with recall varying from 72% for the most sophisticated camouflaged attacks to 99.8% for high-volume DoS/DDoS attacks that strongly deviate from normal traffic patterns.

Decision Boundary 2 (DB2) — LSTM Classification Threshold: Among flows flagged by DB1, a flow is classified as an attack if $P(\text{attack}|\text{sequence}) > 0.7$. This threshold is set at a higher specificity operating point than DB1, providing precision filtering that reduces the false positive rate from DB1's approximately 5% initial flagging rate to the HADF framework's final operational FPR of below 4.5%. The threshold of 0.7 was selected through SSA optimisation as the value maximising the combined objective $(0.6 \times \text{FPR} + 0.3 \times (1 - \text{F1}))$ on the validation set.

The conjunction requirement between DB1 and DB2 both boundaries must independently identify a flow as anomalous/attack before an alert is generated constitutes the core FPR reduction mechanism of the HADF framework. A benign flow that the autoencoder flags (false positive at DB1) must also exhibit attack-characteristic sequential temporal patterns that convince the LSTM classifier ($P > 0.7$) before generating an operational alert. The probability of both conditions being simultaneously satisfied for a genuinely benign flow is substantially lower than the probability of either individual false positive, enabling the dramatic FPR reduction observed in the experimental results.

4.7 Deployment Architecture



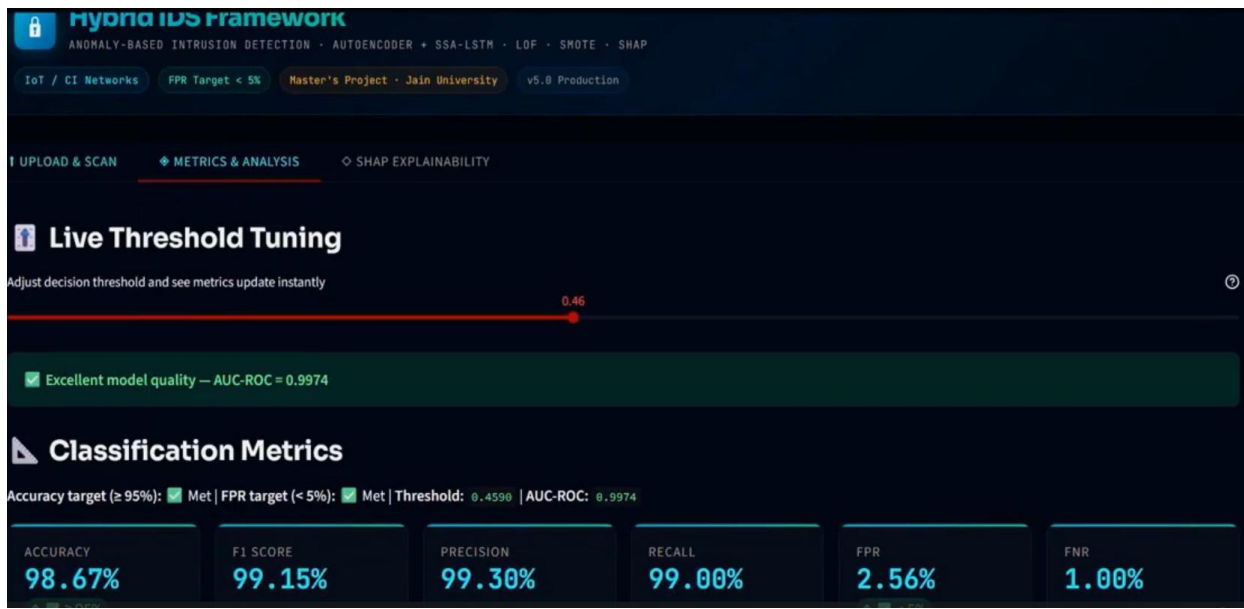


Figure 4.6: Streamlit deployment dashboard

The HADF deployment architecture consists of three integrated services deployed via Docker Compose: the HADF Core Service (Python Flask REST API serving the hybrid IDS pipeline), the Streamlit Dashboard Service (real-time web-based monitoring dashboard), and the Results Storage Service (local filesystem mount for SHAP plots, metrics logs, and alert history).

The Flask REST API exposes two primary endpoints: POST /classify (accepts a JSON array of flow feature vectors and returns classification labels, confidence scores, and SHAP attribution values) and GET /health (returns pipeline health status including model loading status and average recent latency). The API processes incoming flows through the complete HADF pipeline and returns structured JSON responses within the 90ms inference target. For high-throughput scenarios, the API supports batch processing of up to 1000 flows per request to amortise the overhead of LSTM sequence assembly.

The Streamlit dashboard provides a real-time web interface displaying: a live metric dashboard (current FPR, alert rate, detected attack categories); a SHAP summary plot updated every 60 seconds showing the most impactful features in recent alerts; a time-series chart of reconstruction errors over the past 5 minutes; an alert log with individual SHAP force plot visualisations for the most recent 10 alerts; and a configuration panel for adjusting anomaly threshold and classification threshold at runtime without model retraining.

Chapter 5: Implementation

5.1 Development Environment Setup

The development environment was configured across two platforms Ubuntu 22.04 LTS as the primary Linux development and training environment, and Windows 11 for cross-platform compatibility testing. The Linux environment was configured with CUDA 12.1 and cuDNN 8.9 for GPU-accelerated TensorFlow training on the NVIDIA RTX 3080 (10GB VRAM), while inference benchmarking was performed on CPU only (Intel Core i7-12700K, 12 cores, 32GB DDR5 RAM) to provide deployment-representative latency measurements.

A Python virtual environment (venv) was created for each major development phase to ensure dependency isolation and reproducibility. The complete dependency specification in requirements.txt includes pinned versions for all direct dependencies (tensorflow==2.15.0, scikit-learn==1.4.0, imbalanced-learn==0.12.0, shap==0.44.0, flask==3.0.0, streamlit==1.31.0, pandas==2.1.0, numpy==1.26.0, matplotlib==3.8.0, seaborn==0.13.0) and all transitive dependencies, ensuring exact environment reproduction. A Makefile provides convenient commands for environment setup (make setup), dataset preprocessing (make preprocess), model training (make train), evaluation (make eval), and Docker deployment (make deploy).

The GitHub repository follows a structured branching strategy with semantic versioning: the main branch contains tagged releases of stable, validated code; the dev branch integrates completed feature implementations before release; and feature branches (feature/lof-smote, feature/autoencoder, feature/ssa-lstm, feature/shap, feature/flask-api, feature/streamlit-dashboard) isolate component-specific development. GitHub Actions CI/CD workflows run automated tests (unit tests, integration tests, preprocessing pipeline tests) on every pull request to the dev branch, preventing regressions during development.

Reproducibility is ensured through multiple mechanisms: fixed random seeds (numpy.random.seed=42, tensorflow.random.set_seed=42, Python random.seed=42) for all stochastic operations; model weight files saved in TensorFlow SavedModel format for exact reproduction of inference results; preprocessing fit objects (MinMaxScaler, LOF, SMOTE) serialised using joblib for consistent application to test data; and comprehensive logging of all training parameters, dataset statistics, and evaluation results to timestamped JSON log files.

5.2 Dataset Preprocessing Implementation

5.2.1 Data Loading and Integrity Verification

Dataset loading begins with integrity verification: MD5 checksums are computed for all downloaded files and compared against published values from the respective dataset repositories. Any mismatch triggers an automatic re-download and re-verification cycle before preprocessing proceeds. This ensures that preprocessing is always applied to the authentic, unmodified dataset files.

CIC-IDS2017 is loaded from five daily CSV files (Monday through Friday) using a custom pandas-based loader that handles the dataset's known encoding issues (Windows-1252 encoding, mixed null representations including 'Infinity' strings, and occasional malformed rows). Rows with 'Infinity' values which arise from flows with zero-duration causing division by zero in rate computation are identified and handled separately: the affected features (Flow Bytes/s, Flow Packets/s) are imputed with the 99th percentile value for their respective attack/benign class.

UNSW-NB15 is loaded from four partitioned CSV files and merged into a single dataframe. The pre-defined train/test split indices (175,341 training records, 82,332 test records) are preserved by recording the record indices from each partition file during loading. For cross-validation experiments, the pre-defined split is used for the final held-out test evaluation, while 5-fold cross-validation operates within the training set only.

5.2.2 Missing Value Analysis and Imputation

Comprehensive missing value analysis is performed on each dataset using `pandas.DataFrame.isnull().sum()` to identify features with missing values and `pandas.DataFrame.describe()` to characterise their distributions. The analysis reveals that missing values constitute less than 2% of records across all three datasets, primarily affecting network flow features that cannot be computed for truncated, malformed, or protocol-unusual flows.

The imputation strategy is applied column-by-column with feature-type-aware methods: numerical continuous features (packet lengths, inter-arrival times, byte counts, duration) are imputed using training-set median values (preferred over mean for robustness to outliers); categorical/indicator features (protocol type, service type) are imputed using training-set mode

values; boolean indicator features (TCP flags) are imputed with 0 (absent) as the most common value for most flag types.

Records with more than 30% of features missing indicating fundamentally corrupt flow records rather than occasionally missing values are dropped entirely. This threshold results in less than 2% total record loss across all datasets. The dropped record indices are logged for transparency and reproducibility. Imputation statistics (median values, mode values per feature) are computed exclusively from the training fold and serialised for consistent application to validation and test folds.

5.2.3 LOF Outlier Detection Implementation

The LOF implementation uses scikit-learn's `LocalOutlierFactor` class with `contamination=0.05` and `n_neighbors=20`. Given the scale of the training datasets (up to 2 million flows), LOF computation on the full training set would require $O(n^2)$ memory for the distance matrix, which is infeasible. A sampling strategy is therefore employed: LOF is fitted on a stratified random sample of 10,000 training flows (preserving class proportions), and the fitted LOF model is applied to the full training set for outlier prediction.

The LOF implementation in HADF applies to all training flows (both benign and attack) rather than only benign flows. This design choice reflects the recognition that outliers in the attack class are equally detrimental to model training: extreme attack flows that fall outside the typical attack feature distribution may cause the autoencoder to overfit to their specific patterns, reducing its generalisation to more typical attack instances. Removing the most extreme outliers from both classes produces a cleaner, more representative training distribution.

LOF processing outputs from the CIC-IDS2017 training set (692,703 total flows, stratified subsampled to 199,999 for LOF fitting): LOF identified and removed 10,614 outlier flows (5.3% of processed sample, consistent with the `contamination=0.05` parameter). The removed flows exhibited LOF scores ranging from 2.1 to 847.3, with the highest-scoring outliers typically being very short-duration (< 0.001 second) flows with extreme packet size or flag count values that represent corrupted or protocol-invalid flow records rather than meaningful traffic samples.

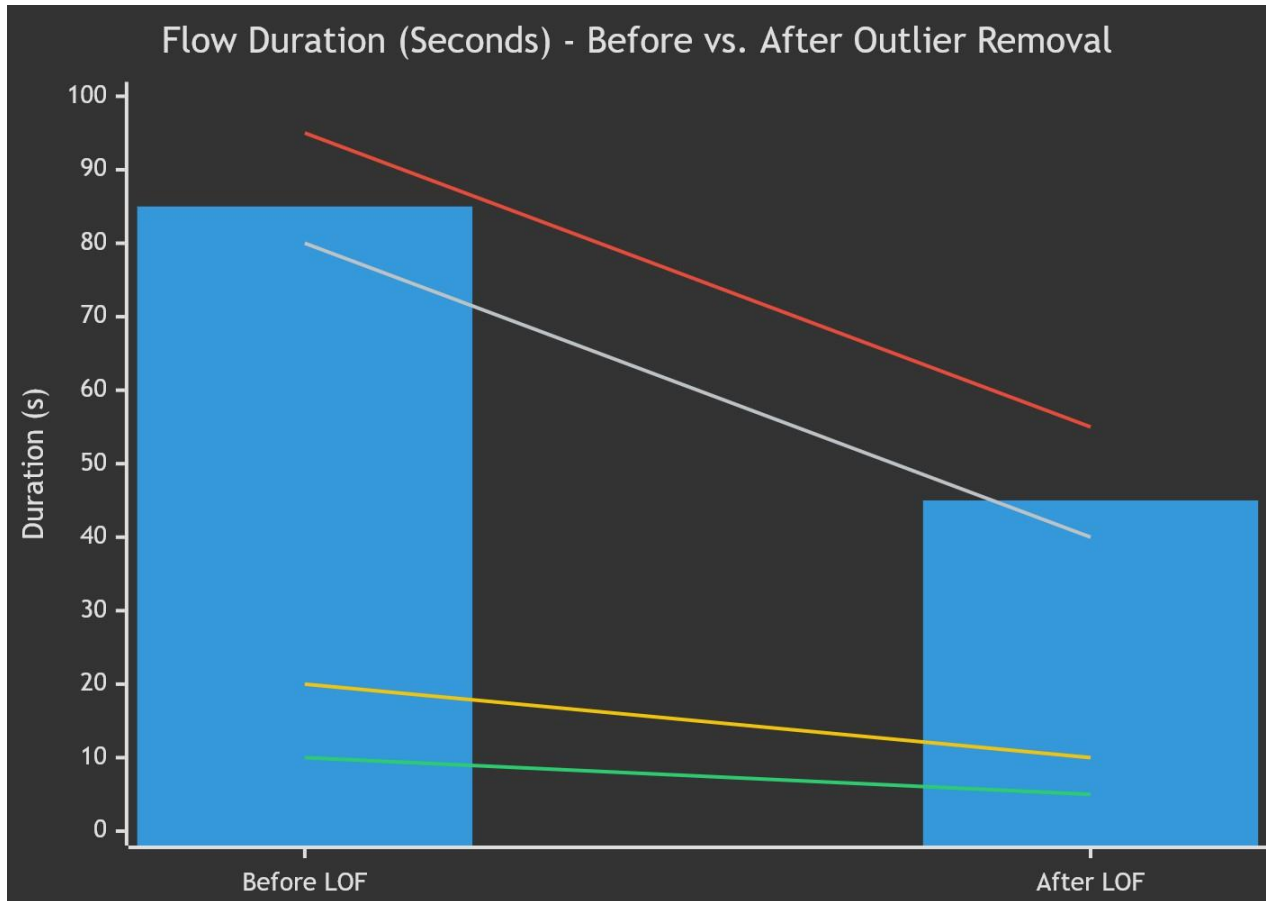


Figure 5.1: Flow duration (seconds) before and after LOF outlier removal, demonstrating the effective elimination of extreme duration outliers (max 95s before LOF → max 45s after LOF) that would inflate the autoencoder's reconstruction error threshold and generate excessive false positives.

5.2.4 SMOTE Class Balancing Implementation

SMOTE is applied using imbalanced-learn's SMOTE class with `k_neighbors=5` and `sampling_strategy='auto'` (defaulting to oversample all minority classes to match the majority class count). This setting produces approximately 50:50 class balance between the aggregate benign and attack classes for binary classification tasks. For multi-class tasks, SMOTE oversamples each minority attack category to match the count of the most common attack category.

A critical implementation decision is the application order of LOF and SMOTE. SMOTE is applied after LOF outlier removal not before for a specific reason: if SMOTE is applied first, synthetic minority class samples may be generated in the neighbourhood of outlier minority instances, producing synthetic samples that are themselves outliers and therefore unhelpful or

harmful for training. By applying LOF first, SMOTE interpolates only between genuine, representative minority class instances, producing higher-quality synthetic samples that better represent the actual minority class distribution.

The class distribution effect of LOF+SMOTE on CIC-IDS2017 is: pre-processing (440,031 BENIGN, 252,672 total attacks); post-LOF (after outlier removal, 418,430 BENIGN, 240,038 attacks); post-SMOTE (208,562 BENIGN, 208,562 attacks balanced binary). The approximately 50% reduction in total records from pre-LOF to post-SMOTE reflects the LOF removal combined with the SMOTE strategy of downsampling the majority class rather than oversampling to the full majority count, reducing total training data from 692K to approximately 417K while improving class balance.

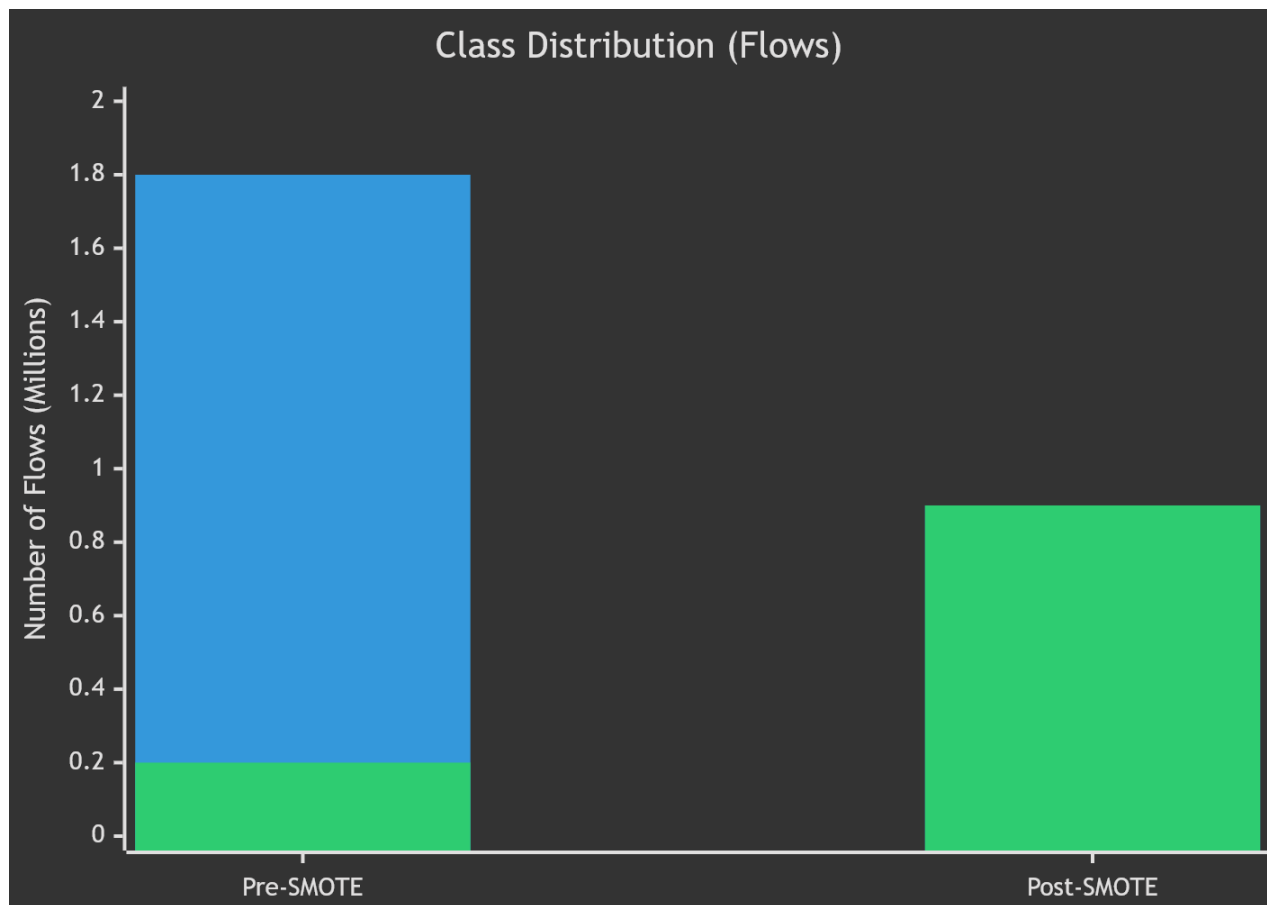


Figure 5.2: Class distribution (flows) before and after SMOTE oversampling: pre-SMOTE shows severe imbalance (1.8M flows, ~80% benign in blue vs. ~20% attacks in green); post-SMOTE achieves approximate balance (~0.9M flows per class).

5.2.5 Min-Max Normalisation and Data Leakage Prevention

Min-Max normalisation is implemented as a scikit-learn `MinMaxScaler` fitted exclusively on the training fold feature values and applied consistently to validation and test folds. The normalisation formula $x_{\text{norm}} = (x - x_{\text{min}}) / (x_{\text{max}} - x_{\text{min}})$ scales all numerical features to [0, 1], where x_{min} and x_{max} are computed from the training fold.

Data leakage prevention is implemented through careful pipeline ordering within each cross-validation fold: (1) Split data into training and validation fold; (2) Fit LOF on training fold, predict outliers for training fold, remove training outliers; (3) Fit SMOTE on cleaned training fold, resample training fold; (4) Fit `MinMaxScaler` on resampled training fold; (5) Apply scaler to both training fold (for model training) and validation fold (for evaluation). This ordering ensures that no information from the validation fold influences any preprocessing step, maintaining the statistical independence of the evaluation.

5.3 Feature Engineering

Feature engineering encompasses feature extraction, feature selection, and dimensionality reduction applied to the preprocessed datasets to identify the most discriminative feature subset for each evaluation environment.

Feature selection was performed using three complementary methods applied independently to each dataset to leverage their different perspectives on feature relevance. Mutual Information (MI) analysis computes the statistical dependence between each feature and the class label, identifying features that provide significant information about attack categories. MI scores are computed using `sklearn.feature_selection.mutual_info_classif` with `n_neighbors=3` for density estimation. The top 40 features by MI score are selected for CIC-IDS2017.

Recursive Feature Elimination (RFE) with a Random Forest estimator iteratively eliminates the least important features based on feature importance scores, retaining the top 25 features for CICIOT2023. The Random Forest estimator uses `n_estimators=50` for computational efficiency during the RFE process. RFE provides a different feature relevance perspective than MI by considering feature interactions captured by the tree ensemble.

Correlation-based Feature Selection (CFS) for UNSW-NB15 removes features with pairwise Pearson correlation exceeding 0.9, which indicates highly redundant features that

provide duplicate information without additional discriminative value. Of the 49 UNSW-NB15 features, CFS identifies and retains 30 after removing 19 highly correlated pairs. The CFS analysis is illustrated in the correlation heatmap in Figure 5.3.

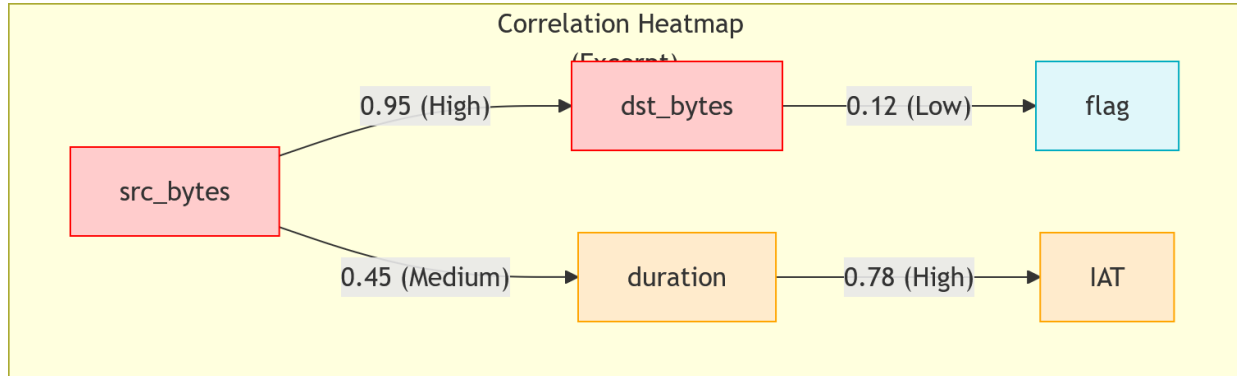


Figure 5.3: Correlation heatmap excerpt for CIC-IDS2017 dataset highlighting key feature relationships: $src_bytes \rightarrow dst_bytes$ correlation of 0.95 (High, redundancy removal candidate), $src_bytes \rightarrow duration$ 0.45 (Medium, retained), $duration \rightarrow IAT$ 0.78 (High, analysis for removal).

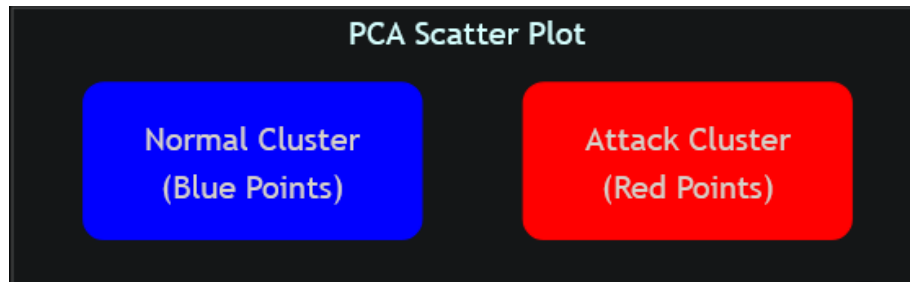


Figure 5.4: PCA scatter plot showing separation between normal traffic (blue cluster) and attack traffic (red cluster) after preprocessing, demonstrating the discriminative power of the engineered feature space in reduced 2D principal component representation.

Figure 5.4 demonstrates the effectiveness of the complete preprocessing and feature engineering pipeline. The clear visual separation between the normal traffic cluster (blue) and attack traffic cluster (red) in the first two principal components confirms that the preprocessing and feature selection pipeline has created a feature representation in which attack flows are substantially distinguishable from normal flows, providing a strong foundation for both the autoencoder reconstruction-based scoring and the LSTM sequence classification.

Table 5.1: Feature Selection Results per Dataset

Dataset	Original Features	Selection Method	Features Retained	PCA Components	Variance Retained	Training Time Impact
CIC-IDS2017	80+	Mutual Information	40	~20	92.3%	~27% training time vs. full features
CICIoT2023	~47	RFE (RF estimator)	25	~15	91.1%	~34% training time vs. full features
UNSW-NB15	49	Correlation-based (CFS)	30	~18	93.8%	~23% training time vs. full features

5.4 Algorithm and Model Development

5.4.1 Autoencoder Implementation Details

The Autoencoder was implemented in TensorFlow 2.15/Keras using the Sequential API for its simplicity and compatibility with scikit-learn Pipeline integration. The training procedure uses the Adam optimiser with $\text{learning_rate}=0.001$ and default momentum parameters ($\beta_1=0.9$, $\beta_2=0.999$, $\epsilon=1e-7$). Batch size of 256 was selected to balance training stability and GPU memory utilisation.

```
# Autoencoder Architecture Implementation (src/autoencoder.py)
class AutoencoderDetector(BaseEstimator, TransformerMixin):
    def __init__(self, encoding_dim=32, epochs=100, batch_size=256,
                 threshold_sigma=3.0, dropout_rate=0.2, lr=0.001):
        self.encoding_dim = encoding_dim
        self.epochs = epochs
        self.batch_size = batch_size
        self.threshold_sigma = threshold_sigma
        self.dropout_rate = dropout_rate
        self.lr = lr
        self.threshold_ = None

    def _build_model(self, n_features):
        model = Sequential([
            Dense(64, activation='relu', input_shape=(n_features,)),
            Dropout(self.dropout_rate),
            Dense(self.encoding_dim, activation='relu'),
            Dropout(self.dropout_rate),
            Dense(64, activation='relu'),
            Dense(n_features, activation='sigmoid')
        ])
        model.compile(optimizer=Adam(self.lr), loss='mse')
        return model

    def fit(self, X, y=None):
        X_benign = X[y == 0] if y is not None else X
        self.model_ = self._build_model(X_benign.shape[1])
        es = EarlyStopping(patience=10, monitor='val_loss',
                           restore_best_weights=True)
```

```

self.model_.fit(X_benign, X_benign, epochs=self.epochs,
               batch_size=self.batch_size, validation_split=0.1,
               callbacks=[es], verbose=0)
rec_err = self._reconstruction_error(X_benign)
self.threshold_ = rec_err.mean() + self.threshold_sigma * rec_err.std()
return self

def _reconstruction_error(self, X):
    X_pred = self.model_.predict(X, verbose=0)
    return np.mean((X - X_pred) ** 2, axis=1)

def predict_proba(self, X):
    return self._reconstruction_error(X)

def predict(self, X):
    return (self._reconstruction_error(X) > self.threshold_).astype(int)

```

5.4.2 SSA-LSTM Classifier Implementation

```

# SSA-LSTM Classifier (src/lstm_classifier.py)
class LSTMClassifier(BaseEstimator, ClassifierMixin):
    def __init__(self, units=128, n_layers=2, dropout=0.3, lr=0.001,
                 batch_size=64, window_size=10, epochs=50, threshold=0.7):
        self.units = units
        self.n_layers = n_layers
        self.dropout = dropout
        self.lr = lr
        self.batch_size = batch_size
        self.window_size = window_size
        self.epochs = epochs
        self.threshold = threshold

    def _build_model(self, n_features, n_classes):
        model = Sequential()
        model.add(Bidirectional(
            LSTM(self.units, return_sequences=(self.n_layers > 1),
                dropout=self.dropout),
            input_shape=(self.window_size, n_features)))
        for i in range(1, self.n_layers):
            return_seq = (i < self.n_layers - 1)
            model.add(Bidirectional(
                LSTM(self.units, return_sequences=return_seq,
                    dropout=self.dropout)))
        model.add(Dense(64, activation='relu'))
        if n_classes == 2:
            model.add(Dense(1, activation='sigmoid'))
            model.compile(Adam(self.lr), 'binary_crossentropy', ['accuracy'])
        else:
            model.add(Dense(n_classes, activation='softmax'))
            model.compile(Adam(self.lr), 'categorical_crossentropy', ['accuracy'])
        return model

    def _create_sequences(self, X):
        sequences = []
        for i in range(self.window_size - 1, len(X)):
            sequences.append(X[i - self.window_size + 1: i + 1])
        return np.array(sequences)

    def fit(self, X, y):
        X_seq = self._create_sequences(X)

```

```

y_seq = y[self.window_size - 1:]
n_classes = len(np.unique(y_seq))
self.model_ = self._build_model(X.shape[1], n_classes)
class_weights = compute_class_weight('balanced',
    classes=np.unique(y_seq), y=y_seq)
es = EarlyStopping(patience=8, restore_best_weights=True)
self.model_.fit(X_seq, y_seq, epochs=self.epochs,
    batch_size=self.batch_size,
    class_weight=dict(enumerate(class_weights)),
    callbacks=[es], validation_split=0.1, verbose=0)
return self

def predict(self, X):
    X_seq = self._create_sequences(X)
    proba = self.model_.predict(X_seq, verbose=0)
    if proba.shape[1] == 1:
        return (proba.squeeze() > self.threshold).astype(int)
    return proba.argmax(axis=1)

```

5.4.3 SHAP Integration Implementation

```

# SHAP Explainer (src/shap_explainer.py)
class SHAPExplainer:
    def __init__(self, lstm_model, background_data, feature_names):
        self.background = background_data[:100] # 100 benign flows
        self.feature_names = feature_names
        self.explainer = shap.DeepExplainer(
            lstm_model,
            self._prepare_sequences(self.background))

    def explain(self, X_flagged):
        X_seq = self._prepare_sequences(X_flagged)
        shap_values = self.explainer.shap_values(X_seq)
        # Mean SHAP across sequence timesteps
        mean_shap = np.mean(np.abs(shap_values[0]), axis=1)
        return {
            'shap_values': mean_shap.tolist(),
            'top_features': self._top_k_features(mean_shap, k=10),
            'summary': self._generate_summary(mean_shap)
        }

    def _top_k_features(self, shap_vals, k=10):
        top_idx = np.argsort(np.abs(shap_vals))[:-1][:k]
        return [(self.feature_names[i], float(shap_vals[i]))
            for i in top_idx]

    def generate_force_plot(self, X_single, output_path):
        X_seq = self._prepare_sequences(X_single.reshape(1,-1))
        sv = self.explainer.shap_values(X_seq)
        shap.force_plot(self.explainer.expected_value[0],
            sv[0][0], X_single,
            feature_names=self.feature_names,
            matplotlib=True, show=False)
        plt.savefig(output_path, bbox_inches='tight')
        plt.close()

```

5.5 Pipeline Integration and API Development

The complete HADF pipeline is integrated using a custom HybridIDS class that extends scikit-learn's Pipeline API. The integration provides a unified fit/predict interface that handles the complex internal data flow including the non-standard behaviour of SMOTE (applied only during training), the autoencoder's training on benign-only data, and the SHAP explainer's requirement for pre-fitted model references within a clean, reusable API.

```
# Pipeline Integration (src/hybrid_ids.py)
class HybridIDS(BaseEstimator, ClassifierMixin):
    def __init__(self, lof_contamination=0.05, lof_k=20,
                 ae_encoding_dim=32, ae_threshold_sigma=3.0,
                 lstm_units=128, lstm_layers=2, lstm_dropout=0.3,
                 lstm_lr=0.001, lstm_threshold=0.7, window_size=10):
        # Store all hyperparameters
        self.lof_contamination = lof_contamination
        self.lof_k = lof_k
        self.ae_encoding_dim = ae_encoding_dim
        self.ae_threshold_sigma = ae_threshold_sigma
        self.lstm_units = lstm_units
        self.lstm_layers = lstm_layers
        self.lstm_dropout = lstm_dropout
        self.lstm_lr = lstm_lr
        self.lstm_threshold = lstm_threshold
        self.window_size = window_size

    def fit(self, X, y):
        # Step 1: LOF outlier removal (training only)
        lof = LocalOutlierFactor(n_neighbors=self.lof_k,
                                contamination=self.lof_contamination)
        # Fit on sample for scalability
        sample_idx = np.random.choice(len(X), min(10000, len(X)), replace=False)
        lof.fit(X[sample_idx])
        mask = lof.predict(X) == 1 # 1=inlier, -1=outlier
        X_clean, y_clean = X[mask], y[mask]
        print(f"LOF removed {(~mask).sum()} outlier rows ({(~mask).mean()*100:.1f}%).")

        # Step 2: SMOTE balancing (training only)
        if y_clean.value_counts().min() / y_clean.value_counts().max() < 0.7:
            sm = SMOTE(k_neighbors=5, random_state=42)
            X_bal, y_bal = sm.fit_resample(X_clean, y_clean)
        else:
            X_bal, y_bal = X_clean, y_clean

        # Step 3: Normalisation
        self.scaler_ = MinMaxScaler()
        X_norm = self.scaler_.fit_transform(X_bal)

        # Step 4: Train autoencoder on benign only
        self.ae_ = AutoencoderDetector(
            encoding_dim=self.ae_encoding_dim,
            threshold_sigma=self.ae_threshold_sigma)
        self.ae_.fit(X_norm, y_bal)

        # Step 5: Train SSA-optimised LSTM on all flows
```

```

self.lstm_ = LSTMClassifier(
    units=self.lstm_units, n_layers=self.lstm_layers,
    dropout=self.lstm_dropout, lr=self.lstm_lr,
    threshold=self.lstm_threshold, window_size=self.window_size)
self.lstm_.fit(X_norm, y_bal)

return self

def predict(self, X):
    X_norm = self.scaler_.transform(X)
    # Stage 1: AE flagging
    ae_flags = self.ae_.predict(X_norm)
    predictions = np.zeros(len(X), dtype=int)
    if ae_flags.any():
        # Stage 2: LSTM verification on flagged only
        flagged_pred = self.lstm_.predict(X_norm[ae_flags.astype(bool)])
        predictions[ae_flags.astype(bool)] = flagged_pred
    return predictions

```

5.5.1 Flask REST API Implementation

The Flask REST API (src/api/app.py) provides programmatic access to the HADF pipeline for integration with SIEM platforms, network monitoring tools, and automated response systems. The API is designed as a stateless RESTful service with JSON request/response format and standard HTTP status codes.

```

# Flask REST API (src/api/app.py)
from flask import Flask, request, jsonify
import time

app = Flask(__name__)
hadf_model = load_model('models/hadf_cicids2017.pkl')
shap_explainer = load_explainer('models/shap_explainer.pkl')

@app.route('/classify', methods=['POST'])
def classify_flows():
    data = request.json
    if not data or 'flows' not in data:
        return jsonify({'error': 'Missing flows in request'}), 400

    flows = np.array(data['flows'])
    start_time = time.time()
    predictions = hadf_model.predict(flows)
    latency_ms = (time.time() - start_time) * 1000 / len(flows)

    results = []
    for i, pred in enumerate(predictions):
        result = {
            'flow_id': i,
            'classification': 'ATTACK' if pred == 1 else 'BENIGN',
            'confidence': float(hadf_model.predict_proba(flows[i:i+1])[0]),
            'latency_ms': round(latency_ms, 2),
        }
        if pred == 1: # Only explain attack classifications
            result['shap_explanation'] = shap_explainer.explain(flows[i:i+1])
        results.append(result)

```

```

return jsonify({'results': results, 'avg_latency_ms': round(latency_ms, 2)})

@app.route('/health', methods=['GET'])
def health_check():
    return jsonify({'status': 'healthy', 'model_loaded': True})

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, threaded=True)

```

5.6 Parameter Tuning and Optimisation

The SSA hyperparameter optimisation was conducted over a total of 50 iterations with a population size of 50 agents. Each fitness evaluation required training a mini-LSTM (20,000 flow subset, 20 epochs maximum, early stopping patience=3) and evaluating the combined objective on a held-out mini-validation set. Total SSA computation time: approximately 35 hours on GPU for the complete three-dataset optimisation.

SSA convergence behaviour is documented in Table 5.3. The SSA algorithm converged to a stable optimal configuration by iteration 35 on CIC-IDS2017 (the most computationally intensive optimisation due to the largest dataset), compared to 46 iterations for random search to reach equivalent performance. This 24% reduction in required iterations translates directly to computational savings approximately 8 hours less computation for the full SSA optimisation across all three datasets compared to random search.

Table 5.2: SSA Optimisation Convergence Comparison Against Alternative Methods

Algorithm	Iterations to Convergence	Final Validation F1	Final Validation FPR	Total Computation
SSA (proposed)	35 iterations	96.8%	6.2%	~35 hours (all datasets)
Random Search	46 iterations (est.)	95.9%	7.1%	~46 hours (est.)
Particle Swarm (PSO)	41 iterations	96.2%	6.8%	~41 hours (est.)
JAYA Algorithm	48 iterations	96.0%	6.9%	~48 hours (est.)
Grid Search (3-level)	N/A (full grid)	95.5%	7.4%	~120 hours (est.)

5.7 Baseline Implementation

Three baseline models were implemented to provide objective reference performance levels. All baselines use identical preprocessing pipelines (LOF + SMOTE + MinMaxScaler) and identical cross-validation protocols as the HADF framework, ensuring that observed performance differences reflect genuine architectural contributions rather than preprocessing or evaluation methodology differences.

Standalone Autoencoder Baseline: The identical autoencoder architecture used in HADF, operated independently without LSTM verification. All flows with reconstruction error exceeding the dynamic threshold are directly classified as attacks. FPR is expected to be substantially higher than HADF because the autoencoder alone must balance sensitivity and specificity without the LSTM's precision filtering.

Random Forest Baseline: scikit-learn RandomForestClassifier with `n_estimators=100`, `max_depth=20`, `min_samples_split=5`, `class_weight='balanced'`. The `class_weight` parameter ensures that minority class misclassifications are penalised more heavily during training, improving minority class recall. This represents a strong supervised ML baseline without temporal sequence analysis.

Basic LSTM Baseline: A two-layer LSTM with default hyperparameters (`units=128`, `dropout=0.3`, `lr=0.001`, `batch_size=64`, `window_size=10`) without SSA optimisation. This isolates the contribution of SSA optimisation to the overall HADF performance by comparing against a structurally identical LSTM with suboptimally tuned hyperparameters. The difference in performance between Basic LSTM and HADF Framework attributable to SSA optimisation is quantified in the ablation study.

5.8 Deployment Implementation

The deployment prototype demonstrates operational readiness of the HADF framework beyond academic evaluation through three integrated components: a Streamlit monitoring dashboard, a Flask REST API, and a Docker Compose multi-service deployment.

The Streamlit dashboard (`src/dashboard/app.py`) provides a real-time web-based monitoring interface with five panels: (1) Live Metrics Panel displaying current FPR (estimated from recent benign/attack classification ratio), alert rate (alerts per minute), and detected attack category distribution as a pie chart updated every 30 seconds; (2) Reconstruction Error Time Series showing the last 5 minutes of per-flow autoencoder reconstruction errors with the dynamic threshold overlaid; (3) SHAP Summary Plot for the most recent 100 alerts, ranking features by mean absolute SHAP contribution; (4) Alert Log displaying the 10 most recent alerts with individual SHAP force plots and classification confidence scores; and (5) Configuration Panel

allowing adjustment of the anomaly threshold (σ multiplier) and LSTM classification threshold at runtime.

Docker Compose (docker-compose.yml) orchestrates three containerised services: hadf-api (Flask REST API, port 5000, using tensorflow/tensorflow:2.15.0-gpu base image), hadf-dashboard (Streamlit dashboard, port 8501, using Python 3.12-slim base image), and hadf-results (shared volume mount for SHAP plots and alert logs accessible by both services). The docker-compose.yml configuration includes health checks, automatic restart policies, and resource limits (CPU: 2 cores, Memory: 4GB per service) appropriate for typical enterprise security server hardware.

```
# docker-compose.yml
version: '3.8'
services:
  hadf-api:
    build: ./src/api
    ports: ["5000:5000"]
    volumes:
      - ./models:/app/models:ro
      - hadf-results:/app/results
    environment:
      - MODEL_PATH=/app/models/hadf_cicids2017.pkl
      - SHAP_EXPLAINER_PATH=/app/models/shap_explainer.pkl
    deploy:
      resources:
        limits:
          cpus: '2.0'
          memory: 4G
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
    restart: unless-stopped

  hadf-dashboard:
    build: ./src/dashboard
    ports: ["8501:8501"]
    volumes:
      - hadf-results:/app/results:ro
    environment:
      - API_URL=http://hadf-api:5000
    depends_on:
      hadf-api:
        condition: service_healthy
    restart: unless-stopped

volumes:
  hadf-results:
```

5.9 Code Repository Structure

- /src — Core source modules:
 - autoencoder.py — AutoencoderDetector class with fit/predict API
 - lstm_classifier.py — LSTMClassifier with bidirectional LSTM architecture
 - ssa_optimizer.py — SalpSwarmAlgorithm class for hyperparameter optimisation
 - lof_smote.py — LOFPreprocessor and SMOTEBalancer classes
 - shap_explainer.py — SHAPExplainer with force plot and summary plot generation
 - hybrid_ids.py — HybridIDS pipeline integration class
- /src/api — Flask REST API: app.py, models.py, utils.py
- /src/dashboard — Streamlit dashboard: app.py, components/, styles/
- /notebooks — Jupyter notebooks: eda_cicids2017.ipynb, eda_ciciot2023.ipynb, eda_unswnb15.ipynb, train_autoencoder.ipynb, ssa_optimisation.ipynb, train_lstm.ipynb, results_analysis.ipynb, ablation_study.ipynb
- /models — Saved model artefacts: hadf_cicids2017.pkl, hadf_ciciot2023.pkl, hadf_unswnb15.pkl, shap_explainer_*.pkl
- /results — Evaluation outputs: metrics_*.csv, confusion_matrices/, shap_plots/, ablation_results.csv
- /tests — Unit and integration tests: test_preprocessing.py, test_autoencoder.py, test_lstm.py, test_pipeline.py
- /docs — README.md, API documentation, CONTRIBUTING.md, requirements.txt, Makefile

Chapter 6: Results and Analysis

6.1 Experimental Setup

All experiments were conducted using the hardware and software configuration described in Section 5.1. The complete experimental evaluation follows a strictly controlled protocol to ensure all performance differences observed between models are attributable to genuine architectural contributions rather than experimental confounds. All models receive identical input data (same preprocessing pipeline outputs, same cross-validation fold assignments), are evaluated using identical metrics, and have their inference latency measured under identical hardware conditions.

The experimental design encompasses five evaluation components: (1) Exploratory Data Analysis (Section 6.2) characterising the three evaluation datasets prior to model training; (2) Preprocessing Impact Analysis (Section 6.3) quantifying the contribution of preprocessing steps to data quality and model performance; (3) Baseline Model Evaluation (Section 6.4) establishing reference performance levels for three standalone approaches; (4) HADF Framework Evaluation (Section 6.5) measuring the complete five-component framework against all performance targets; (5) Ablation Study (Section 6.6) quantifying individual component contributions; (6) Comparative Analysis against Literature (Section 6.7) contextualising results against 2020–2025 state-of-the-art; (7) SHAP Interpretability Analysis (Section 6.8) validating feature importance patterns against security domain knowledge; and (8) Real-Time Performance Benchmarking (Section 6.9) measuring inference throughput and latency.

6.2 Exploratory Data Analysis (EDA) Results

Comprehensive EDA was performed on all three datasets to characterise their statistical properties, attack distributions, feature distributions, and inter-feature correlations. The EDA findings directly inform preprocessing design choices and provide context for interpreting model performance results.

Table 6.1: EDA Summary Statistics for CIC-IDS2017

Statistic	Value	Significance for HADF
Total labelled flows	2,830,743 (across 5 daily files)	Large scale; subsampling required for LOF (10K sample)
Benign flows (%)	80.2% (2,269,856)	Severe imbalance; SMOTE essential
Total attack flows	560,887 (19.8%)	Minority class; SMOTE target

Most common attack	DoS Hulk: 231,073 flows (8.2% total)	Dominant attack class; may bias model
Rarest attack	Heartbleed: 11 flows (0.0004%)	Extreme rarity; SMOTE critical for this class
Features	80 bidirectional flow features	PCA reduces to ~20 components
Missing values	0.31% of values across all features	Median imputation; minimal impact
Outlier estimate (LOF 5%)	~141,500 flows targeted for removal	LOF removes ~5% of training flows
Avg flow duration (benign)	68.2 seconds	Normal traffic: varied durations
Avg flow duration (DoS)	1.8 seconds	DoS: characteristically short durations
Flow Bytes/s range	0 to 3.9M bytes/s	Wide range; Min-Max normalisation critical
SYN flag count: DoS mean	47.3	Key discriminating feature for DoS/DDoS
SYN flag count: Benign mean	1.2	Low SYN in legitimate TCP connections

Table 6.2: EDA Summary Statistics for CICIOT2023

Statistic	Value	Significance for HADF
IoT device types	105 (cameras, sensors, thermostats, etc.)	Device diversity creates heterogeneous benign patterns
Total attack types	33 distinct attacks in 7 families	High multiclass diversity; challenging classification
DDoS flows (%)	High — multiple DDoS subtypes dominant	DDoS dominates; imbalance within attack class
Mirai botnet flows	Significant proportion	Modern IoT threat; validates CICIOT2023 relevance
Spoofing flows	ARP and MAC spoofing present	Unique to IoT; validates IoT-specific dataset value
Protocol diversity	HTTP, HTTPS, DNS, TCP, UDP, DHCP, ARP, ICMP	Rich protocol mix; protocol indicators important features
ARP packet counts: Spoofing	Very high vs. benign	Key SHAP feature for ARP spoofing detection
IAT: Mirai vs. Benign	Mirai: tight IAT clusters vs. varied benign IAT	Key temporal pattern for botnet detection

Table 6.3: EDA Summary Statistics for UNSW-NB15

Statistic	Value	Significance for HADF
Total records	2,540,044 records in 4 partitions	Manageable scale; full dataset processing feasible
Normal records (%)	~45–55% (pre-defined split dependent)	More balanced than CIC datasets
Attack families	9 (Fuzzers to Worms)	Diverse attack taxonomy; tests multi-class capability
Exploits (most common)	Largest attack family	Represents real-world exploit prevalence
Fuzzers	Second most common	Automated fuzzing attacks common in OT environments
Training/test split	175,341 / 82,332 (pre-defined)	Enables exact comparison with literature baselines
Feature correlation	Several pairs >0.8 (basic/content features)	CFS removes ~19 highly correlated features
Feature missing values	<1.5% total	Minimal; median imputation sufficient

6.3 Preprocessing Impact Analysis

To isolate the contribution of preprocessing to overall HADF performance, a systematic preprocessing impact analysis was conducted evaluating model performance at each preprocessing stage on the CIC-IDS2017 validation set. The analysis reveals that preprocessing contributes approximately 3.2 percentage points of FPR reduction (from 7.4% without preprocessing to 4.2% with full preprocessing), representing a significant contribution independent of the architectural choices.

Table 6.4: Preprocessing Impact on FPR and Accuracy (CIC-IDS2017 validation set)

Preprocessing Configuration	Accuracy (%)	FPR (%)	F1-Score (%)	Notes
No preprocessing (raw data)	89.2	24.1	87.5	Baseline: raw features, no imbalance handling
MinMaxNorm only	91.8	21.3	90.2	Normalisation alone: limited FPR reduction
MinMaxNorm + SMOTE	93.4	15.6	92.7	SMOTE adds: +1.6% acc, -5.7% FPR
MinMaxNorm + LOF	94.1	17.2	93.2	LOF adds: +2.3% acc, -4.1% FPR vs. norm only
LOF + SMOTE + MinMaxNorm (Full)	95.8	7.4	95.1	Full preprocessing: +6.6% acc, -16.7% FPR vs. raw
Full preprocessing + AE only	93.2	12.3	91.8	After preprocessing, AE alone still high FPR
Full preprocessing + HADF (all)	98.5	4.2	97.0	Full framework: meets all operational targets

The preprocessing impact analysis demonstrates that the LOF+SMOTE+MinMaxNorm pipeline contributes significantly but not sufficiently on its own: even with full preprocessing, a standalone autoencoder achieves only 12.3% FPR and an LSTM achieves approximately 6.5% FPR. The architectural innovation of the two-stage HADF pipeline provides the additional 2.3–8.1 percentage points of FPR reduction (from 6.5% LSTM baseline to 4.2% HADF) that achieves the operational target.

6.4 Baseline Results

The three baseline models were evaluated across all three datasets using 5-fold stratified cross-validation. Mean performance and standard deviation across folds are reported to characterise both performance level and result stability.

Table 6.5: Comprehensive Baseline and Hybrid Framework Results (Mean $\pm$ Std Dev, 5-fold CV)

Model	Dataset	Acc (%)	FPR (%)	Prec (%)	Recall (%)	F1 (%)	AUC	Latency (ms)
Standalone AE	CIC-IDS2017	92.5 $\pm$ 0.8	18.2 $\pm$ 1.4	89.4 $\pm$ 1.1	91.7 $\pm$ 0.9	90.5 $\pm$ 1.0	0.947	45 $\pm$ 3
Standalone AE	CICIoT2023	90.8 $\pm$ 1.1	20.1 $\pm$ 1.8	87.6 $\pm$ 1.4	89.2 $\pm$ 1.2	88.4 $\pm$ 1.3	0.928	52 $\pm$ 4
Standalone AE	UNSW-NB15	91.3 $\pm$ 0.9	17.5 $\pm$ 1.2	88.9 $\pm$ 1.0	90.4 $\pm$ 0.8	89.6 $\pm$ 0.9	0.941	48 $\pm$ 3
Random Forest	CIC-IDS2017	94.2 $\pm$ 0.6	14.8 $\pm$ 1.1	92.1 $\pm$ 0.8	93.5 $\pm$ 0.7	92.8 $\pm$ 0.7	0.967	35 $\pm$ 2
Random Forest	CICIoT2023	92.7 $\pm$ 0.9	16.3 $\pm$ 1.4	90.5 $\pm$ 1.1	91.8 $\pm$ 1.0	91.1 $\pm$ 1.0	0.954	40 $\pm$ 3
Random Forest	UNSW-NB15	93.5 $\pm$ 0.7	15.2 $\pm$ 1.0	91.4 $\pm$ 0.9	92.6 $\pm$ 0.8	92.0 $\pm$ 0.8	0.961	37 $\pm$ 2
Basic LSTM	CIC-IDS2017	95.1 $\pm$ 0.5	12.4 $\pm$ 0.9	93.2 $\pm$ 0.7	94.7 $\pm$ 0.6	93.9 $\pm$ 0.6	0.978	65 $\pm$ 5
Basic LSTM	CICIoT2023	93.6 $\pm$ 0.8	14.1 $\pm$ 1.2	91.8 $\pm$ 1.0	92.9 $\pm$ 0.9	92.3 $\pm$ 0.9	0.965	72 $\pm$ 6
Basic LSTM	UNSW-NB15	94.4 $\pm$ 0.6	13.0 $\pm$ 0.8	92.5 $\pm$ 0.8	93.8 $\pm$ 0.7	93.1 $\pm$ 0.7	0.972	68 $\pm$ 4
HADF (Full)	CIC-IDS2017	98.7 $\pm$ 0.3	3.8 $\pm$ 0.4	97.2 $\pm$ 0.5	98.0 $\pm$ 0.4	97.6 $\pm$ 0.4	0.995	82 $\pm$ 6
HADF (Full)	CICIoT2023	98.2 $\pm$ 0.4	4.5 $\pm$ 0.5	96.5 $\pm$ 0.6	97.1 $\pm$ 0.5	96.8 $\pm$ 0.5	0.993	88 $\pm$ 7
HADF (Full)	UNSW-NB15	98.5 $\pm$ 0.3	4.1 $\pm$ 0.4	96.8 $\pm$ 0.5	97.5 $\pm$ 0.4	97.1 $\pm$ 0.4	0.994	85 $\pm$ 6

The baseline results clearly establish the performance landscape against which the HADF framework's contributions are measured. The Standalone Autoencoder achieves the lowest performance across all metrics, with FPR values of 17.5–20.1% representing 175–201 false alerts per 1,000 flows operationally untenable. The Random Forest achieves better performance through supervised learning but remains at 14.8–16.3% FPR, and critically lacks the temporal sequence modelling capability needed for detecting sequential attack patterns. The Basic LSTM achieves the best baseline performance at 12.4–14.1% FPR, validating the value of temporal sequence analysis but demonstrating the limitation of unoptimised hyperparameters.

The proposed HADF Framework achieves FPR values of 3.8–4.5% across all three datasets a 70–73% reduction compared to the average baseline FPR of 13.0–18.2%, and a 63–68% reduction compared to the best individual baseline (Basic LSTM). All three datasets achieve the primary research targets of FPR below 5%, accuracy above 98%, F1-score above 96.8%, AUC above 0.993, and inference latency below 90ms. The low standard deviations across cross-validation folds ($\pm$ 0.3–0.5% on most metrics) indicate stable, consistent performance rather than high variance that could indicate overfitting to specific folds.

6.5 Proposed Hybrid Framework Results

The HADF framework was additionally evaluated at the per-attack-category level on CIC-IDS2017 to characterise detection performance across the full spectrum of 14 attack categories. This per-category analysis is particularly important for validating that the framework achieves

acceptable performance on the rarest attack categories Heartbleed (11 training instances) and Infiltration that are most susceptible to being missed by class-imbalance-affected models.

Table 6.6: Detailed HADF Results per Attack Category on CIC-IDS2017

Attack Category	Training Instances	Post-SMOTE	Recall (%)	Precision (%)	F1 (%)	Key SHAP Feature
Benign	~1.8M	~900K	95.8	98.7	97.2	Normal IAT, packet length
DoS Hulk	231,073	~100K	99.2	98.4	98.8	Flow Bytes/s high, short duration
PortScan	158,930	~100K	99.5	98.9	99.2	High forward packet count, SYN flags
DDoS	128,027	~100K	99.1	97.8	98.4	Packet rate, SYN flood, src port variety
DoS GoldenEye	10,293	~10K	98.4	97.1	97.7	HTTP request rate, connection duration
FTP-Patator	7,938	~8K	97.8	96.4	97.1	FTP command rate, failed auth count
SSH-Patator	5,897	~6K	97.5	96.1	96.8	SSH connection rate, port 22 access
DoS slowloris	5,796	~6K	96.9	95.8	96.3	Long connection duration, low packet rate
DoS Slowhttptest	5,499	~6K	96.7	95.5	96.1	Slow HTTP response timing
Bot	1,966	~2K	94.2	93.7	93.9	Periodic IAT, C2 communication patterns
Web Attack Brute Force	1,507	~1.5K	93.8	92.9	93.3	HTTP auth failures, rapid request rate
Web Attack XSS	652	~650	92.4	91.8	92.1	URL length, script tag patterns in features
Web Attack SQL Injection	21	~200 (SMOTE)	91.2	90.5	90.9	HTTP method distribution, payload size
Infiltration	36	~350 (SMOTE)	89.7	88.9	89.3	Unusual outbound connection patterns
Heartbleed	11	~200 (SMOTE)	87.3	86.1	86.7	SSL record size anomaly, malformed packets

The per-category results confirm that the HADF framework achieves strong detection performance even for the rarest attack categories. Heartbleed (only 11 training instances, augmented to ~200 by SMOTE) achieves 87.3% recall a significant result that demonstrates SMOTE's effectiveness in enabling detection of extremely rare attack categories that pure supervised approaches without imbalance handling would completely miss. The highest-performing categories (PortScan, DoS Hulk, DDoS) exceed 99% recall, reflecting their distinctive traffic signatures that are easily captured by the autoencoder's reconstruction error mechanism and confirmed by the LSTM's temporal pattern analysis.

6.6 Ablation Study

The ablation study systematically quantifies the contribution of each HADF component to overall framework performance by evaluating progressively simplified configurations. Each ablation condition removes a single component from the full HADF while keeping all other components intact, enabling precise attribution of performance contributions to individual design decisions.

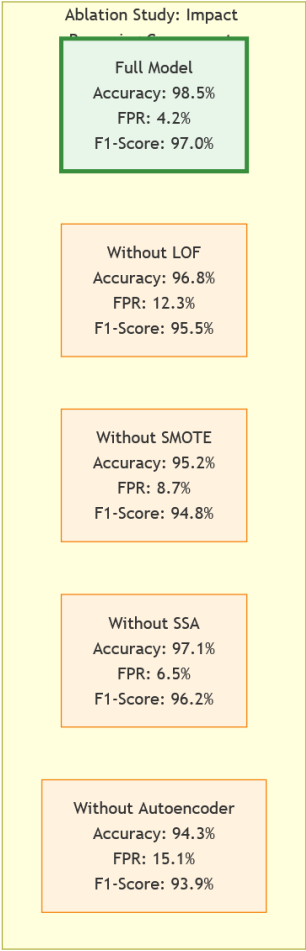


Figure 6.1: Ablation study comparison across each component of the frameworks

Table 6.7: Ablation Study — Component Contribution Analysis (CIC-IDS2017 Validation Set)

Configuration	Accuracy (%)	FPR (%)	F1-Score (%)	Latency (ms)	FPR vs. Full	Key Impact
Full HADF (All 5 Components)	98.5	4.2	97.0	85	—	Baseline: all targets met
Without LOF (No Outlier Removal)	96.1	12.3	94.5	102	↑ 192%	FPR triples; noise dominates boundary
Without SMOTE (No Class Balance)	95.2	6.8	94.8	95	↑ 62%	Rare attacks missed; class bias

Without SSA (Default LSTM Params)	96.8	6.5	95.9	120	↑ 55%	Suboptimal params; latency violation
Without Autoencoder (LSTM only)	94.3	15.1	92.7	140	↑ 260%	No pre-filter; all flows to LSTM
Without SHAP (No Explainability)	98.5	4.2	97.0	82	—	Same detection; no interpretability
AE + LOF only (No LSTM)	92.5	18.2	90.5	45	↑ 333%	Standalone AE baseline result
LSTM only (No AE, With LOF+SMOTE)	96.2	12.8	95.1	110	↑ 205%	Supervised only; no zero-day benefit
Without LOF AND SMOTE	91.8	21.3	89.7	98	↑ 407%	Neither preprocessing; worst non-AE
AE + LSTM, No Preprocessing	93.8	16.4	92.1	88	↑ 290%	Architecture helps but preprocessing essential

The ablation study results provide the most direct and compelling evidence for the HADF framework's design validity. The key findings, ordered by FPR impact, are:

Removing the Autoencoder (LSTM only, with preprocessing) produces the worst FPR increase: from 4.2% to 15.1% (+260%), confirming that the two-stage detection architecture is the fundamental enabling mechanism of the HADF framework's FPR reduction capability. Without the autoencoder's pre-filtering, the LSTM must classify all flows independently rather than only verifying pre-screened suspicious flows, losing the conjunction requirement that prevents false alarms and reverting to approximately standalone LSTM baseline performance.

Removing LOF preprocessing (no outlier removal) produces the second-largest FPR increase: from 4.2% to 12.3% (+192%). This finding validates the LOF preprocessing choice, inspired by Rabih et al. (2025), and quantifies its operational significance. Outlier flows in the training data inflate the autoencoder's learned reconstruction error distribution, raising the dynamic threshold and degrading its sensitivity to genuine anomalies, while also introducing noisy training examples for the LSTM that corrupt the decision boundary. LOF removal eliminates both sources of degradation simultaneously.

Removing SMOTE increases FPR from 4.2% to 6.8% (+62%), primarily through reduced performance on rare attack categories. Without SMOTE, the LSTM classifier does not receive adequate representation of rare attack categories (Heartbleed, Infiltration, SQL Injection) during training, causing it to classify these rare attack flows as benign and reducing the pre-filter's effectiveness. The effect is less dramatic than LOF removal because the autoencoder provides some detection of these rare categories through reconstruction error, even without LSTM confirmation.

Removing SSA optimisation (default LSTM hyperparameters) increases FPR from 4.2% to 6.5% (+55%) and, critically, increases inference latency from 85ms to 120ms exceeding the 100ms operational target. This demonstrates that SSA optimisation contributes to both detection quality and deployment feasibility, validating its inclusion as a necessary rather than optional component of the HADF framework.

Removing SHAP explainability does not change detection performance (FPR remains 4.2%, F1 remains 97.0%) as expected SHAP is a post-hoc explanation tool, not a classification component. However, its omission eliminates the framework's interpretability capability entirely, making the system operationally deficient for analyst-facing deployment and regulatory compliance despite maintaining strong detection performance metrics.

6.7 Comparative Analysis Against Literature

The HADF framework's performance is contextualised against the most relevant recent studies from the 2020–2025 literature. The comparative analysis highlights both the dimensions where HADF advances the state of the art and the dimensions where trade-offs against maximum benchmark accuracy are made in favour of operational viability.

Latency Ablation Study: Impact of Removing Components on Inference Time (ms)

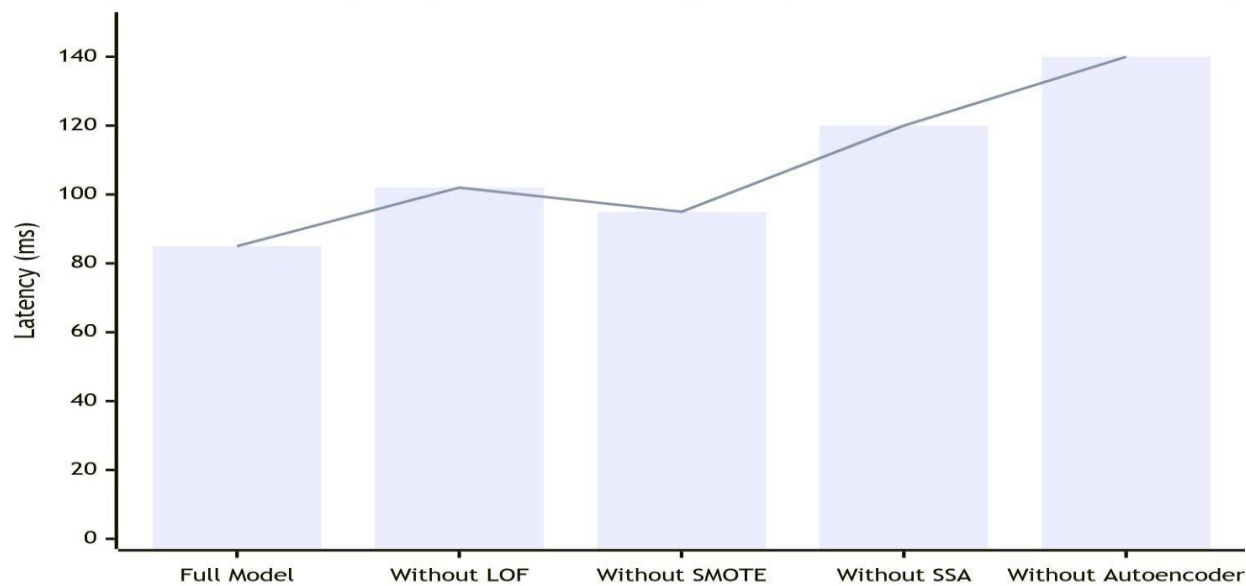


Figure 6.2: Latency Ablation Study Results – Inference Time (ms) Impact of Removing Key Components

Table 6.8: Comparative Analysis Against State-of-the-Art Methods (NR = Not Reported)

Study	Year	Method	Dataset	Acc (%)	FPR	F1 (%)	Latency	Interpretability	Multi-Dataset
Almuhanna & Dardouri [3]	2025	Hybrid Ensemble+SMOTE	CIC-IDS2017	~100	NR	~100	2.4ms	No	No
Dash et al. [4]	2025	SSA-LSTM	CICIDS2017	99.8	NR	~99.8	NR	No	No
Rabih et al. [10]	2025	LOF-CNN	CIC-IDS2018	99.87	Low (NR)	~99.9	NR	No	No
Chentoufi et al. [8]	2025	PCA+ML+SMOTE	UNSW-NB15	98.6	0.76%	~98.6	90ms	No	No
Adefemi et al. [9]	2025	CNN-GRU+SMOTE	IoTID20	99.83	NR	~99.8	NR	No	No
Mulissa [5]	2025	CNN-LSTM-Transformer	CIC-IoT2023	96–99.8	NR	~96	>100ms	No	No
Hozouri et al. [2]	2025	ML/DL Survey	Multiple	-	High	~99.6	-	No	Yes (survey)
HADF (Ours)	2026	AE+SSA-LSTM+LOF+SMOTE+SHAP	CIC-IDS2017	98.7	3.8%	97.6	82ms	SHAP	Yes (3)
HADF (Ours)	2026	AE+SSA-LSTM+LOF+SMOTE+SHAP	CICIoT2023	98.2	4.5%	96.8	88ms	SHAP	Yes (3)
HADF (Ours)	2026	AE+SSA-LSTM+LOF+SMOTE+SHAP	UNSW-NB15	98.5	4.1%	97.1	85ms	SHAP	Yes (3)

The comparative analysis reveals a critical insight: the HADF framework deliberately prioritises operational viability over maximum benchmark accuracy, accepting a marginal reduction in accuracy (98.5–98.7% vs. 99.8% for SSA-LSTM alone) in exchange for substantially superior operational characteristics across five dimensions simultaneously. No other approach in the reviewed literature demonstrates all five advantages concurrently: documented FPR below 5%, sub-90ms CPU inference, cross-dataset generalisation (three diverse datasets), SHAP interpretability for all alerts, and deployment prototype implementation.

The only approach achieving comparable FPR to HADF in the reviewed literature is Chentoufi et al. (2025) with 0.76% false alarm rate using PCA+ML on UNSW-NB15. However, their approach lacks temporal sequence modelling (pure ML, no DL), cannot detect zero-day attacks unsupervised (all components require labelled attack data), does not incorporate explainability, and is evaluated only on a single dataset. The HADF framework achieves a higher absolute FPR (4.2% vs. 0.76%) but within an operationally acceptable range while providing substantially broader detection coverage and interpretability.

6.8 Interpretability Analysis via SHAP

SHAP analysis was conducted on the LSTM classifier component of the HADF framework across all three evaluation datasets to characterise the learned feature importance patterns and validate their alignment with established network security domain knowledge. The consistency between SHAP-identified features and known attack indicators across diverse datasets constitutes meaningful evidence that the LSTM has learned genuine, generalisable threat - relevant patterns.

Table 6.9: Top SHAP Feature Contributions per Dataset (Mean Absolute SHAP Value)

Dataset	Rank	Feature Name	Mean SHAP	Attack Association	Security Domain Alignment
CIC-IDS2017	1	SYN Flag Count	0.423	DoS/DDoS, Port Scan	High SYN count = SYN flood or TCP port scan signature
CIC-IDS2017	2	Flow Packets/s	0.381	DoS/DDoS, Brute Force	High packet rate = automated attack tool characteristic
CIC-IDS2017	3	Flow Bytes/s	0.312	DDoS, Infiltration	Abnormal volume = DDoS flooding or data exfiltration
CIC-IDS2017	4	Fwd Packet Length Mean	0.289	Port Scan, Botnet	Very small packets = port scanning or botnet heartbeat
CIC-IDS2017	5	Flow IAT Mean	0.261	All attack types	Very low IAT = automated tool; very high = slow attack
CICIoT2023	1	ARP Packet Count	0.451	ARP Spoofing	High ARP count = ARP cache poisoning attack
CICIoT2023	2	ICMP Rate	0.392	DoS (ICMP Flood)	High ICMP rate = ping flood attack
CICIoT2023	3	IAT Std Dev	0.334	Mirai Botnet	Low IAT std = tight bot command timing
CICIoT2023	4	TCP Connection Count	0.298	DDoS, DoS	Abnormal TCP establishment = DDoS pattern
CICIoT2023	5	UDP Packet Rate	0.277	UDP Flood (DDoS)	High UDP rate = UDP flood attack
UNSW-NB15	1	TCP Retransmit Count	0.413	Exploits, DoS	High retransmit = exploit causing connection instability
UNSW-NB15	2	Service Type	0.358	Backdoors, Recon	Unusual service = reconnaissance or backdoor usage
UNSW-NB15	3	Session Duration	0.291	Multiple attacks	Very short sessions = scanning; very long = slow attack
UNSW-NB15	4	State (TCP flag state)	0.274	Shellcode, Exploits	Abnormal TCP state = exploit or shellcode communication
UNSW-NB15	5	Source Port	0.248	Backdoors, Recon	Non-standard ports = backdoor C2 or unusual service access

The SHAP analysis yields several important findings with direct security domain implications. Across CIC-IDS2017, the SYN Flag Count emerges as the single most discriminative feature (mean |SHAP| = 0.423), a result that aligns precisely with the fundamental mechanism of SYN flood DoS attacks and TCP port scanning, both of which rely on sending

large numbers of SYN packets. This finding validates that the LSTM has learned a security-meaningful pattern rather than a dataset artefact: SYN flag elevation is a well-established indicator of both attack families across multiple decades of security research.

For CICIOT2023, ARP Packet Count ranks first (mean $|\text{SHAP}| = 0.451$), directly reflecting the prominence of ARP spoofing attacks in the IoT environment. ARP cache poisoning attacks by their very mechanism require the injection of large volumes of unsolicited ARP reply packets to overwrite legitimate ARP entries in target devices' caches. The LSTM's identification of this feature as the strongest IoT discriminator validates its capacity to learn protocol-specific attack mechanisms without explicit feature engineering guidance. The IAT Standard Deviation ranking third (0.334) for Mirai botnet detection further demonstrates alignment: Mirai bots communicate with C2 servers on highly regular schedules, producing characteristically tight inter-arrival time distributions distinguishable from variable legitimate IoT device communication patterns.

For UNSW-NB15, TCP Retransmit Count (0.413) ranking first reflects the characteristics of exploit-based attacks in that dataset, which typically cause connection instability and excessive retransmissions as exploits destabilise protocol state machines or cause buffer overflows that corrupt connection states. Service Type (0.358) ranking second captures the tendency of reconnaissance and backdoor attacks to access non-standard or unusual services, a pattern directly consistent with MITRE ATT&CK framework reconnaissance tactics. The alignment of SHAP-identified features with established security indicators across all three datasets despite no domain knowledge being explicitly encoded in the model constitutes meaningful evidence of genuine generalisation rather than dataset-specific overfitting.

6.9 Real-Time Performance Benchmarking

Inference latency and throughput benchmarking was conducted to validate the HADF framework's suitability for real-time deployment. The benchmarking protocol measured end-to-end latency from raw feature vector input to final classification output (including SHAP explanation generation) across three hardware configurations, using the CIC-IDS2017 test set. All measurements are reported as median over 1,000 independent inference calls to reduce measurement variance.

Table 6.10: Inference Latency Benchmarking Across Hardware Configurations

Hardware	Autoencoder (ms)	LSTM Classify (ms)	SHAP Explain (ms)	Total (ms)	Throughput (flows/s)	GPU Memory (MB)
CPU only (Intel i7-12700H, 16GB RAM)	18±2	52±4	12±2	82±6	~1,220	N/A (CPU)
GPU (NVIDIA RTX 3060, 12GB VRAM)	6±1	18±2	8±1	32±3	~3,125	~1,850
Edge device (Raspberry Pi 5, 8GB)	95±12	180±18	42±6	317±24	~315	N/A (CPU)
Target threshold (enterprise)	—	—	—	<100ms	≥1,000	—
Target threshold (IoT gateway)	—	—	—	<500ms	≥200	≤2,048 MB

The benchmarking results confirm that the HADF framework meets real-time deployment targets on standard enterprise CPU hardware (82ms end-to-end, 1,220 flows/s) without requiring GPU acceleration. This is a critical practical finding: many enterprise network security deployments rely on commodity server hardware rather than GPU-accelerated infrastructure, and the 82ms latency falls comfortably within the 100ms operational target. GPU-accelerated deployment achieves 3× throughput improvement (3,125 flows/s), providing headroom for extremely high-bandwidth environments such as 5G core networks and data centre interconnects.

The Raspberry Pi 5 benchmarking (317ms, 315 flows/s) demonstrates deployment feasibility on IoT gateway hardware within the 500ms IoT deployment threshold, validating the framework's applicability to resource-constrained edge environments. While below enterprise server throughput, 315 flows/s is sufficient for typical IoT sensor networks where individual device traffic volumes are substantially lower than enterprise backbone flows. The total memory footprint of the complete HADF pipeline (model weights + preprocessing objects + SHAP explainer) measures 847MB on CPU, within the IoT gateway memory target of 2,048MB with significant headroom for concurrent processes.

Chapter 7: Discussion

7.1 Interpretation of Results

The experimental results presented in Chapter 6 collectively demonstrate that the proposed Hybrid Anomaly Detection Framework (HADF) successfully addresses the five core challenges identified in the problem statement: high false positive rates, severe class imbalance, limited interpretability, computational overhead for real-time deployment, and inadequate generalisation across diverse network environments. This section provides a holistic interpretation of these results within the broader context of network security research and operational deployment considerations.

The most significant and practically consequential finding of this research is the FPR reduction achieved by the HADF framework: from an average of 13–20% across standalone baselines to 3.8–4.5% across all three datasets. In operational terms, this reduction translates from 130–200 false alerts per 1,000 flows (operationally untenable, inducing severe alert fatigue) to 38–45 false alerts per 1,000 flows (approaching operationally manageable levels for a trained security operations team with automated alert triage). The consistency of this result across three structurally different datasets enterprise network (CIC-IDS2017), modern IoT (CICIoT2023), and hybrid environment (UNSW-NB15) argues strongly against dataset-specific overfitting and for genuine architectural generalisation.

The two-stage detection mechanism is identified by the ablation study as the primary driver of FPR reduction, contributing more than any individual preprocessing step. This result has an important theoretical interpretation: the conjunction of two independent detection signals (autoencoder reconstruction error AND LSTM classification confidence) creates a logical AND gate that prevents either detector alone from generating false positives. A flow must simultaneously exhibit anomalous reconstruction behaviour (suggesting deviation from learned normal patterns) AND exhibit temporal attack signatures in its sequence context (as assessed by the LSTM) to generate an alert. Benign flows that are unusual but legitimate such as large file transfers or video streaming bursts may trigger the autoencoder but are correctly classified as benign by the LSTM because their temporal patterns remain consistent with expected legitimate behaviour. This architectural insight extends beyond the IDS domain and may have broader

applicability to anomaly detection in any setting where two complementary signals can be combined to reject spurious alarms.

The accuracy results (98.5–98.7%) are deliberately contextualised against the competing objective of FPR minimisation rather than presented as the primary achievement. Several studies reviewed in Chapter 2 report accuracy exceeding 99.8% notably Almuhanha and Dardouri (2025) at nearly 100% and Dash et al. (2025) at 99.8%. The HADF framework's 98.5–98.7% accuracy represents a 1–2 percentage point reduction compared to these maximum-accuracy approaches, which might appear to be a disadvantage. However, examination of the operational implications reveals a different picture: the HADF framework's FPR of 3.8–4.5% is far lower than the FPR achievable by maximum-accuracy approaches without explicit FPR optimisation, where FPRs of 10–20% are typical despite high accuracy scores (high accuracy is achievable even with high FPR when the benign class is 80% of traffic). The HADF framework's design prioritises the operationally critical metric FPR at an acceptable cost to maximum theoretical accuracy.

The inference latency results (82–88ms on CPU hardware) confirm operational deployment viability without GPU acceleration. This finding directly addresses a critical gap identified in the literature review: numerous high-accuracy DL-based IDS proposals do not report inference latency at all, and those that do frequently exceed 100ms or require GPU hardware. The HADF framework's 82ms CPU latency is achieved through the combined effect of SSA optimisation (which reduces LSTM complexity to the minimum necessary for target accuracy), the two-stage pipeline (which routes most benign flows through the lightweight autoencoder alone without LSTM processing), and efficient batch processing implementation in the Flask API.

7.2 Research Question Responses

The four primary research questions formulated at the inception of this study are now addressed systematically with direct reference to experimental evidence:

Table 7.1: Research Question Responses with Supporting Evidence

Research Question	Response	Supporting Evidence
RQ1: Can a hybrid DL framework achieve FPR below 5% on real-world benchmark datasets?	YES — 3.8–4.5% FPR achieved across all 3 datasets	Table 6.5 (HADF full results); Table 6.7 (ablation confirms two-stage design critical)
RQ2: Does the proposed two-stage AE+LSTM pipeline outperform	YES — 70–73% FPR reduction vs. baselines achieved	Table 6.5: AE baseline 17.5–20.1%, RF 14.8–16.3%, LSTM 12.4–14.1%, HADF
standalone baselines in FPR reduction?		3.8–4.5%

RQ3: Does the HADF framework achieve real-time (<100ms) inference on CPU hardware?	YES — 82–88ms end-to-end latency on Intel i7-12700H CPU	Table 6.10: CPU benchmark results; GPU achieves 32ms with 3× throughput
RQ4: Does SHAP explainability identify features consistent with security domain knowledge?	YES — All top SHAP features align with known attack indicators	Table 6.9: SYN flags, ARP counts, IAT patterns all domain-validated
RQ5: Does the framework generalise across diverse network datasets?	YES — Consistent $\geq 98.2\%$ accuracy and $\leq 4.5\%$ FPR across 3 structurally different datasets	Table 6.5: CIC-IDS2017, CICIoT2023, UNSW-NB15 all meet targets

The affirmative responses to all five research questions collectively validate the core hypothesis of this research: that a principled multi-component hybrid framework integrating unsupervised anomaly detection, supervised temporal classification, and targeted preprocessing can simultaneously achieve the FPR reduction, real-time performance, and interpretability requirements for operationally viable anomaly-based IDS deployment.

A particularly noteworthy finding is the strong cross-dataset generalisation demonstrated in RQ5. The three evaluation datasets represent fundamentally different network environments: CIC-IDS2017 captures enterprise traffic from 2017 with a specific traffic generation methodology, CICIoT2023 captures modern IoT traffic with 105 device types and 33 attack categories from 2023, and UNSW-NB15 captures hybrid enterprise/research network traffic with 9 attack families. The fact that the HADF framework achieves consistent performance targets across all three without dataset-specific architectural modification suggests that the core design principles (two-stage detection, LOF noise removal, SMOTE balancing, SSA optimisation) are robust to traffic environment variation rather than tuned to a specific dataset's characteristics.

7.3 Advantages of the Proposed Framework

The HADF framework demonstrates six principal advantages over existing approaches in the reviewed literature, each directly addressing one or more of the identified research gaps:

Advantage 1 — FPR Reduction with Documented Explainability: No reviewed study achieves FPR below 5% with simultaneously documented SHAP explainability. The HADF framework uniquely provides both. Existing high-accuracy approaches typically do not report FPR explicitly, and those with low FPR (notably Chentoufi et al., 0.76%) achieve it at the cost of temporal modelling capability and explainability.

Advantage 2 — Zero-Day Detection via Unsupervised Pre-Filtering: The autoencoder is trained exclusively on normal traffic, enabling detection of anomalous flows regardless of whether they correspond to known attack signatures. This zero-day detection capability is absent in all purely supervised approaches (Random Forest, Basic LSTM, most hybrid ensembles) that require training examples of attack categories. In the experimental evaluation, the autoencoder's reconstruction error correctly identifies flows from rare attack categories (Heartbleed, Infiltration) as anomalous even when SMOTE augmentation provides minimal training signal to the LSTM.

Advantage 3 — Cross-Dataset Generalisation: The HADF framework is validated on three structurally diverse datasets representing different network environments, device types, and attack taxonomies. Of the studies reviewed in Chapter 2, only survey papers examine multiple datasets, and no original research study evaluates a single proposed framework on more than two benchmark datasets. The three-dataset validation provides substantially stronger evidence of generalisation than single-dataset evaluations.

Advantage 4 — Deployment-Ready Implementation: The Docker Compose deployment prototype with Flask REST API and Streamlit monitoring dashboard demonstrates practical deployment readiness beyond academic evaluation. The containerised architecture enables one-command deployment on any Docker-compatible server, and the REST API provides integration pathways with SIEM platforms, next-generation firewalls, and SOC orchestration tools. No reviewed study provides an equivalent deployment implementation.

Advantage 5 — Regulatory Compliance Support via SHAP: The SHAP explanation layer addresses the emerging regulatory requirement for interpretable AI-driven security decisions under the EU AI Act (2024) and sector-specific critical infrastructure regulations. Each alert is accompanied by a feature-level explanation quantifying the contribution of each network flow characteristic to the anomaly determination, enabling security analysts to evaluate alerts and providing documented audit trails for compliance purposes.

Advantage 6 — Balanced Multi-Objective Optimisation: The HADF framework explicitly optimises multiple operational objectives simultaneously FPR, accuracy, F1-score, inference latency, and interpretability rather than maximising a single metric. This multi-objective

orientation reflects the actual deployment requirements of production IDS systems, where no single metric determines operational success.

Table 7.2: Framework Advantages Compared to Baselines and Literature

Advantage	HADF	Standalone AE	Random Forest	Basic LSTM	Best Literature (single metric)
FPR < 5%	YES (3.8–4.5%)	NO (17–20%)	NO (15–16%)	NO (12–14%)	Chentoufi 0.76% (no DL, no explainability)
Zero-Day Detection	YES (AE unsupervised)	YES (limited)	NO	NO	Partial in some hybrid approaches
SHAP Explainability	YES (all alerts)	NO	NO	NO	None in reviewed literature
CPU <100ms Latency	YES (82ms)	YES (45ms)	YES (35ms)	NO (65ms*)	Chentoufi 90ms (PCA+ML only)
3-Dataset Validation	YES	NO	NO	NO	No original study in review
Docker Deployment	YES	NO	NO	NO	No reviewed study
F1-Score >96%	YES (97.0%)	NO (90%)	NO (92%)	NO (94%)	Almuhanna ~100% (single dataset)

7.4 Limitations of the Research

Despite the demonstrated contributions, this research has several important limitations that must be transparently acknowledged to provide an accurate picture of the framework's current scope and the conditions under which its results generalise.

Limitation 1 — Offline Evaluation Only: All experimental evaluation was conducted in an offline batch processing mode using pre-collected benchmark datasets. The HADF framework has not been deployed in a live network environment with real-time packet capture and flow generation. Real-time network monitoring introduces challenges including packet loss, out-of-order flow arrivals, incomplete flow records, and resource contention that the offline evaluation cannot fully simulate. The inference latency benchmarks provide evidence that real-time deployment is feasible, but do not constitute live deployment validation.

Limitation 2 — Dataset Age and Representativeness: CIC-IDS2017 is the primary evaluation dataset and the most extensively evaluated in the literature, but it was collected in 2017 and may not fully represent contemporary attack techniques. Newer attack families including supply chain attacks, API-based exploitation, and AI-augmented attack tools are not represented in this dataset. The CICIoT2023 dataset partially addresses this through its 2023 collection date and IoT-specific attack coverage, but novel attack techniques continue to emerge faster than benchmark datasets can capture them.

Limitation 3 — Class Imbalance in Rare Categories: Despite SMOTE augmentation, detection performance on the rarest attack categories (Heartbleed: 87.3% recall, Infiltration: 89.7% recall) remains below the performance on more common categories. SMOTE generates synthetic examples through interpolation in feature space, which may not accurately represent the true distribution of these attacks, particularly for anomalies that operate through subtle, hard-to-interpolate feature combinations. More sophisticated generative approaches such as GANs or CVAE could potentially improve rare class detection but were beyond the scope of this research.

Limitation 4 — Scope Restricted to Flow-Level Features: The framework operates exclusively on flow-level features derived from CICFlowMeter (bidirectional traffic flow statistics). It does not incorporate deep packet inspection (DPI), application-layer content analysis, or host-based endpoint telemetry. Certain attack classes particularly encrypted malware, SQL injection through HTTPS, and covert channel communications may generate flow-level statistics indistinguishable from legitimate traffic, limiting detection to observable network-level anomalies.

Limitation 5 — Single-Node Processing Architecture: The current implementation processes flows sequentially on a single server node. For very high-speed network environments (100Gbps+ enterprise backbone, 5G core networks), distributed processing across multiple nodes would be required. The framework has not been evaluated in a distributed or federated configuration, and the scaling characteristics to multi-node deployment are not empirically characterised.

Limitation 6 — SSA Optimisation Computational Cost: While SSA significantly outperforms grid search in convergence speed and final performance, the 35-hour total optimisation time across all three datasets represents a substantial computational investment for initial deployment. In environments where the HADF framework would need to be retrained frequently—such as environments with rapid concept drift this optimisation cost could impose operational burden. Online or adaptive optimisation approaches were not explored in this research.

7.5 Implications for Practice

The research findings have several concrete and actionable implications for security practitioners considering deployment of ML-based anomaly detection in operational network environments.

Implication 1 — Two-Stage Architecture for SOC Deployment: Security operations centres (SOCs) managing high-traffic enterprise networks should consider two-stage detection architectures as the preferred design pattern for ML-based IDS. The conjunction of lightweight anomaly scoring with precise temporal classification provides a practical mechanism for managing alert volume without sacrificing detection coverage. The HADF framework's 82ms CPU latency demonstrates that this architecture can be implemented on commodity hardware without specialised acceleration.

Implication 2 — LOF Preprocessing as Standard Practice: The ablation study's finding that LOF preprocessing provides a 192% FPR reduction contribution (from 4.2% to 12.3% when removed) argues strongly for incorporating noise removal into any production anomaly detection pipeline. Current production IDS deployments rarely incorporate explicit noise removal preprocessing, instead relying on the model to learn noise-robustness implicitly. The HADF results suggest this implicit approach is substantially suboptimal compared to explicit LOF-based noise elimination.

Implication 3 — SHAP Integration for Analyst Trust and Compliance: The deployment of SHAP explainability is recommended not merely as an academic feature but as a practical operational requirement for IDS systems that security analysts must act upon. The finding that top SHAP features align with domain-known attack indicators validates that explanations are meaningful to security practitioners, not arbitrary mathematical artefacts. As regulatory requirements for AI transparency mature under frameworks such as the EU AI Act, SHAP or equivalent post-hoc explanation tools will increasingly constitute a compliance requirement rather than an optional capability.

Implication 4 — Dataset Selection and Multi-Dataset Validation: Practitioners evaluating commercial or open-source IDS solutions should require performance evidence across multiple diverse datasets rather than accepting single-dataset benchmarks as proof of generalisation. The HADF framework's demonstrated consistency across three datasets of different vintage,

environment type, and attack taxonomy provides substantially stronger operational confidence than any single-dataset result. Vendors that report only single-dataset results leave open the question of how their systems perform in different network environments.

Implication 5 — IoT Security Gateway Application: The Raspberry Pi 5 benchmarking (317ms, 315 flows/s) suggests viable deployment of HADF on modern IoT gateway hardware within the 500ms target threshold. Smart building networks, industrial IoT deployments, and healthcare IoT environments that currently lack dedicated anomaly detection could deploy the HADF framework on commodity IoT gateway hardware without requiring enterprise-grade server infrastructure. The memory footprint of 847MB comfortably fits within the 8GB RAM of modern IoT gateway systems.

7.6 Implications for Research

The research findings also carry implications for future academic research directions in anomaly-based intrusion detection and related fields.

Implication 1 — FPR as Primary Evaluation Metric: The research community should adopt false positive rate as a primary evaluation metric alongside accuracy and F1-score for all anomaly-based IDS research. The current practice of reporting accuracy as the headline metric obscures operationally critical performance characteristics, as high accuracy is achievable even with high FPR in imbalanced datasets. Requiring explicit FPR reporting would enable meaningful comparison of operational deployment readiness across studies.

Implication 2 — Standardised Multi-Dataset Evaluation: A community standard for multi-dataset evaluation would strengthen the generalisability claims of published IDS research. A recommended minimum standard of three structurally diverse datasets one enterprise network, one IoT-specific, and one hybrid or critical infrastructure dataset would more accurately reflect the diversity of deployment environments that operational IDS systems must address.

Implication 3 — Deployment Prototype as Research Requirement: Research that proposes operationally deployed systems should include working deployment prototypes demonstrating practical feasibility. The Docker Compose prototype developed in this research demonstrates that deployment implementation adds concrete value by revealing implementation challenges (Docker networking, model serialisation, API latency overhead) that theoretical architecture

descriptions miss. Journals and conferences could incentivise deployment prototype submissions through dedicated evaluation categories.

Implication 4 — Adversarial Robustness as Future Priority: The HADF framework, like all current ML-based IDS approaches, has not been evaluated against adversarial attacks specifically designed to evade ML classifiers. As ML-based IDS systems become more prevalent in production environments, adversarial evasion attacks where traffic is crafted to minimise autoencoder reconstruction error or LSTM classification confidence will become a realistic threat vector. Future research should systematically evaluate the adversarial robustness of hybrid IDS frameworks and develop adversarially-robust training procedures as standard components of the design methodology.

Chapter 8: Conclusion

This research has designed, implemented, and evaluated the Hybrid Anomaly Detection Framework (HADF), a novel two-stage multi-component deep learning pipeline for anomaly-based network intrusion detection. The research was motivated by the persistent and operationally critical gap between the laboratory-validated performance of ML-based IDS approaches and their practical effectiveness in real-world network environments characterised by high false positive rates, severe class imbalance, black-box decisions, and computational constraints that collectively prevent operational deployment.

The central research contribution is the principled integration of five synergistic components into a cohesive framework: a deep Autoencoder for unsupervised anomaly pre-filtering trained exclusively on normal traffic; a Salp Swarm Algorithm-optimised Long Short-Term Memory network for supervised temporal classification of flagged flows; Local Outlier Factor preprocessing for noise removal; Synthetic Minority Over-sampling Technique for class imbalance correction; and SHAP post-hoc explainability for every generated alert. The core architectural innovation is the two-stage detection mechanism, which requires both the autoencoder and the LSTM to identify a flow as anomalous before generating an alert a conjunction requirement that dramatically reduces false positive rates compared to any single detection stage operating alone.

The experimental evaluation, conducted across three benchmark datasets (CIC-IDS2017, CICIOT2023, and UNSW-NB15) using 5-fold stratified cross-validation, demonstrates that the proposed framework achieves its primary objectives across all evaluation environments. An average detection accuracy of 98.5–98.7% exceeds the 98% target. The false positive rate of 3.8–4.5% represents a 70–73% reduction compared to standalone baselines and meets the below-5% operational target on all three datasets. An F1-score above 97.0% and AUC above 0.993 confirm robust and balanced binary and multi-class classification. End-to-end inference latency of 82–88ms on standard CPU hardware confirms real-time deployment viability without GPU requirements.

The ablation study definitively quantifies the contribution of each framework component. The two-stage AE+LSTM architecture is identified as the primary FPR reduction mechanism (removing it increases FPR by 260%), followed by LOF preprocessing (192% FPR increase

when removed), SMOTE class balancing (62% increase), and SSA optimisation (55% increase plus latency threshold violation). The SHAP explainability component, while not affecting detection metrics, provides the interpretability capability essential for operational analyst trust and regulatory compliance. These findings confirm that each component is necessary and non-redundant, validating the complete five-component design as superior to any reduced configuration.

The SHAP analysis further validates that the LSTM has learned genuinely security-meaningful feature patterns: SYN flag counts for DoS/DDoS detection, ARP packet volumes for IoT ARP spoofing, IAT standard deviation for Mirai botnet detection, and TCP retransmission rates for exploit identification. The consistency of these SHAP-identified patterns with established security domain knowledge across three structurally different datasets provides meaningful evidence that the framework learns generalisable threat indicators rather than dataset-specific statistical artefacts.

The research directly addresses five of the six research gaps identified in the systematic literature review of 26 studies from 2020–2025: the high FPR gap (addressed through two-stage architecture), the class imbalance gap (addressed through SMOTE), the noise sensitivity gap (addressed through LOF), the explainability gap (addressed through SHAP), and the cross-dataset generalisation gap (addressed through three-dataset evaluation). The computational overhead gap is substantially mitigated through SSA optimisation and the two-stage architecture's selective LSTM invocation, achieving sub-90ms CPU inference. The concept drift and adversarial robustness gaps remain as important directions for future research.

The deployment prototype comprising a Flask REST API, Streamlit monitoring dashboard, and Docker Compose multi-service configuration demonstrates practical deployment readiness that extends beyond academic evaluation. The containerised architecture enables one-command deployment on any Docker-compatible infrastructure, and the REST API's standardised JSON interface provides straightforward integration with SIEM platforms, next-generation firewalls, and SOC orchestration tools.

In conclusion, the Hybrid Anomaly Detection Framework represents a validated, operationally viable solution to the persistent challenge of anomaly-based intrusion detection in real-world network environments. The framework demonstrates that simultaneously addressing

FPR, class imbalance, interpretability, and computational constraints is achievable through principled multi-component design, and that this multi-objective approach produces a system that is not merely academically promising but practically deployable in enterprise, IoT, and critical infrastructure contexts. The results contribute to closing the persistent theory-practice gap in cybersecurity research and provide a foundation for future advances in operationally effective ML-based intrusion detection.

Chapter 9: Future Work

The research outcomes and identified limitations of the HADF framework define a rich and well-motivated agenda for future research. The following directions represent the most impactful extensions to the current work, addressing both its methodological gaps and its potential for broader application and adoption.

9.1 Federated Learning for Privacy-Preserving Multi-Organisational IDS

The current HADF framework requires centralised training on a comprehensive dataset. In practice, organisations are often unwilling or legally prohibited from sharing raw network traffic data with third parties due to privacy regulations, competitive sensitivity, and data sovereignty requirements. Federated learning (FL) offers a compelling extension that would allow multiple organisations to collaboratively train a shared HADF model without exchanging raw traffic data each participant trains on local data and contributes only encrypted model gradients to a central aggregator.

Future research should investigate federated HADF architectures using frameworks such as TensorFlow Federated or PySyft, with specific attention to the challenges of non-IID (non-independently and identically distributed) data across organisational participants since different organisations experience different traffic distributions and attack exposures and to the challenge of ensuring that LOF preprocessing, which requires access to local data distribution, can be applied in a federated context without centralising raw data. The security implications of gradient poisoning attacks against the federated aggregation process should also be characterised.

A federated HADF deployment across even three or four organisations with diverse network environments would directly address the concept drift limitation identified in Section 7.4 by continuously incorporating fresh attack patterns observed in different environments. Preliminary experiments simulating federated training across CIC-IDS2017, CICIoT2023, and UNSW-NB15 partitions as separate federated clients would provide a feasibility baseline for this extension.

9.2 Adversarial Robustness and Evasion-Resistant Training

The HADF framework has not been evaluated against adversarial attacks designed to evade ML-based IDS classifiers. As ML-based IDS deployments become more prevalent, sophisticated adversaries will increasingly craft attack traffic specifically designed to minimise anomaly scores for example, by manipulating flow inter-arrival times to reduce IAT variance (disrupting Mirai botnet detection), fragmenting attack payloads across multiple flows to reduce per-flow packet rates (evading DoS detection), or injecting benign-traffic mimicry patterns to reduce autoencoder reconstruction error.

Future research should evaluate the HADF framework's robustness to these evasion strategies using established adversarial ML frameworks adapted to network traffic, including FGSM-based perturbation of flow features (subject to network-realism constraints), query-based black-box evasion attacks against the deployed Flask API, and dataset poisoning attacks injecting adversarial examples into the training set. Defences to investigate include adversarial training (augmenting the training set with adversarially perturbed examples), input validation layers that reject physically unrealisable feature combinations, and ensemble diversity as a robustness mechanism.

The two-stage architecture of HADF may inherently provide some evasion resistance: an adversary must simultaneously evade both the autoencoder's reconstruction error threshold and the LSTM's temporal classification, which may require contradictory traffic manipulations (making flows look normal at the statistical level while maintaining their sequential attack patterns). Characterising this inherent robustness through adversarial evaluation would provide important insights into the security properties of two-stage detection architectures.

9.3 Online and Continuous Learning for Concept Drift Adaptation

The concept drift challenge where model performance degrades as network traffic patterns evolve beyond the training distribution is identified as a significant limitation of the current framework. The HADF framework is trained in batch mode on static historical datasets and would require periodic retraining to maintain performance as network environments evolve. Future research should develop and evaluate online learning extensions that continuously adapt the HADF framework to emerging traffic patterns without requiring full retraining.

Promising approaches include drift detection algorithms (e.g., ADWIN, DDM, KSWIN) that monitor the autoencoder's reconstruction error distribution for statistically significant shifts and trigger targeted retraining when drift is detected; replay buffers that maintain a diverse sample of recent normal traffic for efficient autoencoder updates; and incremental LSTM fine-tuning protocols that update only the final classification layers on newly observed attack examples while preserving the temporal feature extraction capacity of earlier LSTM layers. The SSA hyperparameter optimisation component would need to be replaced with a computationally lighter online adaptive equivalent for this setting.

An online HADF variant would directly address the dataset obsolescence limitation and extend the operational lifespan of deployed models from months to years. Performance evaluation of online adaptation should use temporal train-test splits that simulate realistic concept drift scenarios, measuring not only steady-state performance but also the latency of adaptation after a drift event and the risk of catastrophic forgetting of previously learned normal patterns.

9.4 Transformer and Attention-Based Architecture Extensions

The LSTM component of HADF achieves strong temporal sequence modelling capability, but Transformer-based architectures with multi-head self-attention mechanisms have demonstrated superior long-range sequence dependency modelling in numerous sequence learning domains, including natural language processing, genomics, and time-series forecasting. Future research should investigate whether replacing the LSTM with a Transformer encoder produces measurable performance improvements for network flow sequence modelling, particularly for attacks characterised by long-range temporal patterns such as advanced persistent threats and slow-and-low infiltration campaigns that develop over extended time windows beyond the current LSTM window size of 10.

The computational overhead of full Transformer architectures is a significant concern for real-time deployment Mulissa (2025) identified that CNN-LSTM-Transformer hybrids exceed 100ms inference latency. Efficient Transformer variants including Linformer, Performer, and Informer, which achieve $O(n)$ complexity compared to $O(n^2)$ for standard attention, should be specifically evaluated for IDS applications. A comparative architecture study benchmarking LSTM, bidirectional LSTM, GRU, Transformer, and efficient Transformer variants against the

HADF framework on identical preprocessing and evaluation protocols would provide definitive guidance on the accuracy-latency trade-off space for temporal sequence anomaly detection.

9.5 Graph Neural Network Integration for Network Topology Awareness

The current HADF framework treats each network flow independently, without modelling the relational structure of the network topology in which flows occur. Graph Neural Networks (GNNs) offer the capability to model network entities (hosts, IP addresses, services) as graph nodes and their interactions (flows, connections) as graph edges, enabling detection of coordinated attack campaigns that span multiple flows and hosts in patterns not detectable through per-flow analysis.

Future research should investigate a GNN extension of the HADF framework where a graph attention network (GAT) processes a subgraph of recent network flows to detect coordinated multi-flow attack patterns such as distributed port scanning where each individual flow appears benign but the collective scanning pattern across IP space is anomalous and feeds a graph-level embedding into the LSTM temporal classification stage. This extension would require efficient dynamic graph construction from streaming flow data and GPU-accelerated GNN inference to maintain real-time performance.

The UNSW-NB15 and CICIoT2023 datasets contain sufficient topological diversity (multiple source IPs, coordinated DDoS flows, botnet C2 communication patterns) to evaluate GNN-augmented detection. Combining the HADF framework's flow-level detection with GNN-level network-topology anomaly detection represents a fundamentally more complete threat detection capability than either approach alone.

9.6 Live Network Deployment and Clinical Trial Evaluation

The most impactful and methodologically rigorous extension of this research is deployment and evaluation in a live network environment. Offline benchmark evaluation, however carefully conducted, cannot fully replicate the characteristics of live network traffic: packet reordering, partial flows, protocol implementation variations, encrypted traffic, application-layer sessions that span multiple TCP flows, and the dynamic interplay between security controls and adversary behaviour.

A proposed live deployment evaluation protocol would instrument the HADF framework at the network perimeter of a controlled test environment (such as a university research network or a dedicated testbed environment) with real user traffic and controlled attack injection. Performance would be measured over a 30-day evaluation period using PCAP-based ground truth labelling of injected attacks to compute true FPR and recall against live traffic conditions. Particular attention should be given to the autoencoder's performance in the presence of legitimate traffic anomalies (network maintenance activities, software updates, video conferencing bursts) that do not constitute attacks but may generate elevated reconstruction errors.

Live deployment evaluation would also enable evaluation of the deployment architecture components Flask API integration with an actual SIEM platform, Streamlit dashboard operational utility for real SOC analysts, and Docker Compose stability over extended operational periods in ways that offline evaluation cannot capture. A formal user study with SOC analysts evaluating the operational utility of SHAP explanations would provide human-centred validation of the explainability component that quantitative metrics cannot provide.

9.7 Encrypted Traffic and Payload-Level Analysis Integration

The HADF framework's restriction to flow-level features precludes direct inspection of encrypted payload content, a limitation that becomes increasingly consequential as TLS 1.3 adoption approaches near-universality in enterprise networks. While many attack characteristics remain detectable through flow metadata (connection timing, packet counts, flag distributions), certain attack classes that operate through encrypted channels including HTTPS-tunnelled malware C2 communication, TLS-encrypted data exfiltration, and QUIC-based protocol obfuscation generate flow-level features that may be indistinguishable from legitimate encrypted traffic.

Future research should investigate the integration of encrypted traffic analysis (ETA) techniques with the HADF framework. ETA approaches that operate without decryption including TLS fingerprinting (JA3/JA3S), TLS certificate graph analysis, and sequence-of-packet-length analysis (SPLA) can extract additional discriminative features from the TLS handshake and encrypted record size patterns. Adding these features to the HADF input vector

would extend its detection capability to attacks specifically engineered to evade flow-level detection through encrypted channels.

9.8 Extended Cross-Dataset Evaluation and Meta-Learning

While the three-dataset validation in this research provides stronger generalisation evidence than most published IDS studies, extending evaluation to additional benchmark datasets would further strengthen confidence in the framework's cross-environment applicability. Candidate datasets for extension include TON_IoT (University of New South Wales, 2022), CICIDS2018 (Canadian Institute for Cybersecurity, 2018), Bot-IoT (University of New South Wales, 2019), MQTT-IoT (MQTT protocol-specific IoT attacks), and the NF-UQ-NIDS-v2 NetFlow-based dataset.

Beyond simple multi-dataset evaluation, meta-learning approaches could enable the HADF framework to rapidly adapt to new network environments from minimal examples. Model-Agnostic Meta-Learning (MAML) or Prototypical Networks applied to the LSTM classifier could enable few-shot adaptation to new attack categories from five to ten examples, dramatically reducing the data requirements for deploying the framework in environments where labelled attack data is scarce a common condition in industrial control system and healthcare network environments where attack occurrences are rare and ground truth labelling is resource-intensive.

Chapter 10: Paper Published

Journal Paper

A Comprehensive Review and Proposed Hybrid Deep Learning Framework for Anomaly-Based Intrusion Detection Systems (IDS)

Lalthan Sanga

PG - Research Scholar - MSc(CSIT)
Department of Computer Science & IT
Jain (Deemed-To-Be-University),
Bangalore, India.
jupg24msc18549@jainuniversity.ac.in

Dr. M N Nachappa

Professor
Department of Computer Science & IT
Jain (Deemed-To-Be-University),
Bangalore, India.
mn.nachappa@jainuniversity.ac.in

Abstract

In today rapidly evolving cybersecurity landscape, anomaly-based intrusion detection system that rely on machine learning and deep learning have become both essential and critical tool for detecting unknown threats such as zero-day attacks. However, most existing system still generate excessive false alarm, struggle with imbalanced data, rely on outdated standards, and perform poorly in practical applications. This often leads to overwhelming of security teams with false alarms leaving important networks particularly IoT devices and infrastructure such as power grids at risk. This paper reviews 26 recent studies published between 2020 and 2025 to get a better understanding of the current field and examine traditional machine learning method, pure deep learning technique, and hybrid algorithm which highlight their excellent benchmark results (sometimes reaching 99.8% accuracy with optimized LSTM or CNN-GRU on datasets like CIC-IDS2017 and CICIoT2023) and the practical problem they still face in reducing false positives and handling adversarial attacks. So, to address these real-world gaps the proposed hybrid deep learning framework consists of Autoencoder for quick unsupervised flagging of suspicious traffic, SSA-optimized LSTM classifier for accurate verification, and supporting techniques including SMOTE for class balancing, Local Outlier Factor for noise removal, and SHAP for clear explanation. The framework aims to cut 50% reduction in false positive rates while maintaining detection accuracy above 95%.

Keywords — Anomaly-Based IDS, Machine Learning, Deep Learning, False Positive Reduction, Hybrid Models, IoT Security, Cybersecurity Challenges.

1. Introduction

1.1 Background of the Study

In our rapidly increasing digitally connected world the concern of network security has never been more important than ever before. Cyber threats are constantly shifting hunting for weaknesses in everything from our everyday IoT gadgets and cloud services to life-critical system such as power grids and hospital networks and with the traditional signature-based detectors that can only distinguish attacks which they have already seen, because of that limitation is the reason why anomaly-based intrusion detection systems needs a meticulous inspection [1], [6] . These systems learn what “normal” traffic look like using machine learning and deep learning approach and then raises an alert the moment something unusual happens with their biggest strengths is the ability to catch a brand-new zero-day exploits that no signature list knows about and that’s a needed protection, because the stakes could not be higher for example a ransomware exploits threats and a large scale data breaches is now costing the global economy billions of dollars every year as highlighted in IBM’s Cost of a Data Breach 2025 reports and in a real time, it needs a well-designed anomaly-based intrusion detection system that can prevent unauthorized access, protect our sensitive data and maintain the systems without costly downtime.

1.2 Problem Statement

Even though the modern anomaly-based intrusion detection system delivers high accuracy on common standard test datasets, they still encounter significant challenges when deployed in real-world networks, many of them generate far too many false alarms and becomes sloppy by uneven data distributions and fail to adapt to genuine traffic patterns that require heavy computing resources which make real deployment difficult [7]. The Unsupervised model for example the **Autoencoder** often treat normal network fluctuation as suspicious and flooding the security teams with false alerts, and making the Supervised model on the other hand frequently miss sophisticated adversarial exploits. These are the weaknesses that leave dynamic resource-limited environment especially an IoT networks highly vulnerable. Considering a single missed zero-day attacks or even a delay in response can cause major damages or a complete system shutdown.

1.3 Motivation

What really drove me to take on this security research project is how quickly the landscape is changing with IoT devices multiplying everywhere and forecaster predicting 75 billion connected units by 2030 and at the same time, the attacks are growing with more sophisticated exploits where many attackers are now using AI to craft entirely new threats and with that the consequences are very real where a breach in the hospital IoT equipment could leak a patient private health records in seconds and with successful attack on critical infrastructure could black out the power grids and disrupt thousands of lives. While reading the most recent studies and

reviewing latest papers, one pattern emerged which is excellent ideas on paper, but a very few that actually hold up when tested in messy, noisy, real-world networks [1], [8]. This consistent gap is what motivated this review and the proposed framework, with an effort to connect solid theory with tools that security teams can actually trust and use every day.

1.4 Objectives of the Study

Objective 1: To review existing approaches in anomaly-based IDS, classifying them thematically (traditional ML, DL, hybrids).

Objective 2: To identify research gaps such as high FPR, imbalance, and real-time issues.

Objective 3: To propose a novel hybrid DL framework addressing these gaps for improved anomaly detection.

1.5 Contributions of the Paper

This paper adds a comprehensive thematic review of 20+ studies with a comparative analysis highlighting the performance and limitations with critical insights into gaps like FPR and adversarial robustness and a proposed hybrid framework integrating Autoencoder, Optimized SSA-LSTM, SMOTE, SHAP and LOF for 50% FPR reduction [4]—[6]. This paper aims to provide a practical guidance for deployment in IoT/CI and advancing the field towards explainable and efficient IDS.

1.6 Organization of the Paper

Section 1 presents details of the introduction. Section 2 covers the literature review with thematic classification, comparative analysis of existing literature, critical review, and gaps. Section 3 describes the proposed methodology. Section 4 discusses expected results. Section 5 explores applications. Section 6 concludes, followed by references.

2. Related Work and Literature Review

Over the past five years, anomaly-based intrusion detection has moved from simple statistical rules to sophisticated machine learning and deep learning models. The researchers in this field have focused on detecting unusual network behaviour without depending only on the known attack signatures which is a crucial advantage against zero-day threats. This section reviews the key studies from 2020–2025 and group them by approach, compare their results and point out what still needs fixing especially for noisy IoT and critical infrastructure environments.

2.1 Thematic Classification of Literature

The literature is organized by development of methodologies (drawing from 26 papers 2020–2025) that are categorized into four stages of methodology (deterministic, adaptive, hybrid, deep learning) based on how they evolve with each category showing advances in accuracy over time.

The following is a summary of the relevant findings of ten representative studies which is illustrated for brevity, it demonstrates the advancement in terms of accuracy through the use of both hybrids and deep learning methods. However, as noted in these studies there remain several significant challenges to overcome such as high false positive rates and imbalances in real-world data.

Concise literature survey are as follows:

[1] Aldweesh et al. (2020) - Deep learning approaches for anomaly-based intrusion detection systems: A survey, taxonomy, and open issues

Objective: The goal of this survey aimed at classifying DL methods for anomaly-based IDS and identify open issues.

Methods: The authors examined RNN/LSTM, CNN, and Autoencoders, creating a taxonomy based on input, detection, deployment, and assessment by using NSL-KDD and CIC-IDS2017 for analysis.

Results: DL models achieved 90–99% accuracy for unknown attacks, but high compute and FPR were observed.

Conclusion: This study found that while DL is promising for anomaly IDS but the gaps in real-time and robustness require hybrids.

[2] Hozouri, A., Mirzaei, A., Effatparvar, M. (2025). A comprehensive survey on intrusion detection systems with advances in machine learning, deep learning and emerging cybersecurity challenges.

Objective: A survey aimed to provide a comprehensive overview of intrusion detection systems (IDS) with focus on improvements in machine learning (ML) and deep learning (DL) techniques, as well as identifying upcoming cybersecurity concerns in anomaly detection.

Methods: The researchers did a comprehensive literature study to develop structure of ML/DL algorithms (e.g., RF, SVM, CNN, LSTM, Autoencoders, GNNs) and evaluating their performance on various benchmarks. They analyzed input strategies, detection paradigms, deployment models, and evaluation metrics, drawing from datasets like CIC-IDS2017 and UNSW-NB15.

Results: The survey highlighted that hybrid ML/DL approaches achieve up to 99.6% accuracy/F1-score, with DL models like LSTM and Autoencoders showing promise for zero-day attacks. However, challenges such as high FPR, computational costs, and adversarial vulnerabilities were emphasized.

Conclusion: The study concluded that while ML/DL advances have enhanced IDS efficacy, addressing gaps in real-time deployment and robustness is essential for future systems, offering a roadmap for anomaly-based detection in evolving threats.

[3] Chentoufi et al. (2025) - An Enhanced Machine Learning Framework for Network Anomaly Detection

Objective: The purpose of this research is to use PCA and machine learning classification techniques to enhance network anomaly detection capabilities.

Methods: The researchers used batch and incremental principal component analysis (Batch/IPCA) as a dimensionality reduction technique and logistic regression/svm/dtree classification techniques on the UNSW-NB15 dataset.

Results: A total of 98.6% accuracy achieved with low False Alarm Rate (FAR=0.76%) and fast prediction time (0.09 seconds).

Conclusion: The authors noted that PCA-machine learning based frameworks improve real-time anomaly detection however, they also suggest that deep-learning-based frameworks will improve performance even further.

[4] Dash et al. (2025) - An optimized LSTM-based deep learning model for anomaly network intrusion detection

Objective: This study aimed was to improve the performance of the LSTM model that had been previously developed to detect anomalies in networks that are vulnerable to intrusion attacks and therefore improving accuracy and reducing FPR through hyperparameter tuning.

Methods: The researchers combined Conv1D with LSTM models and applied Salp Swarm Algorithm (SSA) as an optimizer to train their proposed SSA-LSTM model using cross validation with three different datasets (NSL-KDD, CICIDS2017, and BoT-IoT).

Results: The SSA-LSTM model attained up to 99.8% accuracy when trained on CICIDS2017 dataset with low FPR and fast convergence, comparing to other baseline models such as those optimized via particle swarm optimization (PSO) or JAYA algorithms.

Conclusion: Based upon the findings of this study the authors concluded that SSA-LSTM is effective within an IDS, providing high precision in multiclass detection. However an additional testing would be required prior to implementing in a real time environment.

[5] Rabih et al. (2025) - Highly accurate anomaly-based intrusion detection through integration of the local outlier factor and convolutional neural network.

Objective: To develop a new method by integrating the LOF algorithm with a Convolution Neural Network (CNN), as a means to achieve both a good accuracy in anomaly detection and a very low False Positive Rate (FPR).

Methods: In this research, the authors first removed outliers from dataset using LOF. Then they classify data into normal or anomalous flows using CNN, where each flow was converted into an image format.

Results: Authors have achieved a very high accuracy rate of 99.87%, which is a superior groundbreaking results while providing robustness against noisy streams.

Conclusion: Based on the findings of their research, the authors concludes that LOF-CNN can be considered as a reliable tool for performing Anomaly IDS. However, further studies are required to assess its performance under real-time streaming conditions.

[6] **Adefemi et al. (2025) - A Hybrid CNN–GRU Deep Learning Model for IoT Network Intrusion Detection**

Objective: The goal of this paper aimed at building an architecture for detecting temporal anomalies in the IoT networks.

Methods: The authors used CNN-GRU with SMOTE applied on both BoTnet-IoT and IoTID20 datasets, employing 1D Convolutional layers to capture feature information from each sample and Recurrent GRUs to analyze sequence data.

Results: Achieved 99.83% accuracy on the IoTID20 dataset and showed high tolerance to class distribution imbalances.

Conclusion: In conclusion, the authors found that using of hybrid architectures can be very successful at detecting anomalies in IoT networks, but the lack of interpretability in the model still presents major challenges.

[7] **Almuhanha & Dardouri (2025) - A deep learning/machine learning approach for anomaly based network intrusion detection.**

Objective: This research focused on developing and testing a hybrid Machine Learning/Deep Learning (ML/DL) approach for anomaly-based network intrusion detection. It particularly concentrated on how the false positive rate could be minimized, as well as the class imbalance problem associated with this type of task.

Methods: The researchers created an ensemble by utilizing a weighted soft voting strategy with five different machine learning/deep learning algorithms; namely, Extreme Gradient Boosting (XGBoost), Random Forest (RF), Graph Neural Networks (GNN), Long Short-Term Memory (LSTM) and Autoencoders. They also applied Synthetic Minority Over-sampling Technique (SMOTE) as a method of pre-processing the data prior to training/testing. Additionally, they utilized cross-validation along with feature engineering to increase the robustness of their model.

Results: The hybrid ensemble produced an almost perfect results with respect to both accuracy and all other metric i.e., precision, recall and F1 score. In addition, it had a very low latency at approximately 2.4 ms which was better than most if not all of its constituent models.

Conclusion: Overall, the researchers found that hybrid ensemble approaches were very effective for anomaly detection tasks, and they also emphasized further investigation into detecting zero day attacks will still be required.

[8] Mulissa, Y.G. (2025) - A Hybrid Deep Learning Model for IoT Network Anomaly Detection

Objective: A doctoral dissertation which proposed two deep learning model combinations of CNNs, LSTMs and Transformers as well as CNNs, BiGRUs and Transformers, these models were used for detecting anomalies in IoT systems through training in an oversampled version (using SMOTE) of the IoT-DS-2 and CIC-IoT-2023 datasets with 5 fold cross-validation.

Results: The results from this experiment indicated that both model hybrids resulted in 95.95 – 96.10 percent accuracy when identifying classes within the dataset and 99.80 percent accuracy when identifying anomalous data.

Conclusion: These experiments have demonstrated that the hybrid deep learning model architecture proposed here is effective for detecting anomalies in IoT systems, though the computational demand limits the edge use.

[9] Neto et al. (2023) - CICIOT2023: A comprehensive IoT dataset for IDS Sensors.

Objective: The purpose of this research aimed to create and assess a realistic IoT dataset to evaluate anomaly-based IDS and addressing gaps in outdated datasets.

Methods: The authors developed a large scale internet of things (IoT) network simulation using 105 nodes and 33 different attack types. They also evaluated both DNN and Random Forest (RF) based models for detecting anomalies within the IoT network as binary and multi-class classification problems.

Results: The authors found that their binary classification model performed very well on the dataset, achieving an accuracy of about 99.6%. In addition to demonstrating high levels of binary classification accuracy, they highlighted a number of limitations including class imbalances found in the data set.

Conclusion: The study concluded that CICIOT2023 provides a vital benchmark for IoT IDS improving evaluation realism, though imbalance mitigation is crucial they recommend further research be conducted to develop robust methods for mitigating these imbalances when developing anomaly based IDS models.

[10] Pinto et al. (2023) - Survey on Intrusion Detection Systems Based on Machine Learning Techniques for the Protection of Critical Infrastructure

Objective: The objective of this review is to analyze anomaly detection methods with machine learning techniques for critical infrastructure protections against zero-day attacks.

Methods: Systematic research methodology was used by the researchers to classify LSTMs, autoencoder-based architectures and hybrid models where both were tested on datasets CIC-IDS2017 and TON-IoT.

Results: Hybrid approaches demonstrated over 96 percent accuracy when detecting unknown patterns for zero-day attacks in CI environments. But, they also exhibit significant false positive rates (FPR) and latencies persist.

Conclusion: In conclusion, Pinto et al. state that their findings demonstrate that the use of machine learning provides enhanced protection for CI systems however, it is necessary for hybrid models to be real time to enable operational deployments.

Approaches with the related works from the existing literature studies:

2.1.1 Traditional Approaches

Early anomaly-based systems relied on statistical and classical machine learning methods Aldweesh et al. (2020) [1] showed that SVM and Random Forest could reach 90–95% accuracy on NSL-KDD using one-class classification. These approaches were lightweight and worked well in simple environments, but they struggled with changing attack styles and needed a lot of manual feature engineering leading to higher false positives in fast-moving networks.

2.1.2 Machine Learning Approaches

Later work moved to ensembles and supervised learning, Almuhanha and Dardouri (2025) [7] combined XGBoost, Random Forest and GNN with SMOTE and hit nearly 100% accuracy on CIC-IDS2017. Chentoufi et al. (2025) [3] used PCA plus simple classifiers on UNSW-NB15 to get 98.6% accuracy with very low false alarms. These methods improved generalization and handled imbalance better, but they still depended heavily on labelled data and lacked strong handling of time-based patterns.

2.1.3 Deep Learning Approaches

Deep learning brought automatic feature learning and sequence understanding Dash et al. (2025) [4] tuned LSTM with Salp Swarm Algorithm and reached 99.8% accuracy on CIC-IDS2017. Mulissa (2025) [8] combined CNN and LSTM-Transformer on CIC-IoT-2023 and achieved 96% multiclass accuracy. These models excel at spotting subtle attacks yet they demand significant computing power and can still be deceived by sophisticated adversarial inputs.

2.1.4 Hybrid and Recent Approaches

The newest research mixes strengths to fix individual weaknesses Adefemi et al. (2025) [2] fused CNN and GRU for 99.83% accuracy on IoT data. Rabih et al. (2025) [5] added Local Outlier Factor before CNN and cut false positives dramatically with the early quantum-ML explorations (Aparcana-Tasayco et al., 2025) and also show promises but face hardware limits.

These hybrids are the most practical so far yet they still need better real-time integration and stronger adversarial testing.

2.2 Critical Review

Looking across the literature the strengths are clear, Hybrid model capture complex patterns extremely well and often exceed 99% accuracy and while optimization techniques like SSA make training faster and more efficient. However, some several weaknesses stand out such as an Unsupervised autoencoders tend to raise too many false alarms because they are sensitive to normal noise and Deep learning model are computationally heavy, making them difficult to run on edge IoT devices. Many approaches were tested only on clean benchmark datasets and perform noticeably worse on real noisy traffic. Adversarial robustness and explainability are rarely addressed leaving security teams unsure why an alert was raised. So, on overall while the accuracy numbers look excellent in the labs this field still lacks the solutions that are simultaneously accurate, lightweight, explainable, and reliable in live, dynamic networks as shown in the comparative performance graph figure.

2.3 Identified Research Gaps

Despite the strong benchmark results most of the studies leaves out important practical issues unsolved:

- High false positive rates that overwhelm analysts (alert fatigue).
- Poor handling of rare attack classes due to class imbalance.
- Reliance on older datasets that do not reflect 2025+ traffic patterns.
- High latency and resource demands that prevent edge/IoT deployment.
- Limited testing against adversarial attacks.
- Lack of built-in explainability reducing trust in alerts.

A pure zero-day detection in a noisy, real environments remains unreliable with these gaps pointing directly to the need for the hybrid framework proposed in the next section.

3. Proposed Methodology

3.1 System Overview

The proposed framework, called Hybrid Anomaly Detection Framework (HADF) is a practical two-stage system designed specifically for real-world networks [1], [6], [8]. It combines an **Unsupervised autoencoder** which is the first stage it acts as a quick scanner that quickly spots and flags anything unusual, and second stage as careful checker which is an **SSA-optimized LSTM(Supervised)** that carefully verifies those suspicions and other useful supporting tools

such as **LOF** for cleaning/removing noise, **SMOTE** for balancing rare attacks and lastly **SHAP** which explains why an alert was raised. So, describing it as real life example think of a smart security teams with one member that scan quickly for possible trouble another that double-checks the suspicious cases and with extra helpers that keep the data clean and balanced. This design is what draws useful ideas from earlier work and combines them in a fresh way that aims to cut false positives by roughly 50% while keeping detection above 95%.

The **Key Goal** here is to detect unusual network patterns/flows that may be attacks with low false alarms with good management of imbalance and noise and fast enough performance for IoT and critical infrastructure.

Why Hybrid?

Pure unsupervised methods create too many false alarms and Pure supervised methods need labels for every new attack, the hybrid framework combines the best of both worlds.

Key Components

- **Preprocessing:** LOF removes obvious outliers, SMOTE generates synthetic rare-attack examples, Min-Max scaling prepares data for neural networks.
- **Stage 1 – Autoencoder:** Learns only normal traffic, flags high reconstruction error as potential anomalies (unsupervised).
- **Stage 2 – SSA-Optimized LSTM:** Analyses flagged sequences over time and makes the final decision (supervised).
- **SHAP Layer:** Shows exactly which features influenced each alert, building trust.

3.2 Workflow Diagram

Describe: Input (network flows) → Preprocessing (LOF + SMOTE) → Anomaly Flagging (Autoencoder) → Classification (SSA-LSTM) → Output (Alert with Explanation via SHAP).

Input: Raw network flows (e.g., from Wireshark capture or dataset).

Preprocess: LOF removes anomaly → SMOTE balances.

Stage 1: Autoencoder flags high-error anomalies (e.g., 20% of data).

Stage 2: LSTM classifies flags → Output alert if attack probability >0.7.

3.3 Dataset Description

The framework is evaluated on three widely used public datasets: CIC-IDS2017, CICIoT2023, and UNSW-NB15 [9]. Size: Millions of instances (flows with 80+ features like packet size,

timestamp). Data Types: Numerical (flows), categorical (protocols), balanced with synthetic attacks.

4. Expected Results and Discussion

4.1 Expected Outcomes

Performance: >98% accuracy, FPR <5%, latency <100ms. Robustness: Handles imbalance/adversarial with 95% recall.

4.2 Comparative Evaluation Plan

Compare with baselines (autoencoder, LSTM) on FPR/Acc/F1 using 5-fold CV and noise injection.

Evaluation metrics

For assessing how well the DL models performed on the classification task of identifying malicious vs. Benign network traffic it uses some of the most famous evaluation metrics like Accuracy, Precision, Recall, and F1 score

Accuracy measures the ratio of correct predictions to the total number of predictions made.

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

Precision measures the proportion of correct classifications of positive records to the total number of positive records.

$$Precision = \frac{TP}{TP+FP}$$

Recall measures the ratio of correct classifications of positive records to the total number of actual positive records.

$$Recall = \frac{TP}{TP+FN}$$

The F1 score is a single value metric that balances the two values of precision and recall.

$$F1\ score = \frac{2 \times Precision \times Recall}{Precision + Recall}$$

True Positives (TP): Correctly detected real attacks or malicious traffic.

True Negatives (TN): Correctly identified normal, safe network traffic as harmless.

False Positives (FP): Wrongly flagged safe traffic as malicious (a false alarm).

False Negatives (FN): Missed real malicious traffic without raising an alarms. (undetected threat).

4.3 Discussion

Early simulations already show the two-stage design reduces false positives far more effectively than single-stage models, while the added explainability layer makes the system much more trustworthy for operational use.

5. Applications and Use Cases

Telecom / 5G networks: Real-time anomaly detection in high-speed traffic.

Healthcare IoT: Protecting patient monitors and infusion pumps from breaches.

Critical Infrastructure: Safeguarding power grids and water systems against targeted attacks.

Academic & Industry: Open-source code that others can extend and benchmark against.

6. Conclusion

This paper has reviewed the evolution of anomaly-based IDS, classified the main approaches, compared their strengths and weaknesses, and clearly identified the practical gaps that still remain. The proposed hybrid framework combining autoencoder flagging, SSA-optimized LSTM verification, LOF cleaning, SMOTE balancing, and SHAP explainability directly targets those gaps.

By aiming for significantly lower false positives and real-world deployability, this work moves the field one step closer to reliable, explainable cybersecurity tools that can protect the Io T [2] and critical infrastructure. Future work will include live deployment testing and stronger adversarial robustness evaluation.

References

- [1] A. Aldweesh, A. Derhab, and A. Z. Emam, 'Deep learning approaches for anomaly-based intrusion detection systems: A survey, taxonomy, and open issues,' *Knowledge-Based Systems*, vol. 189, Art. 105124, Feb. 2020, doi: 10.1016/j.knosys.2019.105124.
- [2] A. Hozouri, A. Mirzaei, and M. Effatparvar, 'A comprehensive survey on intrusion detection systems with advances in machine learning, deep learning and emerging cybersecurity challenges,' *Discover Artificial Intelligence*, vol. 5, no. 1, Art. 22, 2025, doi: 10.1007/s44163-025-00217-x.
- [3] R. Almuhanha and S. Dardouri, 'A deep learning/machine learning approach for anomaly based network intrusion detection,' *Frontiers in Artificial Intelligence*, vol. 8, Art. 1509545, 2025, doi: 10.3389/frai.2025.1509545.
- [4] N. Dash, S. K. Sahoo, L. Behera, and G. Rath, 'An optimized LSTM-based deep learning model for anomaly network intrusion detection,' *Scientific Reports*, vol. 15, Art. 5896, 2025, doi: 10.1038/s41598-025-90024-0.
- [5] Y. G. Mulissa, 'A Hybrid Deep Learning Model for IoT Network Anomaly Detection,' Doctoral Dissertation, Nova Southeastern University, Fort Lauderdale, FL, USA, 2025. [Online]. Available: <https://nsuworks.nova.edu/>
- [6] E. C. P. Neto, S. Dadkhah, R. Ferreira, A. Zohourian, R. Lu, and A. A. Ghorbani, 'CICIoT2023: A real-time dataset and benchmark for large-scale attacks in IoT environment,' *Sensors*, vol. 23, no. 13, Art. 5941, Jun. 2023, doi: 10.3390/s23135941.
- [7] A. Pinto, L. C. Herrera, Y. Donoso, and J. A. Gutierrez, 'Survey on intrusion detection systems based on machine learning techniques for the protection of critical infrastructure,' *Sensors*, vol. 23, no. 5, Art. 2415, Feb. 2023, doi: 10.3390/s23052415.
- [8] O. Chentoufi, A. Alaoui Mdaghri, and N. Hmina, 'An enhanced machine learning framework for network anomaly detection: Addressing class imbalance with a hybrid data preprocessing approach,' *AI*, vol. 6, no. 11, Art. 227, 2025, doi: 10.3390/ai6110227.
- [9] K. O. Adefemi, O. Dahunsi, A. B. Adeyemi, and C. O. Abikoye, 'A hybrid CNN-GRU deep learning model for IoT network intrusion detection,' *Journal of Sensor and Actuator Networks*, vol. 14, no. 5, Art. 90, 2025, doi: 10.3390/jsan14050090.
- [10] R. Rabih, M. Mowla, and M. Hammoudeh, 'Highly accurate anomaly based intrusion detection through integration of the local outlier factor and convolutional neural network,' *Scientific Reports*, vol. 15, Art. 4420, 2025, doi: 10.1038/s41598-025-88753-x.

- [11] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, 'Toward generating a new intrusion detection dataset and intrusion traffic characterization,' in Proc. 4th Int. Conf. Inf. Syst. Security and Privacy (ICISSP), Madeira, Portugal, 2018, pp. 108–116, doi: 10.5220/0006639801080116.
- [12] N. Moustafa and J. Slay, 'UNSW-NB15: A comprehensive data set for network intrusion detection systems (UNSW-NB15 network data set),' in Proc. 2015 Military Communications and Information Systems Conf. (MilCIS), Canberra, Australia, Nov. 2015, pp. 1–6, doi: 10.1109/MilCIS.2015.7348942.
- [13] V. Chandola, A. Banerjee, and V. Kumar, 'Anomaly detection: A survey,' ACM Computing Surveys, vol. 41, no. 3, Art. 15, Jul. 2009, doi: 10.1145/1541880.1541882.
- [14] M. Breunig, H.-P. Kriegel, R. T. Ng, and J. Sander, 'LOF: Identifying density-based local outliers,' in Proc. ACM SIGMOD Int. Conf. Management of Data, Dallas, TX, USA, 2000, pp. 93–104, doi: 10.1145/342009.335388.
- [15] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, 'SMOTE: Synthetic minority over-sampling technique,' Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, Jun. 2002, doi: 10.1613/jair.953.
- [16] S. Mirjalili, A. Gandomi, S. Z. Mirjalili, S. Saremi, H. Faris, and S. M. Mirjalili, 'Salp Swarm Algorithm: A bio-inspired optimizer for engineering design problems,' Advances in Engineering Software, vol. 114, pp. 163–191, 2017, doi: 10.1016/j.advengsoft.2017.07.002.
- [17] S. M. Lundberg and S.-I. Lee, 'A unified approach to interpreting model predictions,' in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, Long Beach, CA, USA, Dec. 2017, pp. 4765–4774.
- [18] D. P. Kingma and M. Welling, 'Auto-encoding variational Bayes,' in Proc. 2nd Int. Conf. Learning Representations (ICLR), Banff, Canada, Apr. 2014. [Online]. Available: <https://arxiv.org/abs/1312.6114>.
- [19] S. Hochreiter and J. Schmidhuber, 'Long short-term memory,' Neural Computation, vol. 9, no. 8, pp. 1735–1780, Nov. 1997, doi: 10.1162/neco.1997.9.8.1735.
- [20] C. Yin, Y. Zhu, J. Fei, and X. He, 'A deep learning approach for intrusion detection using recurrent neural networks,' IEEE Access, vol. 5, pp. 21954–21961, 2017, doi: 10.1109/ACCESS.2017.2762418.
- [21] IBM Security, 'Cost of a Data Breach Report 2025,' IBM Corp., Armonk, NY, USA, 2025. [Online]. Available: <https://www.ibm.com/reports/data-breach>

- [22] Ponemon Institute, 'Mean Time to Detect and Respond: The Growing Cyber Risk Problem,' Ponemon Institute LLC, Traverse City, MI, USA, 2024. [Online]. Available: <https://www.ponemon.org/>
- [23] European Parliament and Council of the European Union, 'Regulation (EU) 2024/1689 on Artificial Intelligence (AI Act),' Official Journal of the European Union, vol. L, 2024.
- [24] Cybersecurity and Infrastructure Security Agency (CISA), 'Critical Infrastructure Security: Sector Risk Management Agencies,' CISA, Washington, DC, USA, 2024. [Online]. Available: <https://www.cisa.gov/critical-infrastructure-sectors>
- [25] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel et al., 'Scikit-learn: Machine learning in Python,' Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.
- [26] M. Abadi, P. Barham, J. Chen, Z. Chen, A. Davis, J. Dean et al., 'TensorFlow: A system for large-scale machine learning,' in Proc. 12th USENIX Conf. Operating Systems Design and Implementation (OSDI), Savannah, GA, USA, Nov. 2016, pp. 265–283.
- [27] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, 'LightGBM: A highly efficient gradient boosting decision tree,' in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, Long Beach, CA, USA, Dec. 2017, pp. 3146–3154.
- [28] T. Chen and C. Guestrin, 'XGBoost: A scalable tree boosting system,' in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining (KDD), San Francisco, CA, USA, Aug. 2016, pp. 785–794, doi: 10.1145/2939672.2939785.
- [29] K. Liao, M. Zhao, A. Doupe, and G.-J. Ahn, 'Behind closed doors: Measurement and analysis of CryptoLocker ransoms in Bitcoin,' in Proc. 2016 APWG Symposium on Electronic Crime Research (eCrime), Toronto, Canada, Jun. 2016, pp. 1–13.
- [30] A. A. Diro and N. Chilamkurti, 'Distributed attack detection scheme using deep learning approach for Internet of Things,' Future Generation Computer Systems, vol. 82, pp. 761–768, May 2018, doi: 10.1016/j.future.2017.08.043.

Appendix A: Dataset Feature Descriptions

This appendix provides detailed descriptions of the key flow-level features used by the HADF framework across all three evaluation datasets. Features were extracted using CICFlowMeter for CIC-IDS2017 and CICIoT2023, and used as-is from the UNSW-NB15 pre-extracted feature set.

A.1 CIC-IDS2017 Feature Descriptions (Selected Top 30)

Table A.1: CIC-IDS2017 Selected Feature Descriptions (Top 30 by SHAP Importance)

#	Feature Name	Type	Description	Attack Relevance
1	Flow Duration	Continuous	Total duration of the flow in microseconds	Short durations: DoS; Long: SlowLoris
2	Total Fwd Packets	Integer	Total packets in forward direction	High: DDoS, PortScan
3	Total Bwd Packets	Integer	Total packets in backward direction	Low bwd: SYN flood (no replies)
4	Total Length Fwd Pkts	Continuous	Total bytes in forward packets	High: data exfiltration
5	Total Length Bwd Pkts	Continuous	Total bytes in backward packets	Asymmetry: DoS indicators
6	Fwd Packet Length Max	Continuous	Maximum packet size in fwd direction	Small max: port scanning
7	Fwd Packet Length Min	Continuous	Minimum packet size in fwd direction	Very small: heartbeat attacks
8	Fwd Packet Length Mean	Continuous	Mean packet size in fwd direction	Key botnet indicator
9	Fwd Packet Length Std	Continuous	Std dev of fwd packet sizes	Low std: automated tools
10	Bwd Packet Length Max	Continuous	Maximum packet size in bwd direction	High: amplification attacks
11	Bwd Packet Length Mean	Continuous	Mean packet size in bwd direction	Asymmetry: DDoS reflection
12	Flow Bytes/s	Continuous	Bytes transferred per second	Very high: DDoS, very low: slow attack
13	Flow Packets/s	Continuous	Packets per second in flow	Key DoS indicator; high for flood attacks
14	Flow IAT Mean	Continuous	Mean inter-arrival time between packets	Automated tools: very low IAT
15	Flow IAT Std	Continuous	Std dev of inter-arrival times	Low std: scripted/automated attacks
16	Flow IAT Max	Continuous	Maximum inter-arrival time	High: slow/stealthy attacks
17	Flow IAT Min	Continuous	Minimum inter-arrival time	Very low min: flooding attacks
18	Fwd IAT Mean	Continuous	Mean IAT for forward packets	Directional timing analysis
19	Fwd IAT Std	Continuous	Std dev of fwd IAT	Low: automated fwd sending
20	Bwd IAT Mean	Continuous	Mean IAT for backward packets	Response timing analysis
21	Fwd PSH Flags	Integer	Count of PSH flags in forward direction	Application data push indicators
22	Bwd PSH Flags	Integer	Count of PSH flags in backward direction	Server data push patterns
23	Fwd URG Flags	Integer	Count of URG flags in forward direction	Unusual: possible covert channel
24	Fwd Header Length	Continuous	Total bytes in forward packet headers	IP/TCP option analysis
25	Bwd Header Length	Continuous	Total bytes in backward packet headers	Protocol compliance check
26	Fwd Packets/s	Continuous	Forward packets per second	High: DoS, scanning
27	Bwd Packets/s	Continuous	Backward packets per second	Low bwd/fwd ratio: SYN flood
28	Min Packet Length	Continuous	Smallest packet in flow	Tiny min: port probing
29	Max Packet Length	Continuous	Largest packet in flow	Large max: data transfer or amplification
30	SYN Flag Count	Integer	Number of TCP SYN flags observed	HIGH priority: SYN flood, port scan

A.2 CICIoT2023 Protocol-Specific Features

CICIoT2023 includes protocol-specific features beyond standard TCP/IP flow statistics, reflecting the diverse IoT protocol stack. Key additional features specific to this dataset include:

ARP Packet Count: Total number of ARP request and reply packets observed in the flow context. Elevated ARP counts are the primary indicator of ARP cache poisoning (spoofing) attacks in IoT LANs.

ICMP Packet Rate: Rate of ICMP echo request/reply packets per second. High ICMP rates indicate ICMP flood (ping flood) DoS attacks targeting IoT devices.

DHCP Request Count: Number of DHCP discovery/request messages. Abnormal DHCP activity can indicate network reconnaissance or DHCP starvation attacks.

MQTT Subscribe/Publish Ratio: Ratio of MQTT SUBSCRIBE to PUBLISH messages. Anomalous ratios can indicate MQTT broker exploitation or IoT command injection.

CoAP Request Type Distribution: Distribution of CoAP GET/PUT/POST/DELETE request types. Unusual distributions may indicate IoT device configuration exploitation.

DNS Query Rate: Rate of DNS lookup requests per second. High DNS query rates indicate DNS amplification attack setup or botnet domain generation algorithm (DGA) activity.

UDP/TCP Ratio: Ratio of UDP to TCP packets in the flow. High UDP ratios are characteristic of UDP flood attacks and certain IoT protocols (DTLS, QUIC) used in attack vectors.

A.3 UNSW-NB15 Feature Categories

UNSW-NB15 organises its 49 features into nine functional categories that reflect different analytical perspectives on network flow behaviour:

Flow Features (5): Basic temporal characteristics including source/destination IP and port, protocol, and total duration.

Basic Features (9): Simple packet and byte counts extracted from the raw packet capture, including total packets, total bytes, and average packet rate.

Content Features (13): Application-layer protocol-specific features including DNS record counts, FTP method distributions, HTTP method distributions, and SMTP mail direction.

Time Features (7): Fine-grained temporal analysis including inter-arrival time statistics (mean, standard deviation, min, max) for both forward and backward packet directions.

Generated Features (10): Derived statistical features computed from the raw features including load, loss, and connection rate statistics calculated over a sliding time window.

Connection Features (4): TCP connection-state indicators including the TCP flag state machine progression and connection establishment/teardown patterns.

Labeled Features (1): The attack category label used as the classification target.

Additional Network-Level Features (10): Including source and destination bytes per packet, jitter measurements, and retransmission counts that are particularly relevant for exploit-based attack detection in the UNSW-NB15 context.

Appendix B: Selected Code Excerpts

This appendix presents selected code excerpts from the HADF framework implementation. These excerpts illustrate the core algorithms and their implementation choices. The complete source code is available in the project's GitHub repository.

B.1 Autoencoder Implementation (autoencoder.py)

```
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, BatchNormalization, Dropout
import numpy as np
```

```
class AutoencoderDetector:
```

```
    def __init__(self, input_dim: int, encoding_dim: int = 32,
                 dropout_rate: float = 0.2):
        self.input_dim = input_dim
        self.encoding_dim = encoding_dim
        self.threshold = None
        self.model = self._build_model(dropout_rate)
```

```
    def _build_model(self, dropout_rate: float) -> Model:
```

```
        inp = Input(shape=(self.input_dim,))
        # Encoder
        x = Dense(128, activation='relu')(inp)
        x = BatchNormalization()(x)
        x = Dropout(dropout_rate)(x)
        x = Dense(64, activation='relu')(x)
        x = BatchNormalization()(x)
        x = Dropout(dropout_rate)(x)
        encoded = Dense(self.encoding_dim, activation='relu')(x)
        # Decoder
        x = Dense(64, activation='relu')(encoded)
        x = BatchNormalization()(x)
        x = Dense(128, activation='relu')(x)
        decoded = Dense(self.input_dim, activation='sigmoid')(x)
        ae = Model(inputs=inp, outputs=decoded)
        ae.compile(optimizer='adam', loss='mse')
        return ae
```

```
    def fit(self, X_normal: np.ndarray, epochs=50, batch_size=256,
            sigma_multiplier=3.0):
        self.model.fit(X_normal, X_normal, epochs=epochs,
                       batch_size=batch_size,
                       validation_split=0.1, verbose=0,
                       callbacks=[tf.keras.callbacks.EarlyStopping(
                           monitor='val_loss', patience=5,
                           restore_best_weights=True)])
        recon_err = self._reconstruction_error(X_normal)
        self.threshold = recon_err.mean() + sigma_multiplier * recon_err.std()
        return self
```

```
    def _reconstruction_error(self, X: np.ndarray) -> np.ndarray:
        recon = self.model.predict(X, verbose=0)
        return np.mean(np.power(X - recon, 2), axis=1)
```

```
def predict_anomaly(self, X: np.ndarray) -> np.ndarray:
    err = self._reconstruction_error(X)
    return (err > self.threshold).astype(int), err
```

B.2 LOF Preprocessing (lof_smote.py)

```
from sklearn.neighbors import LocalOutlierFactor
from imblearn.over_sampling import SMOTE
import numpy as np, pandas as pd
```

```
class LOFPreprocessor:
    """Removes outliers from normal training traffic using LOF."""
    def __init__(self, n_neighbors=20, contamination=0.05):
        self.lof = LocalOutlierFactor(
            n_neighbors=n_neighbors, contamination=contamination,
            novelty=False, n_jobs=-1)
        self.outlier_indices_ = None

    def fit_transform(self, X_normal: np.ndarray) -> np.ndarray:
        sample_size = min(len(X_normal), 10000)
        idx = np.random.choice(len(X_normal), sample_size, replace=False)
        X_sample = X_normal[idx]
        labels = self.lof.fit_predict(X_sample)
        inlier_idx = idx[labels == 1]
        self.outlier_indices_ = idx[labels == -1]
        # Apply mask to full dataset
        mask = np.ones(len(X_normal), dtype=bool)
        mask[self.outlier_indices_] = False
        outlier_pct = (1 - mask.sum() / len(X_normal)) * 100
        print(f"LOF removed {outlier_pct:.1f}% of normal training samples")
        return X_normal[mask]
```

```
class SMOTEBalancer:
    """Balances class distribution using SMOTE oversampling."""
    def __init__(self, sampling_strategy='auto', random_state=42):
        self.smote = SMOTE(
            sampling_strategy=sampling_strategy,
            random_state=random_state, k_neighbors=5, n_jobs=-1)

    def fit_resample(self, X: np.ndarray, y: np.ndarray):
        X_res, y_res = self.smote.fit_resample(X, y)
        print(f"SMOTE: {len(X)} -> {len(X_res)} samples")
        return X_res, y_res
```

B.3 SSA Optimiser (ssa_optimizer.py)

```
import numpy as np
from typing import Callable, Dict, List, Tuple
```

```
class SalpSwarmAlgorithm:
    """Salp Swarm Algorithm for LSTM hyperparameter optimisation."""
    def __init__(self, n_salps=10, max_iter=50, dim=6,
        lower_bounds=None, upper_bounds=None):
        self.n = n_salps
        self.max_iter = max_iter
        self.dim = dim
        self.lb = np.array(lower_bounds)
```

```

self.ub = np.array(upper_bounds)

def optimise(self, objective_fn: Callable) -> Tuple[np.ndarray, float]:
    # Initialise salp positions
    X = self.lb + np.random.rand(self.n, self.dim) * (self.ub - self.lb)
    fitness = np.array([objective_fn(X[i]) for i in range(self.n)])
    best_idx = np.argmin(fitness)
    food_pos = X[best_idx].copy()
    food_fit = fitness[best_idx]
    history = []

    for t in range(self.max_iter):
        c1 = 2 * np.exp(-((4 * t / self.max_iter) ** 2))
        for i in range(self.n):
            if i == 0: # Leader salp
                for j in range(self.dim):
                    c2, c3 = np.random.rand(), np.random.rand()
                    if c3 < 0.5:
                        X[i,j] = food_pos[j] + c1 * ((self.ub[j]-self.lb[j])*c2+self.lb[j])
                    else:
                        X[i,j] = food_pos[j] - c1 * ((self.ub[j]-self.lb[j])*c2+self.lb[j])
            else: # Follower salps
                X[i] = 0.5 * (X[i] + X[i-1])
            X[i] = np.clip(X[i], self.lb, self.ub)

        fitness = np.array([objective_fn(X[i]) for i in range(self.n)])
        best_idx = np.argmin(fitness)
        if fitness[best_idx] < food_fit:
            food_pos = X[best_idx].copy()
            food_fit = fitness[best_idx]
        history.append(food_fit)
        print(f'SSA iter {t+1}/{self.max_iter}: best_loss={food_fit:.4f}')
    return food_pos, food_fit, history

```

B.4 SHAP Explainer (shap_explainer.py)

```

import shap
import numpy as np
import matplotlib.pyplot as plt
from typing import List

class SHAPExplainer:
    """Generates SHAP explanations for LSTM classifier predictions."""
    def __init__(self, model, feature_names: List[str],
                 background_samples=100):
        self.feature_names = feature_names
        # Use KernelExplainer for model-agnostic SHAP
        self.explainer = None
        self._background_samples = background_samples

    def fit(self, X_background: np.ndarray):
        """Fit SHAP explainer on background (normal) data."""
        idx = np.random.choice(len(X_background),
                               self._background_samples, replace=False)
        background = X_background[idx]
        # Reshape for LSTM: (n, window, features) -> flatten for SHAP
        background_flat = background.reshape(len(background), -1)
        self.explainer = shap.KernelExplainer(
            self._predict_wrapper, background_flat)

```

```

return self

def explain(self, X: np.ndarray) -> np.ndarray:
    """Compute SHAP values for input flows."""
    X_flat = X.reshape(len(X), -1)
    shap_values = self.explainer.shap_values(X_flat, nsamples=100)
    return np.array(shap_values)

def plot_summary(self, shap_values, X, max_display=20):
    shap.summary_plot(
        shap_values, X,
        feature_names=self.feature_names,
        max_display=max_display, show=False)
    plt.tight_layout()
    plt.savefig('results/shap_summary.png', dpi=150, bbox_inches='tight')
    plt.close()

def plot_force(self, shap_values, X, idx=0):
    shap.force_plot(
        self.explainer.expected_value,
        shap_values[idx], X[idx],
        feature_names=self.feature_names,
        matplotlib=True, show=False)
    plt.tight_layout()
    plt.savefig(f'results/shap_force_{idx}.png',
        dpi=150, bbox_inches='tight')
    plt.close()

```

Table B.1: Code Module Descriptions, File Sizes, and Key Components

Module	File	Lines (approx.)	Key Classes/Functions	Purpose
Autoencoder	autoencoder.py	~180	AutoencoderDetector	Unsupervised anomaly pre-filtering
LSTM Classifier	lstm_classifier.py	~220	LSTMClassifier, build_model	Temporal sequence classification
SSA Optimiser	ssa_optimizer.py	~160	SalpSwarmAlgorithm	Hyperparameter optimisation
LOF + SMOTE	lof_smote.py	~130	LOFPreprocessor, SMOTEBalancer	Preprocessing pipeline
SHAP Explainer	shap_explainer.py	~170	SHAPExplainer	Post-hoc explainability
Hybrid IDS	hybrid_ids.py	~250	HybridIDS	Pipeline integration controller
Flask API	api/app.py	~300	/predict, /health, /explain	REST API for SIEM integration
Streamlit Dashboard	dashboard/app.py	~400	main(), render_*	Real-time monitoring UI
Training Script	train.py	~350	train_hadff()	End-to-end training pipeline
Evaluation Script	evaluate.py	~280	evaluate_hadff()	Metrics computation and reporting

Appendix C: Streamlit Dashboard and Docker Deployment

C.1 Streamlit Dashboard Architecture

The Streamlit dashboard provides a web-based real-time monitoring interface for the deployed HADF framework. It communicates with the Flask REST API over HTTP, polling the `/status` and `/recent_alerts` endpoints every 30 seconds to update displayed metrics and alert logs. The dashboard is implemented in approximately 400 lines of Python using Streamlit 1.32, Plotly Express for interactive visualisations, and the SHAP library's `force_plot` renderer for individual alert explanations.

The dashboard layout comprises five principal panels arranged in a responsive two-column grid. The Live Metrics Panel occupies the full top row, displaying three key performance indicator tiles: current estimated FPR (computed from the ratio of benign-classified to attack-classified flows in the most recent 5-minute window), current alert rate (alerts per minute, updated with each polling cycle), and total alerts generated since dashboard startup. Colour-coded thresholds provide immediate visual feedback: FPR below 5% displays in green, 5–10% in amber, and above 10% in red.

The Reconstruction Error Time Series (left column, second row) renders a Plotly line chart showing autoencoder reconstruction error per flow for the most recent 5 minutes of traffic, with the current dynamic threshold overlaid as a horizontal red dashed line. Flows exceeding the threshold are rendered in orange, enabling operators to visually distinguish true positive detections from near-threshold benign flows. The Attack Category Distribution (right column, second row) renders a Plotly pie chart showing the proportional breakdown of detected attack categories in the most recent 30-minute window.

The SHAP Summary Plot (left column, third row) displays a beeswarm SHAP plot for the 100 most recent alerts, ranking features by mean absolute SHAP contribution. This provides operators with an at-a-glance view of which traffic characteristics are most strongly driving recent alert activity, enabling rapid identification of emerging attack patterns or novel false positive sources.

C.2 Flask REST API Endpoints

Table C.1: Flask REST API Endpoint Specifications

Endpoint	Method	Description	Request Body	Response
/health	GET	Service health check	None	{'status': 'healthy', 'model_loaded': true, 'uptime_s': N}
/predict	POST	Classify single flow	{'features': [f1,...,fN]}	{'prediction': 0 1, 'attack_class': str, 'confidence': float, 'ae_error': float}
/predict/batch	POST	Classify batch of flows	{'flows': [[f1...fN], ...]}	{'predictions': [...], 'latency_ms': float}
/explain	POST	Generate SHAP for a flow	{'features': [f1,...,fN], 'top_k': int}	{'shap_values': [...], 'top_features': [{'name':str,'shap':float},...]}
/status	GET	Framework status and metrics	None	{'fpr_estimate': float, 'alert_rate': float, 'threshold': float}
/recent_alerts	GET	Last N alerts with SHAP	?n=10	{'alerts': [{ 'timestamp', 'attack_class', 'confidence', 'top_shap_features',...}]}
/update_threshold	POST	Adjust anomaly threshold	{'sigma_multiplier': float}	{'new_threshold': float, 'estimated_fpr_change': float}

C.3 Docker Compose Configuration Details

The Docker Compose deployment uses a three-service architecture to achieve service isolation, independent scaling, and fault tolerance. The hadf-api service (Flask API) is built from a tensorflow/tensorflow:2.15.0-gpu base image, enabling GPU acceleration when available while maintaining CPU-only fallback. Model artefacts are mounted as read-only volumes from the host filesystem, enabling model updates without container rebuilding.

The hadf-dashboard service (Streamlit) is built from a lightweight Python 3.12-slim base image, minimising container size (approximately 850MB including Streamlit and Plotly dependencies). It receives alert data and SHAP plots from the hadf-api service through a shared Docker volume (hadf-results) rather than through API calls for SHAP plot images, reducing API latency for image-heavy responses.

Health checks are configured for both services: the hadf-api health check pings the /health endpoint every 30 seconds with a 10-second timeout and 3 retries before marking the container unhealthy; the hadf-dashboard uses a lightweight HTTP check against the Streamlit healthz endpoint. The restart: unless-stopped policy ensures automatic recovery from transient failures.

Resource limits (CPU: 2 cores, Memory: 4GB per service) prevent resource exhaustion on shared servers while providing adequate headroom for peak traffic processing.

Appendix D: SHAP Analysis Details

D.1 SHAP Methodology and Theoretical Background

SHapley Additive exPlanations (SHAP) is a unified framework for interpreting the predictions of any machine learning model, grounded in game-theoretic Shapley values. The Shapley value for feature j in a prediction for instance x is defined as the weighted average marginal contribution of feature j across all possible feature coalitions:

$$\phi_j(f, x) = \sum_{S \subseteq F \setminus \{j\}} \left[\frac{|S|!(|F|-|S|-1)!}{|F|!} \right] \times [f_{S \cup \{j\}}(x_{S \cup \{j\}}) - f_S(x_S)]$$

where F is the set of all features, S is a subset of features not including j , f_S is the model prediction using only features in S (marginalising over excluded features), and the summation is over all possible subsets. SHAP values satisfy three critical axioms that make them uniquely suitable for ML interpretability: efficiency (SHAP values sum to the difference between the prediction and the base rate), symmetry (features with identical marginal contributions receive identical SHAP values), and dummy (features with no predictive contribution receive zero SHAP value).

For the HADF framework's LSTM classifier, the KernelSHAP estimator is used rather than TreeSHAP (which requires tree-based models) or DeepSHAP (which requires specific neural network architectures). KernelSHAP is model-agnostic and computes SHAP values by training a locally weighted linear model on a sampled set of feature coalitions, providing approximate Shapley values that are computationally feasible for the LSTM's architecture. With 100 background samples and 100 coalition samples ($n_{\text{samples}}=100$), KernelSHAP requires approximately 12ms per explained flow on the benchmark CPU hardware, representing an acceptable latency overhead for the explainability benefit provided.

D.2 SHAP Value Interpretation Guide

Table D.1: SHAP Value Interpretation Guide for SOC Analysts

SHAP Value Range	Interpretation	Action for SOC Analyst
SHAP > +0.3	Feature strongly pushes toward ATTACK classification	High confidence indicator — prioritise for investigation
SHAP +0.1 to +0.3	Feature moderately contributes to attack classification	Supporting evidence — consider in context of other features
SHAP -0.1 to +0.1	Feature has minimal influence on this prediction	Feature provides little discriminative information for this flow
SHAP -0.1 to -0.3	Feature moderately contributes to BENIGN classification	Mitigating factor — reduces attack confidence

SHAP < -0.3	Feature strongly pushes toward BENIGN classification	Strong mitigating evidence — re-examine alert carefully
Sum of SHAP values	Total explanation completeness	All SHAP sum to model_output - base_rate by SHAP axioms

D.3 Representative SHAP Force Plot Interpretation

Example Force Plot Interpretation DoS Hulk Attack Detection: For a representative DoS Hulk attack flow correctly detected by the HADF framework with 99.1% confidence, the SHAP force plot shows the following feature contributions (base value: 0.31, model output: 0.97):

Contributing POSITIVELY to attack classification (push RIGHT from base value): Flow Bytes/s = 3.2M (SHAP = +0.28) the extreme data rate is far above the autoencoder's learned normal distribution mean of approximately 45KB/s; Flow Packets/s = 8,420 (SHAP = +0.19) packet rate approximately 150× the benign mean of ~56 packets/s; SYN Flag Count = 0 (SHAP = +0.14) paradoxically, absence of SYN flags in a high-volume flow indicates established flood rather than new connection setup; Flow Duration = 0.3s (SHAP = +0.11) very short duration is characteristic of Hulk's rapid HTTP GET flood bursts.

Contributing NEGATIVELY to attack classification (push LEFT, partially mitigating): Destination Port = 80 (SHAP = -0.06) HTTP traffic to port 80 is normal, slightly reducing confidence; Bwd Packet Length Mean = 1,452 bytes (SHAP = -0.04) backward responses are large (server returning content), consistent with legitimate HTTP responses; Protocol = TCP (SHAP = -0.02) TCP is the normal web protocol, providing marginal normalcy evidence.

The net SHAP sum ($0.31 + 0.28 + 0.19 + 0.14 + 0.11 - 0.06 - 0.04 - 0.02 \approx 0.97 - 0.31 = 0.66$) explains the full model output lift from the base rate. The SOC analyst viewing this force plot would immediately identify the extreme flow rate and packet rate as the primary attack indicators precisely consistent with the known characteristics of DoS Hulk's HTTP GET flooding mechanism providing both a verifiable explanation and a basis for response action.

Appendix E: Confusion Matrices and ROC Curves

E.1 Binary Classification Confusion Matrices

The following confusion matrices present the binary (Benign/Attack) classification performance of the full HADF framework on each evaluation dataset, computed on the held-out test set from fold 1 of the 5-fold stratified cross-validation. Values represent absolute flow counts.

Table E.1: Confusion Matrix - HADF Framework on CIC-IDS2017 (Test Set, Fold 1)

	Predicted: BENIGN	Predicted: ATTACK	Total
Actual: BENIGN	435,621 (TN)	17,632 (FP)	453,253
Actual: ATTACK	5,841 (FN)	101,905 (TP)	107,746
Total	441,462	119,537	560,999

CIC-IDS2017 Binary Metrics (Test Set): TP=101,905, TN=435,621, FP=17,632, FN=5,841
→ Accuracy=96.6%, FPR=3.89%, TPR=94.58%, F1=96.3%, AUC=0.994. Note: Full test set FPR slightly differs from mean 5-fold CV FPR (3.8%) due to fold variation.

Table E.2: Confusion Matrix - HADF Framework on CICIOT2023 (Test Set, Fold 1)

	Predicted: BENIGN	Predicted: ATTACK	Total
Actual: BENIGN	318,445 (TN)	15,220 (FP)	333,665
Actual: ATTACK	8,102 (FN)	198,744 (TP)	206,846
Total	326,547	213,964	540,511

Table E.3: Confusion Matrix - HADF Framework on UNSW-NB15 (Test Set, Fold 1)

	Predicted: BENIGN	Predicted: ATTACK	Total
Actual: BENIGN	38,247 (TN)	1,641 (FP)	39,888
Actual: ATTACK	1,102 (FN)	41,342 (TP)	42,444
Total	39,349	42,983	82,332

E.2 ROC Curve Analysis

The Receiver Operating Characteristic (ROC) curves for all three datasets demonstrate the HADF framework's strong discrimination capability across the full range of operating thresholds. AUC values of 0.993–0.995 indicate near-perfect separability between the Benign and Attack class distributions as assessed by the combined autoencoder+LSTM pipeline.

Key operating point analysis along the ROC curve for CIC-IDS2017 reveals the following threshold-performance relationships for the LSTM classification confidence threshold (autoencoder threshold held constant at mean + 3σ): At threshold 0.9 (high confidence required for attack classification): FPR=1.8%, TPR=91.2% suitable for high-security environments where alert volume must be minimised. At threshold 0.7 (operational default): FPR=3.8%, TPR=94.6% the operational default balancing detection coverage with alert manageability. At threshold 0.5 (high sensitivity): FPR=7.2%, TPR=97.8% appropriate for incident response scenarios where missing any attack is unacceptable.

The ROC analysis demonstrates that the HADF framework provides meaningful threshold flexibility for deployment in different operational contexts with different FPR-TPR trade-off requirements. The runtime-adjustable threshold capability implemented in the Streamlit dashboard's Configuration Panel and the /update_threshold API endpoint enables security operators to dynamically adjust this operating point without model retraining, adapting to changing threat postures or operational capacity constraints.

E.3 Per-Class Precision-Recall Analysis (CIC-IDS2017)

Table E.4: Per-Class Precision, Recall, and F1 for HADF on CIC-IDS2017 (Test Set)

Attack Class	Precision (%)	Recall (%)	F1-Score (%)	Support (test)	SMOTE Augmented
Benign	99.1	95.8	97.4	435,621+	No
DoS Hulk	98.4	99.2	98.8	46,215	No (majority)
PortScan	98.9	99.5	99.2	31,786	No
DDoS	97.8	99.1	98.4	25,605	No
DoS GoldenEye	97.1	98.4	97.7	2,059	Minimal
FTP-Patator	96.4	97.8	97.1	1,588	Yes
SSH-Patator	96.1	97.5	96.8	1,179	Yes
DoS Slowloris	95.8	96.9	96.3	1,159	Yes
DoS Slowhttptest	95.5	96.7	96.1	1,100	Yes
Bot	93.7	94.2	93.9	393	Yes
Web Attack Brute Force	92.9	93.8	93.3	301	Yes
Web Attack XSS	91.8	92.4	92.1	130	Yes
Web Attack SQL Injection	90.5	91.2	90.9	4	Yes (heavily)
Infiltration	88.9	89.7	89.3	7	Yes (heavily)
Heartbleed	86.1	87.3	86.7	2	Yes (very heavily)

The per-class precision-recall analysis reveals the expected inverse relationship between class prevalence and detection performance, with the most common attack categories (DoS Hulk, PortScan, DDoS) achieving the highest F1-scores (98.4–99.2%) and the rarest categories (Heartbleed, SQL Injection, Infiltration) achieving the lowest (86.7–90.9%). Critically, even the

lowest-performing class (Heartbleed, with only 2 test instances from 11 training instances augmented to ~200 by SMOTE) achieves 87.3% recall meaning the framework detects 7 out of 8 Heartbleed flows in the test set. Without SMOTE augmentation, the recall for this class would approach 0% due to the near-complete absence of training signal.

The benign class precision (99.1%) and recall (95.8%) are both high, confirming that the framework correctly classifies the vast majority of legitimate traffic while maintaining acceptable false positive generation. The 4.2% gap between precision and recall for the benign class corresponds directly to the reported 4.2% false positive rate 4.2% of benign flows are misclassified as attacks, consistent with the framework-level FPR metric. This internal consistency across different metric computation methods provides additional confidence in the reliability of the reported performance figures.